# Vijeo Citect v7.40

## CitectVBA Reference Guide

August 2013

Schneider Electric

# Legal Information

## DISCLAIMER

Schneider Electric makes no representations or warranties with respect to this manual and, to the maximum extent permitted by law, expressly limits its liability for breach of any warranty that may be implied to the replacement of this manual with another. Further, Schneider Electric reserves the right to revise this publication at any time without incurring an obligation to notify any person of the revision.

The Example Projects are provided to you for the purpose of illustrating how the SCADA software v7.40 could be used in an operational environment ("the Purpose").Schneider Electric grants you a royalty free, non exclusive, non transferable license to use the example projects installed with your SCADA software version v7.40 (“the Example Projects”) for the Purpose only.

The Example Projects are provided by Schneider Electric as part of the SCADA software version v7.40 on an "as is" basis and Schneider Electric does not guarantee the reliability, serviceability or function of the Example Projects.

Should you modify the Example Projects, you bear the risk of any use of such modified Example Projects.

Schneider Electric gives no express warranties, guarantees or conditions and to the extent permitted under applicable laws, Schneider Electric disclaims all implied warranties, including any implied warranties of merchantability, fitness for a particular purpose or non-infringement of third parties’ intellectual property rights.

Schneider Electric shall not be liable for any direct, indirect or consequential damages or costs of any type arising out of any action taken by you or others related to the Example Projects.

## COPYRIGHT

© Copyright 2012 Schneider Electric All rights reserved.

## TRADEMARKS

Schneider Electric has made every effort to supply trademark information about company names, products and services mentioned in this manual.

Citect, CitectHMI, Vijeo Citect, Vijeo Citect Lite PowerSCADA Expert and CitectSCADA are either registered trademarks or trademarks of Schneider Electric.

Pelco, Spectra, Sarix, Endura, are registered trademarks of Pelco, Inc.

IBM, IBM PC and IBM PC AT are registered trademarks of International Business Machines Corporation.

MS-DOS, Windows, Windows NT, Microsoft, and Excel are either registered trademarks or trademarks of Microsoft Corporation in the United States and/or other countries.

DigiBoard, PC/Xi and Com/Xi are trademarks of Digi International Inc.

Novell, Netware and Netware Lite are either registered trademarks or trademarks of Novell, Inc. in the United States and other countries.

dBASE is a trademark of dataBased Intelligence, Inc.

All other brands and products referenced in this document are acknowledged to be the trademarks or registered trademarks of their respective holders.

## GENERAL INFORMATION

Some product names used in this manual are used for identification purposes only and may be trademarks of their respective companies.

August 2013 edition for Vijeo Citect Version v7.40.

Manual Revision Version v7.40.

## PLEASE NOTE

Electrical equipment should be installed, operated, serviced, and maintained only by qualified personnel. No responsibility is assumed by Schneider Electric for any consequences arising out of the use of this material. © 2012 Schneider Electric. All Rights Reserved.

## Validity Note

The present documentation is intended for qualified technical personnel responsible for the implementation, operation and maintenance of the products described. It contains information necessary for the proper use of the products. However, those who wish to make a more "advanced" use of our products may find it necessary to consult our nearest distributor in order to obtain additional information.

**The contents of this documentation are not contractual and in no way constitute an extension to, or restriction of, the contractual warranty clauses.**

**Contact Schneider Electric today at www.schneider-electric.com**

# Contents


Legal Information ..... 1

Contents ..... 3

Safety Information ..... 7

Chapter 1: Introducing CitectVBA ..... 9

Chapter 2: Integrating CitectVBA into your Project ..... 11
- Accessing Cicode Tags with CitectVBA ..... 11
- Using CitectVBA in Command or Expression fields ..... 12
- Accessing ActiveX Objects with CitectVBA ..... 13
- Multithread Considerations with CitectVBA ..... 14
- Calling CitectVBA from Cicode ..... 15
- Calling Cicode from CitectVBA ..... 17

Chapter 3: Using the CitectVBA Test Project ..... 21
- Creating the Test Project ..... 21
- Opening the Test Project ..... 22
- Setting up Test Project Communications ..... 22
- Setting up the Test Project Computer ..... 23
- Adding a Variable Tag ..... 23
- Adding a Graphics Page ..... 24
- Saving Your Graphics Page ..... 24
- Opening the Graphics Page ..... 25

Chapter 4: Understanding CitectVBA Language Basics ..... 27
- CitectVBA Files ..... 27
- Cicode Editor ..... 27

Scope of CitectVBA ..... 28
Procedural (local) level scope ..... 28
Modular level scope ..... 29
Global level scope ..... 29
CitectVBA Statements ..... 30
Comments ..... 30
Header information ..... 31
Labels ..... 32
CitectVBA Line Continuation Character ..... 32
Naming ..... 33
Option Statements ..... 34
Option Explicit statement ..... 34
Option Compare statement ..... 35
Option Base statement ..... 35
CitectVBA Data Types ..... 36
Constants ..... 37
Declaration of constants ..... 38
Intrinsic constants ..... 39
Variables ..... 40
Variable declaration ..... 40
Variable initialization values ..... 42
Arrays of Variables ..... 43
Variant Declaration ..... 48
Numbers ..... 50
Numeric Data Types ..... 50
Exponential Notation ..... 51
Floating Point Calculation Rules ..... 52
Rounding Numbers ..... 53
Date and Time Handling ..... 54
Date Constants ..... 55
Formatting Date Values ..... 57
Date and Time Data Constraints ..... 59
Date Data Type Structure ..... 60
Date-values ..... 61
Time-values ..... 61
Dates in Databases Using Different Calendars ..... 62
Operators ..... 63
Assignment Operator ..... 63
Arithmetical (Math) Operators ..... 64
Relational Operators ..... 65
Logical Operators ..... 65
Operator Precedence ..... 66
Strings ..... 67
String Comparisons ..... 68
String Concatenation ..... 68
Control Structures ..... 69
GoTo statement ..... 70
Do statement ..... 70
While statement ..... 71

For statement ..... 71
If statement ..... 72
Select case statement ..... 74
End statement ..... 76
Exit statement ..... 76
OnError statement ..... 77
Stop statement ..... 77
With statement ..... 78
Subroutines and Functions ..... 78
Subroutines ..... 79
Functions ..... 80
Arguments ..... 82
DLLs and APIs ..... 84
Accessing Functions in DLLs ..... 85
Passing Arguments to DLL Functions from CitectVBA ..... 89
OLE Services ..... 90
OLE terminology ..... 90
OLE automation objects ..... 91
Declaration of OLE automation objects ..... 92
Assigning references to OLE automation objects ..... 93
Using OLE automation objects ..... 94
Accessing the object model of OLE automation server applications ..... 95
Understanding object models in OLE automation ..... 96
Using the Microsoft Word object model ..... 99
OLE automation example using the Microsoft Word object ..... 99
Using the Microsoft Excel object model ..... 100
Deleting OLE automation objects ..... 100
File Input/Output with CitectVBA ..... 101

**Chapter 5: CitectVBA Function Reference ..... 103**

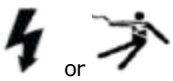
Array Functions ..... 103
Conditional Statements ..... 108
109
Conversion Functions ..... 117
ASCII character code conversion ..... 117
Date conversion ..... 119
Date and time formatting/conversion ..... 123
Number and string conversion ..... 124
Declarations ..... 135
Date and Time Functions ..... 148
DateValue ..... 152
File I/O Functions ..... 160
Math/Trigonometry Functions ..... 194
Numeric functions ..... 194
Trigonometric functions ..... 200
Miscellaneous Functions ..... 203
Procedural Statements ..... 205
String Functions ..... 217

**Chapter 6: ASCII/ANSI Character Code Listings ..... 233**

# Safety Information

**Hazard categories and special symbols**

The following symbols and special messages may appear in this manual or on the product to warn of potential hazards or to call attention to information that clarifies or simplifies a procedure.

| Symbol | Description |
|---|---|
|  or | The addition of either symbol to a "Danger" or "Warning" safety label indicates that an electrical hazard exists which will result in personal injury if the instructions are not followed. |
|  | This is the safety alert symbol. It is used to alert you to personal injury hazards. Obey all safety messages that follow this symbol to avoid possible injury or death. |

**⚠ DANGER**

**DANGER** indicates an imminently hazardous situation, which, if not avoided, will result in death or serious injury.

**⚠ WARNING**

**WARNING** indicates a potentially hazardous situation, which, if not avoided, can result in death or serious injury.

**⚠ CAUTION**

**CAUTION** indicates a potentially hazardous situation which, if not avoided, can result in minor or moderate injury.

***NOTICE***

***NOTICE*** used without a safety alert symbol, indicates a potentially hazardous situation which, if not avoided, can result in property or equipment damage.

**Please Note**

Electrical equipment should be installed, operated, serviced, and maintained only by qualified personnel. No responsibility is assumed by Schneider Electric for any consequences arising out of the use of this material.

## Before You Begin

Vijeo Citect is a Supervisory Control and Data Acquisition (SCADA) solution. It facilitates the creation of software to manage and monitor industrial systems and processes. Due to Vijeo Citect's central role in controlling systems and processes, you must appropriately design, commission, and test your Vijeo Citect project before implementing it in an operational setting. Observe the following:

**⚠ WARNING**

**UNINTENDED EQUIPMENT OPERATION**

Do not use Vijeo Citect or other SCADA software as a replacement for PLC-based control programs. SCADA software is not designed for direct, high-speed system control.

**Failure to follow these instructions can result in death, serious injury, or equipment damage.**

**⚠ WARNING**

**LOSS OF CONTROL**

- The designer of any control scheme must consider the potential failure modes of control paths and, for certain critical control functions, provide a means to achieve a safe state during and after a path failure. Examples of critical control functions are emergency stop and overtravel stop, power outage and restart.
- Separate or redundant control paths must be provided for critical control functions.
- System control paths may include communication links. Consideration must be given to the implications of unanticipated transmission delays or failures of the link.
- Observe all accident prevention regulations and local safety guidelines. [1]
- Each implementation of a control system created using Vijeo Citect must be individually and thoroughly tested for proper operation before being placed into service.

**Failure to follow these instructions can result in death, serious injury, or equipment damage.**

1. For additional information, refer to NEMA ICS 1.1 (latest edition) "Safety Guidelines for the Application, Installation, and Maintenance of Solid State Control", and to NEMA ICS 7.1 (latest edition) "Safety Standards for Construction and Guide for Selection, Installation and Operation of Adjustable-Speed Drive Systems" or their equivalent governing your particular location.

# Chapter 1: Introducing CitectVBA

CitectVBA is a Visual Basic for Applications (VBA) and VBScript-compatible Basic scripting language. Vijeo Citect has embedded support for CitectVBA.

CitectVBA has the following features:

- CitectVBA code is multithreaded and fully scheduled within the Vijeo Citect Kernel.
- CitectVBA uses the same well proven engine that Cicode uses and can be used wherever Cicode is used.
- CitectVBA has a small footprint of under 400K.
- CitectVBA code is directly callable from Vijeo Citect Command and Expression fields.
- CitectVBA code is callable from Cicode and visa-versa.
- CitectVBA code provides native support for ActiveX objects, Vijeo Citect Variable Tags and Alarm Tags.
- CitectVBA makes ActiveX object manipulating easier. It allows direct interaction with the object models from 3rd party applications such as Word, Excel, etc.

> **Note:** You may notice slight differences between CitectVBA and VBA in other applications; this is normal as each application has a different object model.

The Cicode Editor has been upgraded to fully support CitectVBA. New features of the editor include:

- Integrated Cicode and CitectVBA compiler
- Integrated Cicode and CitectVBA source code editor
- Integrated Cicode and CitectVBA debugger

**See Also**

Integrating CitectVBA with Vijeo Citect

# Chapter 2: Integrating CitectVBA into your Project

You can integrate CitectVBA into your Vijeo Citect project in two ways:

- Use CitectVBA code script directly in your Command or Expression fields within Vijeo Citect.
- Store user-defined CitectVBA script in a separate CitectVBA file.

In either case, all procedures within a CitectVBA script can access (read and write) any Vijeo Citect variable tag in the same way as Cicode can access Vijeo Citect tags.

**See Also**

Accessing Cicode Tags with CitectVBA

## Accessing Cicode Tags with CitectVBA

CitectVBA can use your Vijeo Citect project variable tag and alarm tag variables in the same way as could Cicode (except for syntax differences). Both programming languages refer to a project's variable tags by using the name of the tags as defined in the project.

> **Note:** Project variable tags are defined (in Vijeo Citect) by using the Variable Tags form in the Citect Project Editor. For details, see Adding a Variable Tag.

For instance, in the following example, two variable tags in your Vijeo Citect project may be named **B1_PUMP_101_SP** and **B1_PUMP_101_PV** respectively, representing the Set Point and Process Variable values of Pump 101. These variable tag names can be used within a CitectVBA statement (just as you would use any other variable in CitectVBA). Both values can be read-from and written-to directly using CitectVBA:

```
' set pump speed to 500 rpm
B1_PUMP_101_SP = 500
' calculate pump speed error
Dim varPumpSpeedError
varPumpSpeedError = B1_PUMP_101_PV - B1_PUMP_101_SP
```

You should note that CitectVBA does not recognize Vijeo Citect variable tags that are named with an initial digit (0-9).

To access such tags using CitectVBA, you must precede the tag name with a case-insensitive string containing the letters 'V', 'B,' and the underscore character (VB_) as in the following example:

```
Vijeo Citect Tag Name:    "123Pump"
CitectVBA reference     "VB_123Pump"
```

For details about accessing ActiveX objects using CitectVBA, see Accessing ActiveX Objects with CitectVBA. For details of using tags that have a number as their first digit in your Vijeo Citect project, consider using the `[General]TagStartDigit Citect.INI` parameter.

**See Also**

Using CitectVBA in Vijeo Citect Command or Expression fields>Calling CitectVBA from Cicode

Calling CitectVBA from Cicode

## Using CitectVBA in Command or Expression fields

Vijeo Citect expects that all code contained within a Vijeo Citect Command or Expression field to be Cicode by default. When using CitectVBA code script directly in a Vijeo Citect Command or Expression field within Vijeo Citect, you must precede the CitectVBA script with the keyword **CiVBA**, as shown here:

```
CiVBA
TestTag_1 = TestTag_1 + 1
```

This is known as the language override command. When the Vijeo Citect compiler reads the keyword CiVBA, it knows to handle that code (within the same Vijeo Citect Command or Expression field) as CitectVBA script, and compiles it as such. No such override command is required to use Cicode.

The CiVBA language override statement must be placed first in the Vijeo Citect Command or Expression field if you want to use CitectVBA script code instead of Cicode in that Vijeo Citect Command or Expression field.

**Note:** You must use either Cicode or CitectVBA in a Vijeo Citect Command or Expression field. You cannot change or swap between the two programming languages (within the same Vijeo Citect Command or Expression field) once you've started using one or the other.

You can, however, call a single Cicode function from within CitectVBA script if you wrap the Cicode call within special CitectVBA functions `CicodeCallOpen()` and `CicodeCallReturn()`. For details, see Calling Cicode from CitectVBA.

Alternatively, to call a single CitectVBA function (from within the Vijeo Citect Command or Expression field) after you have already used Cicode in that field, you can wrap the CitectVBA within three nested special Cicode functions: `VbCallOpen()`, `VbCallRun()` and `VbCallReturn()`. See Calling Cicode from CitectVBA.

**See Also**

Accessing Cicode Tags with CitectVBA

Accessing ActiveX Objects with CitectVBA

Multithread Considerations with CitectVBA

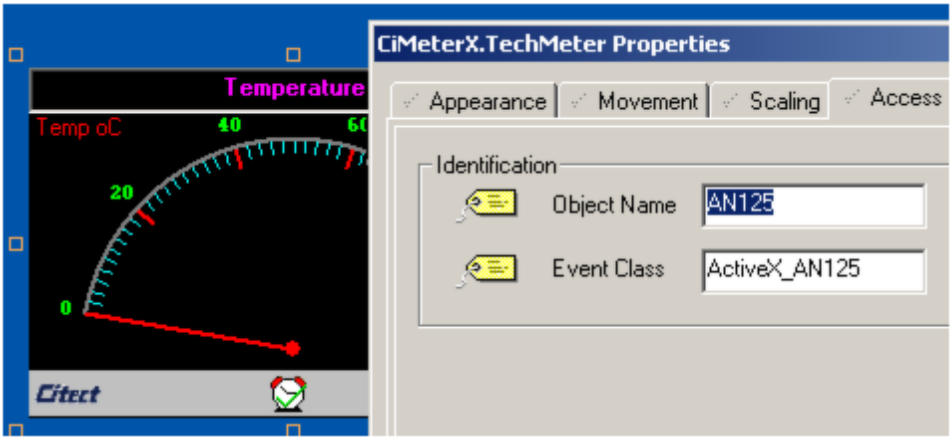
Calling CitectVBA from Cicode

Calling Cicode from CitectVBA

## Accessing ActiveX Objects with CitectVBA

ActiveX objects which have been added to a graphics page in your Vijeo Citect project can be referred to in CitectVBA by constructing a unique reference name using the page name, the underscore character, the letters 'AN', and the animation number of the object.

This reference name is called the Event Class name in Vijeo Citect. To view the reference name, double-click the ActiveX object, select the **Access** tab, then click the **Identification** tab.




In this example, the reference name for the Temperature meter object would be referred to in CitectVBA as ActiveX_AN125.

All object properties can be accessed and manipulated using CitectVBA in the same way that object properties can be manipulated using Cicode.

**See Also**

Accessing Cicode Tags with CitectVBA

Using CitectVBA in Vijeo Citect Command or Expression fields

## Multithread Considerations with CitectVBA

Cicode is pre-empted and executed on an instruction-by-instruction basis. This means that execution of a simple unnested Cicode thread can only switch to another thread after the current Cicode instruction has completed execution.

CitectVBA code is pre-empted and executed on a line-by-line basis (as opposed to an instruction-by-instruction basis), and pre-empting can only occur after the current line has completed execution.

Each line of CitectVBA script is handled as a separate thread in Vijeo Citect. Therefore multiple procedures placed on one line may not complete before another subsequent thread is processed in a multithreading environment. This could cause unpredictable results and consequences, including data invalidation and corruption.

**⚠ WARNING**

**UNINTENDED EQUIPMENT OPERATION**

- Create your CitectVBA program so that every code statement is positioned on a unique line.
- Do not group more than one code statement on a single line in your program. Grouping CitectVBA statements on a single line can cause data corruption during multithreaded execution.

**Failure to follow these instructions can result in death, serious injury, or equipment damage.**

If, for example, you were reading or setting some variable or point in a multi-statement thread, and further processing that data in a later thread,that data might become invalid or incorrect. For this reason, you should separate every statement onto separate lines in CitectVBA.

For example, it is better to write:

```
A = Motor1.speed() + Motor4.speed() + Motor5.speed()
```

as

```
A = Motor1.speed()
A = A + Motor4.speed()
A = A + Motor5.speed()
```

in situations where the method `speed()` may take a long time to execute.

In the first example above, the CitectVBA thread executes for three times longer before it can be pre-empted than in the latter example.

> **Note:** This does not apply to Cicode because the Cicode engine can pre-empt aggregated code.

**See Also**

Accessing Cicode Tags with CitectVBA

Using CitectVBA in Vijeo Citect Command or Expression fields

Calling CitectVBA from Cicode

Calling Cicode from CitectVBA

## Calling CitectVBA from Cicode

Three new Cicode functions allow CitectVBA code to be called from within Cicode script, and be pre-emptively multitasked by Vijeo Citect. These calls `VbCallOpen()`, `VbCallRun()`, and `VbCallReturn()` can be nested to implement the entire function set with a single line of Cicode.

> **Note:** When using the CiVBA language override in a Command field, the compiler constructs the nested call for you. The same mechanism is used even though it is not self evident. For details, see Using CitectVBA in Vijeo Citect Command or Expression fields.

For information on multithreading in CitectVBA, see Multithread Considerations with CitectVBA.

To call a given CitectVBA function or subroutine from Cicode, use the `VbCallOpen` function. This returns a handle which can then be used to execute the call by passing it to the `VbCallRun` function. Upon return from `VbCallRun`, you can call the `VbCallReturn` function to get the return value of the CitectVBA function called.

The Cicode `VbCallOpen()` function is used to initiate a call to the CitectVBA function or subroutine, and returns a handle to the open function.

```
<ReturnValue> = VbCallOpen(<FunctName>, <ArgList>)
```

where:

- <ReturnValue> represents the handle to the opened function.
- <FunctName> represents the name of the CitectVBA function or subroutine being called.
- <ArgList> represents a comma separated list of arguments to pass to the opened CitectVBA function or subroutine named in <FunctName>.

The Cicode VbCallRun() function is used to execute the CitectVBA function or subroutine (previously opened with the Cicode VbCallOpen function), and requires the handle returned from the VbCallOpen function call. The VbCallRun function provides an opportunity for the opened CitectVBA function to complete and return a value in the multi-threaded Vijeo Citect environment. It passes its argument value (of OBJECT data type) through as its return value upon completion.

```
<ReturnValue> = VbCallRun(<CallHandle>)
```

where:

- <ReturnValue> represents the handle to the opened CitectVBA function passed through for the <CallHandle> argument.
- <CallHandle> represents the handle to the previously opened CitectVBA function as returned by the Cicode VbCallOpen function.

The Cicode VbCallReturn() function is used to obtain the return value of the completed CitectVBA function (previously opened with the Cicode VbCallOpen function), and requires the handle returned from the VbCallRun function call.

```
<ReturnValue> = VbCallReturn(<CallHandle>)
```

where:

- <ReturnValue> represents the value returned by the completed CitectVBA function (which was previously opened by the Cicode VbCallOpen function). The data type of the return value is dependent upon the data type of the return value for the CitectVBA function opened.
- <CallHandle> represents the handle to the previously opened CitectVBA function as returned by the Cicode VbCallRun function.

**Example**

```
FUNCTION
TestCitectVBA()
INT iRet;
STRING sMsg = "Hello";
INT iVal = 123;
iRet = VbCallReturn(VbCallRun(VbCallOpen("CiVBATest", iVal)));
Message("TestCitectVBA Function", "CiVBATest = " +
```

```
IntToStr(iRet), 0);
END
```

**Example**

```
Function CiVBATest(Value As Integer) As Integer
CiVBATest = Value * 2
End Function
```

**See Also**

Calling Cicode from CitectVBA

## Calling Cicode from CitectVBA

Calling a Cicode function from CitectVBA is accomplished by two CitectVBA functions: `CicodeCallOpen()` and `CicodeCallReturn()`.

To call a given Cicode function, use the `CicodeCallOpen` function which will create and execute a Cicode thread that runs the function. For multitasking purposes, a separate function `CicodeCallReturn` is used to obtain the return-value of the completed Cicode function most recently called by the `CicodeCallOpen` function.

**⚠ WARNING**

**UNINTENDED EQUIPMENT OPERATION**

Do not nest the CicodeCallOpen and CicodeCallReturn functions. Nesting these functions can lead to unintended equipment operation when your program is run.

**Failure to follow these instructions can result in death, serious injury, or equipment damage.**

The return value is initialized when control is returned to the Vijeo Citect kernel. This occurs only after completion of the line of CitectVBA code containing `CicodeCallOpen`. For details on multithreading in CitectVBA, see Multithread Considerations with CitectVBA.

To call a given Cicode function or subroutine from CitectVBA, use the `CicodeCallOpen` function. Upon return from `CicodeCallOpen`, you can call the `CicodeCallReturn` function to obtain the return value of the Cicode function called.

The `CicodeCallOpen` function is a CitectVBA function used to call a Cicode function from CitectVBA. It is used to initiate and execute a call to the Cicode function and returns an integer value representing the success or the type of error encountered by the CicodeCallOpen function.

```
<ReturnValue> = CicodeCallOpen(<FunctName>, <ArgList>)
```

where:

- `<ReturnValue>` represents the return value of:
  - 0 if `CicodeCallOpen`function was successful
  - 1 for `CicodeCallOpen`function general error
  - 2 for specified Cicode function not found
  - 3 for incorrect number of arguments for specified Cicode function passed in `<ArgList>`.
- `<FunctName>` is a string representing the name of the Cicode function being called. The function name should be enclosed in double quotes.
- `<ArgList>` represents a variable length comma separated argument list of all the arguments to be passed to the Cicode function being opened (dependant upon which Cicode function is being called and the arguments that Cicode function requires). The argument list should not be enclosed within brackets, although when using variable names as arguments, those variable arguments within the list need to be individually enclosed within brackets to force the passing of the variable to Cicode by value.

The `CicodeCallReturn`function is a CitectVBA function used to obtain the return value of the most recently completed Cicode function opened with the CitectVBA `CicodeCallOpen`function.

```
<ReturnValue> = CicodeCallReturn()
```

where:

- `<ReturnValue>` represents the return value of the Cicode function specified in the most recent call of the `CicodeCallOpen`function. Note that the return data type of `CicodeCallReturn`will depend upon the return data type of the Cicode function called in the most recent call of the `CicodeCallOpen`function.

No arguments are passed to the `CicodeCallReturn`function, as it can only return the result of the most recent return-value for the Cicode function called by the CitectVBA `CicodeCallOpen`function.

**Note:**In the following example, a CitectVBA variable is enclosed in brackets to force the passing of the variable by value. See Passing variables Byref and Byval.

**CitectVBA**

```
' declare modular variant variable to store function results
Dim vntRet as Variant
Function TestCicode() As Integer
```

```
' declare local variables
Dim intRet As Integer
Dim strReply as String
Dim intMaxScale as Integer
' copy current tag value to variable
' uses the project variable tag named MAX_SCALE
intMaxScale = MAX_SCALE
' call Cicode function
' for example: TrnSetScale( AN, Pen, Percent, Scale)
intRet = CicodeCallOpen(  "TrnSetScale", 53, -1, 100, (IntMaxScale) )
' Note the syntax used:
'- brackets around the CitectVBA function argument list
'(Only necessary when the CitectVBA function is preceded by an equals (=) sign .)
'     - double quotes around the Cicode function name
'- no brackets around the Cicode function argument list
'- brackets around individual variable arguments
' test results
If  intRet = 0 Then
'
' insert code for successful completion here
'
vntRet = CicodeCallReturn()
strReply = "CicodeCallOpen Function successfully called"
Else
'
' insert code for unsuccessful completion here
'
Select Case intRet
Case = 1
' assign return comment for this case
strReply = "CicodeCallOpen Function call general error"

Case = 2
' assign return comment for this case
strReply = "Cicode Function not found"
Case = 3
' assign return comment for this case
strReply = "Wrong number of arguments "_
& "in Cicode CallOpen function call"
Case Else
' assign return comment for this case
strReply = "Unknown error"
End Select
End If
' display return comment for your information
MsgBox strReply
' assign return value for this function
TestCicode = intRet
End Function
```

**See Also**

Calling CitectVBA from Cicode

# Chapter 3: Using the CitectVBA Test Project

You can use the CitectVBA test project to help you learn the basics programming with CitectVBA. Before you can use the project, you must:

- Create the test project
- Open the test project
- Set up test project communications
- Set up the test project computer
- Add a variable tag to the project
- Set up a graphics page to the project

## Creating the Test Project

Creating the project requires you to specify the destination folder for the test project so that Vijeo Citect can find it.

### To create the CitectVBA test project

1. Start Vijeo Citect (if not already started).
2. Click the **Citect Explorer** button.
3. Choose **New Project** from the **File**menu, or click the **New Project** button.
4. In the Name field, type **CitectVBA Test**.
5. In the **Location** field, check that the path is displaying the project name as a sub-folder name beneath the **User** folder. The User folder is the parent folder where Vijeo Citect expects all projects to be stored in separate subfolders.

   > **Note:** The name of the project is appended as a sub folder name to the **User** folder. Your path will depend upon the actual path of your Vijeo Citect installation.

6. Click **OK**. The test project has been created.

### See Also

Opening the Test Project

## Opening the Test Project

You open the test project from Citect Explorer.

### To open the CitectVBA test project

1. Start Vijeo Citect (if not already started).
2. Click the **Citect Explorer** button.
3. From the **Project List**, click on the folder named **CitectVBA Test**.

> **Note:** Vijeo Citect stores the most recently opened project name in the Citect.ini file, so that the next time Vijeo Citect is started, that project opens automatically ready for further editing.

### See Also

Setting up Test Project Communications

## Setting up Test Project Communications

You only need to perform this procedure once for the test project.

### To set up test project communications

1. Open the test project.
2. Click the **Citect Explorer** button.
3. Double-click the **Communications** folder.
4. Double-click the **Express I/O Device Setup** button.
5. Select **Next** and click the **Create a New I/O Server** check box.
6. In the **Name** field, replace the default name IOServer_1 with **CiVBAIOServer**.

> **Note:**Vijeo Citect stores the communication details as records in a database. Each record name is limited to a maximum of 16 characters. These records are accessible with the Citect Project Editor.

7. Select **Next** and click the **Create a New I/O Device** check box.
8. In the **Name** field, replace the default name **IODev** with **CiVBAIODevice**.
9. Click **Next** and click the **Disk I/O Device** check box.
10. Click **Next** to accept the default**Vijeo Citect Generic Protocol**.

11. Click**Next** to accept the default remaining unlinked to any external tag database.
12. Select **Next** and **Finish** to create the CitectVBA Project communications.

**See Also**

Setting up the Test Project Computer

## Setting up the Test Project Computer

You need to set up the test project computer only once for the project.

**To set up the CitectVBA test project computer**

1. Open the CitectVBA Test Project.
2. Click the **Citect Explorer** button.
3. In the project list column, click the root computer icon named **My Projects**.
4. Double-click the **Computer Setup Wizard** button. (You can also click the **Computer Setup** button, or choose **Computer Setup** from the **Tools** menu.).
5. Select **Next** accepting the **Express Setup** default.
6. Select **Next** accepting the **Standalone Computer - Server and Control Client** default.
7. From the **Project Name** list, select **CitectVBA Test** and click **Next**.
8. Click **Next** and then **Finish** to complete the CitectVBA project communications setup.

**See Also**

Adding a Variable Tag

## Adding a Variable Tag

You need to add a variable tag only once for the project.

**To add a variable tag to the CitectVBA test project**

1. Open the CitectVBA test project.
2. Click the **Project Editor** button.
3. Click the **Variable Tags** button.
4. In the **Variable Tag Name** field, replace the default name **Tag_1** with **TestTag_1**.

> **Note:**Vijeo Citect stores the communication details as records in a database. Each record name is limited to a maximum of 16 characters.

5. In the **I/O Device Name** field, check that the device name selected is **CiV-BAIODevice**. (If other I/O Devices have been created for this project, they will display in this menu.)
6. In the **Data Type** field, select **INT** from the menu.
7. In the **Address** field, typeI1 (the capital letter i and the number one).
8. Click **Add**.

**See Also**

Adding a Graphics Page

## Adding a Graphics Page

Adding a test page to your test project allows you to view the features of the Citect Graphics Builder.

**To add a graphics page to the CitectVBA test project**

1. In Citect Explorer, double-click the **Graphics** folder.
2. Double-click the **Create a new page** button in the **Pages** folder, or click the **Graphics Builder** button.
3. Click **New**, and then click **Page**.

The Graphics Builder appears showing the templates that are available. Accept the default template and click **OK**.

**See Also**

Saving Your Graphics Page

## Saving Your Graphics Page

To save a graphics page, you must give it a name.

**To save the graphics page**

1. Click **Save**.
2. In the **Page** field on the **Page** tab, replace the default name **Untitled1** with **Startup**. When you start this project, this page will be displayed by default.
3. Click **OK**.

**See Also**

Opening the Graphics Page

## Opening the Graphics Page

Opening the graphics page allows you to edit the page.

### To open the CitectVBA test project graphics page

1. Choose **Open** from the **File** menu in Graphics Builder, or click **Open**.
2. In the **Project** field on the **Page** tab, change to the **CitectVBA Test** project (if not already selected).
3. In the **Page** field on the **Page** tab, select the file named **Startup**.
4. Click **OK**.

> **Note:** Double clicking a graphic page icon in the Citect Explorer launches the Citect Graphics Builder and displays the selected graphics page.

### See Also

Accessing Cicode Tags with CitectVBA

# Chapter 4: Understanding CitectVBA Language Basics

This section describes the basics of the CitectVBA programming language.

## CitectVBA Files

CitectVBA code scripts can be saved to file, can include comments, statements, various representations of numbers, can handle many different data types, and can have multiple and nested control structures. However, CitectVBA is primarily provided with Vijeo Citect to interact with ActiveX objects.

CitectVBA files are ASCII text files stored in ANSI format with a BAS extension (*filename*.BAS), and are known as file modules.

CitectVBA file modules can be viewed and edited in any text editor program. They can be used in Vijeo Citect, but must be saved as 'text with linebreaks' with a '.BAS' file extension or Citect will not be able to open the file.

## Cicode Editor

The Cicode Editor is CitectVBA aware and designed to help you create, edit, test, and debug CitectVBA file modules in your Vijeo Citect project.

The Cicode Editor has features suitable for use with CitectVBA file modules including:

- Ability to create, open, edit, and save CitectVBA file modules
- Customizable coloration of CitectVBA code syntax structure
- Recognition of predefined keywords with tooltip prompting and auto-completion functionality
- Fully integrated debugging of CitectVBA file modules
- Separate VB Watch window for viewing runtime CitectVBA variable values

A sample CitectVBA file module named Sample.Bas is included in the User\Example subfolder on the drive on which you installed Vijeo Citect. This module explains most of the CitectVBA functionality.

CitectVBA file modules will never be compiled into standalone Windows executable files; instead, they're included with the compiled Vijeo Citect. As a result, they don't require a `Main`procedure to be declared. Therefore, CitectVBA file modules are structured to contain only their header information, modular constant and variable declarations, then procedures (subroutines, and functions).

CitectVBA file modules are automatically included with a Vijeo Citect project if they are stored in the same file folder as your project. When saving a CitectVBA file module to disk, save it to your project folder.

All files with a BAS extension in your project folder appear in the CitectVBA Files folder of your project in Citect Explorer. To launch the Cicode Editor, double-click the CitectVBA file you want to edit in Citect Explorer.

## Scope of CitectVBA

The scope of an object determines which portions of your code scripts can use that object.

**Note:** The use of **Global**, **Public**, and **Private**keywords has no effect on scope in CitectVBA.

### Procedural (local) level scope

Variables and constants declared (using the `Dim`, `Static`, or `Const`statements) within a CitectVBA procedure (subroutine or function) have local scope to only that within the procedure. This means that procedural level variables and constants cannot be referenced (accessed and used) from anywhere outside of that procedure.

**⚠ WARNING**

**UNINTENDED EQUIPMENT OPERATION**

Do not use the Global, Public, or Private keywords in your CitectVBA procedures. Using these keywords in procedures can lead to unintended equipment operation when your program is run.

**Failure to follow these instructions can result in death, serious injury, or equipment damage.**

Procedural level variables declared using the `Dim` statement do not retain their assigned values when dereferenced. Procedural level variables declared using the `Static` statement, however, do retain their assigned values between references, even after that procedure moves out of scope.

## Modular level scope

Constants declared (using the `Const` statement) and variables declared (using the `Static` statement) at the modular level (outside any procedure) in a CitectVBA file have modular scope to all procedures within that same CitectVBA module (file). This means that modular constants and static variables can only be referenced from a procedure located within the same file module, and cannot be referenced from outside of that file module. This has no effect in CitectVBA, even if declared using the `Global` keyword.

Modular level constants and static variables retain their assigned values for the entire runtime of the project.

## Global level scope

Variables declared (using the `Dim`, `Global`, or `Public` statements) at the modular level (outside any procedure) in a CitectVBA module (file), have global scope within the Vijeo Citect project. This means that modular CitectVBA variables (except statics) can be referenced from both inside and outside of their file module.

Global level variables can be used directly within Vijeo Citect command or expression fields.

Procedures (subroutines or functions) declared within a CitectVBA file module, like global variables, have global scope within a Vijeo Citect project. They can be referenced or called from any CitectVBA module, as well as from any Vijeo Citect command or expression field.

Equally important, all Vijeo Citect variable tags, alarm tags, and ActiveX objects are accessible to all CitectVBA file modules (and their procedures) within that project, in the same manner as they have always been accessible to project Cicode files. For information about referencing Vijeo Citect project tags using CitectVBA, see Integrating CitectVBA with Vijeo Citect.

Global level variables will also retain their assigned values between subsequent references, behaving somewhat similarly to the values stored in Vijeo Citect tags. In this regard, `Global` and `Public` statements are redundant at the modular (global) level in CitectVBA, as they perform the exact same duty as the `Dim` statement.

### See Also

Multithread Considerations with CitectVBA

CitectVBA Files

## CitectVBA Statements

A statement in CitectVBA is an unbroken sequence of syntactically correct code script containing at least one CitectVBA keyword instruction. A single statement in CitectVBA is one complete segment of code script that instructs Vijeo Citect to do something.

In CitectVBA there is no statement terminator. As in other BASIC programming languages, the end of the line containing the statement is treated as the statement terminator by default.

Most often, a statement consists of a single line of CitectVBA script. However, more than one statement can be placed on one line of CitectVBA script, provided each statement is separated by a colon character (:); for example:

```
Pump234.AddPoint( 25, 100): Pump234.AddPoint( 0, 75)
```

is equivalent in CitectVBA to:

```
Pump234.AddPoint( 25, 100)
Pump234.AddPoint( 0, 75)
```

Using complex multi-statement lines of CitectVBA script is not recommended in Vijeo Citect. Multithreading should be considered when using more than one statement per line in CitectVBA. For details, see Multithread Considerations with CitectVBA.

## Comments

Comments are non-executed sections of code that are ignored by the CitectVBA compiler. Comments allow programmers to describe the purpose of a section of code to facilitate code maintenance.

As in other BASIC programming languages, both the apostrophe character ( ' ), and the keyword REMare recognized as the start of a comment in CitectVBA. All characters following an apostrophe or the keyword REMare ignored by the CitectVBA compiler until it reaches the end of the line. Line continuation characters do not work inside comments.

REM, like all other keywords and most names in CitectVBA, is not case sensitive.

```
' This whole line is a comment
rem        This whole line is a comment
Rem        This whole line is a comment
REM        This whole line is a comment
```

Both types of comments can be used on their own separate line, or the apostrophe character can be used to start a comment at the end of a statement on the same line as a statement.

```
Pump234.AddPoint( 25, 100     ' Add point to pump 234
```

Everything placed on the same line after an apostrophe is treated by CitectVBA as a comment. If you want to place a comment on the same line as a statement, the comment must be placed last after all statements on that line. Comments cannot be placed between multiple statements on the same line.

Not every line of code requires a comment. In fact, CitectVBA should contain understandable naming structures and be laid out in such a manner as to make comments unnecessary. However, where a complex function, equation, or logic structure is not readily understandable by viewing the code, it is good practice to include a pertinent comment to make the code more understandable when viewed in isolation.

**See Also**

Comments

## Header information

You should include header information with every file you create or edit. Data such as the file name, author name, creation date, update date, editing history, and the like should be included to form the header information. Each function or subroutine should include a brief comment describing the purpose or function of the procedure.

**CitectVBA file header example**

```
' FILE IDENTIFICATION
' CitectVBA example named CitectVBA.bas
' Created by Citect Documentation Team
' Created in April 2001
```

**See Also**

Header information

CitectVBA Files

## Labels

Labels can be used to divide a large CitectVBA function or subroutine into logical sub-sections of code script. Labels are often used in association with the `GoTo` statement. All of the CitectVBA script following the label and extending through to another label, or to the end of the function or subroutine containing the label, is regarded as belonging to that label. Or more appropriately, the label is said to identify, or be attached to, that particular section of CitectVBA script.

Labels must begin with a letter, be no longer than 40 characters, and cannot be a reserved word. Labels must terminate with the colon character (:). Label names can only contain the letters 'A' to 'Z' and 'a' to 'z', the underscore '_' character, and the digits '0' to '9'. Label names cannot contain the space character.

Label names (once declared), become a keyword in CitectVBA. Like most keywords in CitectVBA, label names are not case sensitive. For example, all of the following label examples are treated identically in CitectVBA:

```
label1:
Label1:
LABEL1:
```

**Note:** Labels as used in CitectVBA are not the same as labels used in Vijeo Citect.

**See Also**
CitectVBA Files

## CitectVBA Line Continuation Character

The underscore is the line continuation character in CitectVBA. There must be a space before and after the line continuation character. Line continuation characters do not work inside comments.

The following sample code statements are treated identically in CitectVBA:

```
Pump234.AddPoint _
( 25, 100)

Pump234.AddPoint( 25, 100)
```

Strings cannot be separated between lines using the line-break character in CitectVBA, unless the strings are properly enclosed within double quotes on each line, and appended together as per the following example:

```
Dim strSample as String
strSample = "This sentence on the first line in my code. " _
& "This sentence is on the second line in my code. " _
& "Yet all would display on the same line " _
& "if the display were wide enough."
```

## Naming

Function, subroutine, variable, constant, and label naming in CitectVBA must begin with a letter, be no longer than 40 characters, and cannot be a reserved word. Names can only contain the letters 'A' to 'Z' and 'a' to 'z', the underscore '_' character, and the digits '0' to '9'. Names cannot contain the space character. You cannot use the name of a CitectVBA predefined function as a name. For a list of predefined functions, see CitectVBA Function Reference.

Function, subroutine, variable, constant, and label object names (once declared), become a keyword in CitectVBA. Like most keywords in CitectVBA, these names are not case sensitive. For example, all of the following examples are treated identically in CitectVBA:

```
pump234.addpoint(25, 100)
Pump234.AddPoint(25, 100)
PUMP234.ADDPOINT(25, 100)
```

When naming in CitectVBA, make the name an appropriately descriptive term that is easily recognizable. For example:

```
X.addpoint(25, 100)
```

doesn't make as much sense as:

```
Pump234.AddPoint(25, 100)
```

Combining upper- and lowercase letters between words in the name is an acceptable common programming practice, and aids in readability.

Identically named objects cannot be declared more than once per Vijeo Citect project, even though they may exist in different CitectVBA code file modules. However, if an object declared locally within a procedure has the same name as an object declared in a module, CitectVBA will reference the local procedure scope object instead of the modular scope object.

### See Also

Scope of CitectVBA

CitectVBA Files

Accessing Cicode Tags with CitectVBA

## Option Statements

CitectVBA supports the use of file scope `Option`statements which determine the default behaviour of some CitectVBA functions. For instance, the `Option Explicit`statement causes the CitectVBA compiler to produce compile errors whenever it encounters the use of previously undeclared variables. The `Option Compare`statement sets the default comparison method for string comparisons. The `Option Base`statement sets the default base number for CitectVBA variable arrays to either zero or one.

You must declare all `option`statements in CitectVBA at the beginning of your CitectVBA code files.

**See Also**

Option Explicit statement

Option Compare statement

Option Base statement

CitectVBA Function Reference

### Option Explicit statement

As in other BASIC programming languages, CitectVBA supports the declaration of variables both implicitly and explicitly. An unfortunate consequence of implicit variable declaration is the possible misspelling of the variable name in subsequent code writing, with unreliable program behaviour and unpredictable consequences.

To minimize implicit declaration, and to foster good, consistent programming standards, use the `option explicit` statement at the beginning of all your CitectVBA files:

```
Option Explicit
```

This causes the CitectVBA compiler to produce a compile error whenever it encounters an undeclared variable. This can be useful in locating and identifying variable name typing errors in your CitectVBA code at compile time, thus trapping and minimizing the likelihood of runtime errors caused by such mistakes.

**See Also**

Option Explicit statement

Variable declaration

Option Statements

## Option Compare statement

The `Option Compare` statement determines how strings are compared within a CitectVBA file, and like other `Option` statements in CitectVBA, should be declared at the beginning of your CitectVBA code files.

When strings are compared using CitectVBA functions such as `StrComp()` or `InStr()`, CitectVBA determines whether they contain equivalent characters and how they differ if they do not match.

> **Note:** When comparing strings, CitectVBA compares the ANSI values of each character in the strings. For example, the character capital 'A' has the ANSI value of 65, and the character lowercase 'a' has the ANSI value of 97. For a listing of ANSI character values, see ASCII/ANSI Character Code Listings.

You can use the `Option Compare` statement to specify the default case-sensitivity behavior for CitectVBA functions when making string comparisons.

The `Option Compare` statement in CitectVBA has two settings:

- **Option Compare Binary**: String comparisons are case-sensitive, and this is the default string-comparison setting.
- **Option Compare Text**: String comparisons are case-insensitive.

### See Also

Strings

Option Statements

## Option Base statement

The `Option Base` statement determines the default base number for the indexing of variable arrays created within a CitectVBA file, and like other `Option` statements in CitectVBA, should be declared at the beginning of your CitectVBA code files.

There are two settings for the `Option Base` statement in CitectVBA:

- **Option Base 0**: Variable arrays are indexed from number zero, and this is the default setting.
- **Option Base 1**: Variable arrays are indexed from number one.

For an example of using the Option Base statement, see Fixed Size Arrays

### See Also

Arrays of Variables

Option Statements

## CitectVBA Data Types

CitectVBA uses ten predefined data types:

| Variable | Char | Type Declaration | Size | Value Range |
|---|---|---|---|---|
| Byte | | Dim bytVar As Byte | 1 byte (8 bits) | 0 to 255 |
| Boolean | | Dim binVar As Boolean | 2 bytes | True or False |
| String | $ | Dim strVar As String | 10 bytes + 1 byte per character | 0 to 65,535 characters |
| Integer | % | Dim intVar As Integer | 2 bytes | -32,768 to 32,767 |
| Long Integer | & | Dim lngVar As Long | 4 bytes | -2,147,483,648 to 2, 147,483,647 |
| Single precision | ! | Dim sglVar As Single | 4 bytes | 3.4E-38 to 3.4E+38 |
| Double Precision | # | Dim dblVar As Double | 8 bytes | 1.79D-308 to 1.79D+308 |
| Variant | | Dim vntVar As Any | 16 bytes | Same ranges as data types stored |
| Object | | Dim objVar As Object | 4 bytes | Any OLE Object reference |
| Date/Time | | Dim dtmVar As Date | 8 bytes | Jan 1, 100 to Dec 31, 9999 |

**Note:** CitectVBA does not support user-defined data types.

### See Also

Numeric Data Types

Numbers

Variables

CitectVBA Function Reference

## Constants

Your CitectVBA code may contain frequently recurring constant values like Pi, or may contain numbers that are difficult to remember or have no obvious meaning. You can make your CitectVBA code much easier to read and maintain using constants to represent those values.

Unlike variables, constants can't be changed once your Vijeo Citect project is compiled and running. Constants are either symbolic or intrinsic:

- Symbolic or user-defined constants are declared by using the `const` statement.
- Intrinsic constants are provided in object libraries of ActiveX objects and you cannot use them in CitectVBA: they cause compile errors as there is no way to provide early-binding to the object type library.

You can create a constant in CitectVBA named `Pi`, assign it the numeric value once in your code, then refer to it by using the constant name, as shown here:

```
'modular level constant declaration
Const Pi = 3.1415926
Function CircleArea(Byval Radius)
' calculate and return area of circle
' using radius passed in as argument
CircleArea = Pi * (Radius * Radius)
End Function
Function CircleCircumference(Byval Radius)
' calculate and return circumference of circle
' using radius passed in as argument
CircleCicumference = Pi * Radius * 2
End Function
```

These CitectVBA functions would be called from a Vijeo Citect command or expression field like this:

```
CiVBA
TestTag_1 = CircleArea(TestTag_1)
```

or

```
CiVBA
TestTag_1 = CircleCircumference(TestTag_1)
```

### See Also

Declaration of constants

Passing variables Byref and Byval

Integrating CitectVBA with Vijeo Citect

Intrinsic constants

Scope of CitectVBA

CitectVBA Function Reference

## Declaration of constants

CitectVBA constants can only be declared and referenced within CitectVBA file modules. CitectVBA modular constants have modular scope and cannot be referenced (accessed and used) from outside their CitectVBA module (file).

> **Note:** CitectVBA constants cannot be used directly in Vijeo Citect command or expression fields.

Once declared within a CitectVBA module, CitectVBA constants can be referenced and used in any procedure within the same code module. A constant declared outside a procedure has modular scope to all procedures within that same CitectVBA module (file). See Scope of CitectVBA. Constants declared in a Sub or Function procedure have local scope only within that procedure.

CitectVBA constants are declared with the `Const`statement in the following format.

```
Const <ConstantName> [ As <DataType> ] = <expression>
```

where:

- `Const` is the required constant declaration statement BASIC keyword
- `<ConstantName>` represents the required name of the constant being declared
- `<DataType>` represents the optional CitectVBA data type of the constant being declared
- `<expression>` represents the required value being assigned to the constant

> **Note:** Do not include the brackets from the explanation in the actual code statement.

If no data type is declared, CitectVBA automatically assigns one of the following data types to the constant:

- Long (if it is a long or integer).
- Double (if a decimal place is present).
- String (if it contains quote marks).

Constant statements can only be declared and assigned using simple expressions. Constants cannot be assigned values from variables, user-defined functions, intrinsic CitectVBA functions (such as `Chr`), or from any expression that involves an operator. A constant needs to be defined before it can be used.

**Example**

```
' Correct declaration examples
Const Seven = 7

' long assignment
Const Pi = 3.14159

' double assignment
Const Lab = "Laboratory"

' string assignment
' Incorrect declaration examples. Note that the following
declarations demonstrate incorrect assignments because each
contains an operator
Const conPi = 4 * Atn(1)
' will cause a CitectVBA compile error
Const conDegToRad = (conPi / 180)
' will cause a CitectVBA compile error
```

For an example of using constants in CitectVBA, see Constants.

**Note:** The use of Global, Public, and Private keywords has no effect on scope in CitectVBA.

**See Also**

Constants

Intrinsic constants

Variables

CitectVBA Data Types

CitectVBA Function Reference

## Intrinsic constants

CitectVBA has no predefined intrinsic (built-in and declared) constants, however, does provide limited support for intrinsic constants provided in object libraries of ActiveX objects when the object they refer to is loaded using the predefined CitectVBA `CreateObject()` function.

**See Also**

Declaration of constants

Constants

## Variables

Variables are used in CitectVBA to temporarily store data values. Variables let you assign a descriptive name to the data you are working with. You can create a variable once only in your code, and reference (refer to) it thereafter as many times as you like, by using its name in your code in place of the data value. Unlike constants, the value that a variable holds can be changed during the runtime of the project.

All variables declared within a CitectVBA procedure (subroutine or function) have local scope to that procedure only. Procedural level variables declared using the `Dim` statement do not retain their assigned values when dereferenced. Procedural level variables declared using the `Static` statement, however, retain their assigned values between references, even after that procedure moves out of scope.

CitectVBA code used within a Vijeo Citect command or expression field is treated as if the command or expression is a separate CitectVBA procedure. Variables declared within such a command procedure have procedural scope and lifetime, as described above.

Variables declared using the `static`statement at the modular level (outside any procedure) in a CitectVBA file, have modular scope to all procedures within that same CitectVBA module (file). Modular level `static`variables retain their assigned values for the entire runtime of the project.

Variables declared (using the `dim`, `global`, or `public`statements) at the modular level (outside any procedure) in a CitectVBA file do, however, have global scope within the Vijeo Citect project.

> **Note:** `Global` and `public` statements are redundant at the modular (global) level in CitectVBA, as they perform the exact same duty as the `dim` statement.

### Variable declaration

In CitectVBA, variables are declared (dimensioned) with the `dim` statement in the following format.

```
Dim <VariableName> [ As <DataType> ]
```

where:

- `Dim` is the required Variable declaration statement BASIC keyword
- `<VariableName>` represents the required name of the variable being declared (dimensioned)
- `<DataType>` represents the optional CitectVBA data type of the variable being declared

**Note:** In the variable declaration statement:

- Every placeholder shown inside arrow brackets ( <*placeholder*>) should be replaced in any actual code with the value of the item that it describes. The arrow brackets and the word they contain should not be included in the statement, and are shown here only for your information.
- Statements shown between square brackets ( `[ ]`) are optional. The square brackets should not be included in the statement, and are shown here only for your information.

If no data type is declared, the data type is Variant by default. To declare a variable other than a Variant, the variable declaration needs to be immediately followed by `As <datatype>` (where `<datatype>` represents one of the 10 data types), or appended by a type declaration character such as a $, %, &, !, or # for string, integer, long, single, or double data types respectively. For example:

```
Dim intVar As Integer
Dim dblVar As Double
Dim vntVar    ' as variant by default
Dim strName$, Age%    ' multiple declarations on one line
```

Be aware that multiple declarations in the same statement require individual data type assignment if you want them be other than the variant type. In the following example, only the first variable is not a variant. For example:

```
Dim strName As String, vntAge, vntAddress
```

The same statement with data type assignment for every variable would look like the following example:

```
Dim strName As String, intAge As Integer, strAddress As String
```

**See Also**

CitectVBA Data Types

Variable initialization values

Constants

Variant Declaration

Arrays of Variables

CitectVBA Function Reference

## Variable initialization values

CitectVBA variables are initialized when first declared. Numeric variables are initialized to 0 (zero). Variable-length strings are initialized to zero-length strings (""). Fixed length strings are filled with zeros. Variant variables are initialized to Empty.

To be sure of the contents of a variable, a valid value needs to be assigned to it before it is used as a operand in a CitectVBA statement. For details, see Assignment Operator.

> **Note:** Only implicitly declared variables can be assigned an initial value in the declaration. However, as explicit declaration is preferred practice in CitectVBA, explicit variables need to be declared before they can be assigned a value.

Every call to a procedure will reinitialize the value of all objects (except `static` variables) declared within that procedure.

> **Note:** In CitectVBA, use a static variable, a modular variable, or a Vijeo Citect tag to store variable values between procedures. For details, see Scope of CitectVBA.

Objects (including variables) declared in CitectVBA are only initialized when referenced by a running piece of code, and are removed from memory when all references are closed.

In the Vijeo Citect multithreaded environment, CitectVBA remains active in memory only so long as a procedure is being processed. At the completion of a CitectVBA procedure, all objects no longer referenced by that procedure are removed from memory. For details, see Multithread Considerations with CitectVBA.

### See Also

CitectVBA Data Types

Variable initialization values

Constants

Variant Declaration

Arrays of Variables

## Arrays of Variables

Arrays of variables allow you to group like variables together, somewhat similar to the grouping of like items in fields of a database. An array is an ordered group of variables of the same name, containing values of the same data type. Individual member elements of the array are identified by a separate index number. Arrays in CitectVBA start their indexing sequence by default at zero. This default base value can be changed in a CitectVBA file module by using the `option base` statement.

CitectVBA supports single and multi-dimension arrays of variables. CitectVBA creates single dimension arrays by default. Multi-dimension arrays must be specifically declared.

CitectVBA allocates memory space for each element of the array. To minimize the amount of memory used storing arrays, and to minimize the time required to access array data, arrays should not be declared any larger than required.

All elements in an array must be of the same data type. CitectVBA supports arrays of bytes, booleans, longs, integers, singles, doubles, strings, and variants. For details about CitectVBA data types, see CitectVBA Data Types.

Arrays declared in a `sub` or `function` procedure have local scope only within that procedure. An array declared outside a procedure has modular (global) scope to all procedures within the project.

> **Note:** CitectVBA arrays cannot be used directly in Vijeo Citect command or expression fields. Also, CitectVBA does not support user-defined data types.

Arrays declared (using the `dim` statement within procedures,) do not retain their values between procedure calls in CitectVBA.

### See Also

Fixed Size Arrays

Multi-Dimensional Arrays

Dynamic Size Arrays

#### Variable Array Declaration

Arrays of variables are declared within a CitectVBA file module, function, or subroutine, using the `dim` statement with parentheses positioned after the array name, in the following syntax:

```
Dim <ArrayName>( [<Subscripts>] ) [As <DataType>]
```

where:

- `dim` is the required variable declaration statement BASIC keyword.
- `<ArrayName>` represents the required name of the array being declared (dimensioned).
- `( )` are the required parentheses to hold the array subscript range (dimensions).
- `<Subscripts>` represents the optional subscript ranges and dimensions for the array.
- `As` is the optional As statement keyword declaring the array data type.
- `<DataType>` represents the optional CitectVBA data type declaration for the array.

In the variable array declaration statement:

- Every placeholder shown inside arrow brackets ( `<placeholder>` ) should be replaced in any actual code with the value of the item that it describes. The arrow brackets and the word they contain should not be included in the statement, and are shown here only for your information.
- Statements shown between square brackets ( `[ ]` ) are optional. The square brackets should not be included in the statement, and are shown here only for your information.

**See Also**

Fixed Size Arrays

Multi-Dimensional Arrays

Dynamic Size Arrays

Array Subscripts

Arrays of Variables

Dim

### Array Subscripts

Arrays can be declared with default or defined boundaries known as bounds. Unless specifically defined in the array declaration statement, default lower bound settings are used. The default lower bound is zero, unless set by the module `option base` statement setting.

CitectVBA does not have an arbitrary upper bound on array dimensions. The upper bound of the array dimension must be defined before the array can be used. All bound values must be whole integers.

Subscripts are contained within one set of parentheses positioned immediately after the array name in the array declaration statement.

Subscripts are used to specify the bounds of each dimension of an array when the array is declared. If a single value is used, for instance (5), this represents the upper bound for that dimension of the array. If a range is specified, for instance (1 `to` 9), this specifies both the lower and upper bounds for that dimension of the array. If more than one subscript is used, for instance ( 5, 1 `To` 9), each subscript must be separated by a comma, and each subscript represents a separate dimension of the array.

The syntax of an array subscript range consists of a numeric value range separated by the to clause:

```
(<LowerBound> To <UpperBound>)
```

where:

- `( )` are the required parentheses to hold an array subscript range (dimensions).
- `<LowerBound>` represents the lower bound of the subscript range for the array dimension.
- `To` is the clause linking the lower and upper bounds of the subscript range.
- `<UpperBound>` represents the upper bound of the subscript range for the array dimension.

**See Also**

Fixed Size Arrays

Multi-Dimensional Arrays

Dynamic Size Arrays

Arrays of Variables

Dim

### Fixed Size Arrays

To declare a fixed size array, the array name must be followed by the upper bound subscript enclosed within parentheses. The upper bound must be an integer.

```
Dim ArrayName(10) As Integer
Dim Sum(20) As Double
```

Unless specifically defined in the array declaration statement, default lower bound settings are used. The default lower bound is zero, unless set by the module `option base` statement setting. For details, see Array Subscripts.

The first declaration in the previous example creates an array with 11 elements, with index numbers running from 0 to 10. The second creates an array with 21 elements (if base 0). One way to specify the lower bound is to provide it explicitly (as an integer in the range -32,768 to 32,767) using the `To` clause within the subscript:

```
Dim intCounters (1 To13) As Integer
Dim strSums (100 To126) As String
```

In the preceding example, the index numbers of `intCounters` run from 1-13, and the index numbers of `strSums` run from 100-126.

**Note:** An array in CitectVBA must be declared before it can be referenced.

Loops often provide an efficient way to manipulate arrays. For example, the following `for` loop initializes all elements in the array to 5:

```
Dim int As IntegerDim Counters(1 To 20) As Integer
For int = 1 To 20
Counters(int) = 5
Next int
```

Arrays declared (using the `dim` statement within procedures) do not retain their values between procedure calls in CitectVBA.

**See Also**

Multi-Dimensional Arrays

Dynamic Size Arrays

Arrays of Variables

Array Subscripts

Option Base statement

### Multi-Dimensional Arrays

CitectVBA supports multi-dimensional arrays, declared using multiple subscripts. Each subscript must be separated by a comma, and each subscript represents a separate dimension of the array.

The following example declares a two-dimensional array.

```
Dim dblMat(20, 20) As Double
```

Unless specifically defined in the array declaration statement, default lower bound settings are used. The default lower bound is zero, unless set by the module `option base` statement setting. For more information on bounds, see "Array Subscripts in CitectVBA."

Reusing the previous example, either or both dimensions can be declared with explicit lower bounds.

```
Dim dblMat(1 To10, 1 To10) As Double
```

Arrays can be more than two dimensional. This declaration creates an array that has three dimensions with sizes 6 elements by 4 elements by 3 elements, using base 0:

```
Dim ArrTest(5, 3, 2)
```

You can efficiently process a multi-dimensional array with the use of for loops. In the following statements the elements in a multi-dimensional array are set to a value.

```
Dim L As Integer, J As Integer
Dim TestArray(1 To 10, 1 to 10) As Double
For L = 1 to 10
    For J = 1 to 10
        TestArray(L,J) = I * 10 + J
    Next J
Next L
```

Arrays declared (using the `dim` statement within procedures,) do not retain their values between procedure calls in CitectVBA.

**See Also**

Multi-Dimensional Arrays

Dynamic Size Arrays

Arrays of Variables

Array Subscripts

Option Base statement

Fixed Size Arrays

### Dynamic Size Arrays

To declare a dynamic sized array, the array must first be declared using the `dim` statement with an empty pair of parentheses following the array name. For example:

```
Dim ArrayName( ) As Integer
```

Once declared as dynamic in this manner, the array can then ONLY be resized within a function or subroutine using the `redim` statement.

```
ReDim ArrayName(20) As Integer
```

**Note:** You cannot resize an array whose size was predefined in its initial declaration.

In the above examples, the first declaration creates an array with 0 elements. The second recreates the array to contain 21 elements, with index numbers running from 0 to 20, unless the `option base` statement has been set previously in the code module (file), in which case the array will contain 20 elements with index numbering ranging from 1 to 21.

Unless specifically defined in the array declaration statement, default lower bound settings are used. The default lower bound is zero, unless set by the module `option base` statement setting. For more information on bounds, see "Array Subscripts in CitectVBA."

`Redim` erases all values the array may have held. To preserve the contents of the array when resizing, precede the Redim statement with the `preserve` keyword.

```
Preserve ReDim ArrayName(20) As Integer
```

Redimensioning an array to a smaller value, will erase any values it may have contained in the removed portions.

Arrays declared (using the `dim` statement within procedures,) do not retain their values between procedure calls in CitectVBA.

**See Also**

Multi-Dimensional Arrays

Arrays of Variables

Array Subscripts

Option Base statement

Fixed Size Arrays

## Variant Declaration

As is the case with Visual Basic, when a variable is introduced in CitectVBA, it is not necessary to declare it first (see Option Explicit statement for an exception to this rule). When a variable is used but not declared, it is implicitly created as a variant data type. Variants can also be declared explicitly using `As Variant`. Both of the following example declarations as treated identically in CitectVBA:

```
Dim vntVar    ' implicit variant declaration
Dim vntVar As Variant     ' explicit variant declaration
```

The `IsEmpty( )` function can be used to find out if a variant variable has been previously assigned a value.

### Variant Data Types and Coercion

The variant data type is capable of storing numbers, strings, dates, and times. When using a variant, you do not have to explicitly convert a variable from one data type to another. This data type conversion is handled automatically, and is termed data type coercion.

```
' declares variant variable
Dim vntVar
' assign numeric 10 to variant
vntVar = 10
' add numeric 8 to numeric variant value
vntVar = vntVar + 8
' convert variant to string value and concatenates strings
vntVar = "F" & vntVar
' print string "F18"
print vntVar
```

Numeric characters inside quotes ("567") will be stored and treated as a string in a variant data type variable. If this string (containing numeric characters) is subsequently used in a numeric operation, it will be coerced into a numeric data type and treated as a number in the operation. Conversely, numeric characters stored as a number data type in a variant variable, and subsequently used in an operation with a string, will be coerced into a string data type, and treated as a string value in the operation.

**Note:** To determine the type of a variant variable, use the function `VarType( )`, which returns a value that corresponds to the explicit data types. See VarType for return values.

### Numbers in Variants

When storing numbers in variant variables, the data type used is the most compact type possible. For example, if you first assign a whole number to the variant it will be stored as an integer or long. If you assign a number with a fractional component, it is stored as a single or double.

For doing numeric operations on a variant variable, it is sometimes necessary to determine if the value stored is a valid numeric, thus avoiding an error. This can be done with the `IsNumeric( )` function.

**See Also**

CitectVBA Data Types

Variables

Constants

Strings

Numbers

CitectVBA Function Reference

## Numbers

CitectVBA supports three representations of numbers: decimal, octal, and hexadecimal.

To indicate the use of octal (base 8) or hexadecimal (base 16) numbers, prefix the number with `&O` or `&H` respectively. If no prefix is included with a number, it is treated as decimal (base 10). For example:

```
Dim vntVar as Variant
vntVar = 12345    ' assign decimal value
vntVar = &o12345    ' assign octal value
vntVar = &h12345    ' assign hexadecimal value
```

Most numbers used in CitectVBA formulas are decimal numbers. Decimal numbers consist of integral values (known as integers) positioned to the left of the decimal point, and fractional values (known as fractions) positioned to the right of the decimal point. If the decimal point is omitted, the number is treated as an integer (whole number with no fraction).

When using numbers in CitectVBA, consideration needs to be given to the data type of the variables that hold and store the numbers, as well as to the behaviour of CitectVBA when dealing with numbers. For details, see Numeric Data Types, Floating Point Calculation Rules, and Rounding Numbers.

### See Also

Date Handling

Variant Declaration

Strings

Variables

Constants

CitectVBA Data Types

CitectVBA Function Reference

### Numeric Data Types

Numbers are expressed in CitectVBA in decimal format by default, and can be stored in variables of different numeric data types-integer, long, single, double, and variant-providing different levels of numeric accuracy.

- **Integer** (Integer data type) variables can only store whole number values (no decimal or fraction values) within the range -32,000 to +32,000. If you use a number outside this range, the long integer (Long data type) can store whole number values in the range -2.1 million to +2.1 million.
- **Floating point** numbers contain both integer and fractional values with a floating decimal point. CitectVBA provides both single precision (Single data type) and Double Precision (Double data type) variables for handling floating-point numbers.
- **Single-precision** variables can store floating-point numbers within the range of approximately 3.4E-38 to 3.4E+38, with 7 significant digits and occupying 4 bytes of memory.
- **Double-precision** variables can store floating-point numbers within the range of approximately 1.79D-308 to 1.79D+308, with 15 significant digits and occupying 8 bytes of memory.

For an explanation of exponential notation, see Exponential Notation.

The principal differences between single and double data types, are the significance they can represent, the storage they require, and their range. Double data type variables have a smaller range, but hold more precision and occupy more memory than single data type variables.

If precision is less of a concern than storage, consider using single for floating-point variables. Conversely, if precision is the most important criterion, use double.

Variant (variant data type) variables in CitectVBA can store numbers by storing them as integer, long, single, or double data types within the variant structure. See Variant Declaration.

**See Also**

Numbers

## Exponential Notation

CitectVBA can handle very large numbers, up to a value of 1.7976931486232 raised to the power of 308, (1.7 308). To represent very large numbers such as these, exponential notation is used.

Commonly, exponential notation uses the letter 'E' in the number, followed by the sign of the number (+ or -), and then the exponential value (power) of the number. CitectVBA uses the letter 'E' for Single data type exponential values, and the letter 'D' for Double data type exponential values. The maximum size number for a double precision data type, as quoted above, would be represented using exponential notation as 1.7976931486232D+308, or abbreviated to 1.79D+308.

You can use exponential notation in your CitectVBA calculations, so long as the data types of all the variables in the calculation are capable of storing floating point values; i.e.: Single, Double or Variant.

For details about precision, accuracy, and rounding issues with using floating point variables in CitectVBA, see Numeric Data Types, Floating Point Calculation Rules, and Rounding Numbers.

**See Also**

Numbers

## Floating Point Calculation Rules

Often precision, rounding, and accuracy in floating-point calculations can generate unexpected results. To avoid this situation, follow these rules:

1. In a calculation involving both single and double precision, the result will not usually be any more accurate than single precision. If double precision is required, be certain all terms in the calculation, including constants, are specified in double precision.
2. Never assume that a simple numeric value is accurately represented in the computer. Most floating-point values can't be precisely represented as a finite binary value. For example .1 is .0001100110011... in binary (it repeats forever), so it can't be represented with complete accuracy on a computer using binary arithmetic, which includes all PCs.
3. Never assume that the result is accurate to the last decimal place. There are always small differences between the "true" answer and what can be calculated with the finite precision of any floating point processing unit.
4. Never compare two floating-point values to see if they are equal or not-equal. This is a corollary to rule three. There are almost always going to be small differences between numbers that "should" be equal. Instead, always check to see if the numbers are nearly equal. That is, check to see if the difference between them is very small or insignificant.

**See Also**

Calculating disk storage

Numbers

Date Handling

## Rounding Numbers

Rounding occurs when you convert a number of greater precision into a number of lesser precision. For instance, when converting a floating-point number (single precision, double precision, or variant data types) into an integer or long data type number. The possible ways of numeric rounding are discussed below.

### Rounding down

The simplest form of rounding is truncation. Any digits after the desired precision are ignored and dropped. The CitectVBA `Fix()` function is an example of truncation. For example, `Fix(3.5)` is 3, and `Fix(-3.5)` is -3.

The `Int()` function rounds down to the highest integer less than the value. Both `Int()` and `Fix()` act the same way with positive numbers (truncating), but give different results for negative numbers: `Int(-3.5)` gives -4.

The `Fix()` function is an example of symmetric rounding because it affects the magnitude (absolute value) of positive and negative numbers in the same way. The `Int()` function is an example of asymmetric rounding because it affects the magnitude of positive and negative numbers differently.

### Rounding up

CitectVBA does not have a specific round-up function. However, for negative numbers, both `Fix()` and `Int()` can be used to round upward, in different ways:

- `Fix()` rounds towards 0 (up in the absolute sense, but down in terms of absolute magnitude). For example: Fix(-3.5) is -3.5.
- `Int()` rounds away from 0 (up in terms of absolute magnitude, but down in the absolute sense). For example: Int(-3.5) is -4.

### Arithmetic rounding

When continually rounding in one direction (down or up), the resulting number is not necessarily the closest to the original number. For example, if you round 1.9 down to 1, the difference is a lot larger than if you round it up to 2. It is easy to see that numbers from 1.6 to 2.4 should be rounded to 2. However, what about 1.5, which is equidistant between 1 and 2? By mathematical convention, the half-way number is rounded up.

To implement rounding half-way numbers in a symmetric fashion, -.5 is rounded down to -1, or in an asymmetric fashion, where -.5 is rounded up to 0.

CitectVBA does not have a function for arithmetic rounding.

### Banker's rounding

When you add rounded values together, always rounding .5 in the same direction results in a bias that grows with the more numbers you add together. One way to minimize the bias is with banker's rounding.

Banker's rounding rounds .5 up sometimes and down sometimes. The convention is to round to the nearest even number, so that both 1.5 and 2.5 round to 2, and 3.5 and 4.5 both round to 4. Banker's rounding is symmetric.

In CitectVBA, the `CByte()`, `CInt()`, `CLng()`, `CCur()`, and `Round()` numeric functions perform banker's rounding.

#### Random rounding

Even banker's rounding can bias totals. You can take an extra step to remove bias by rounding .5 up or down in a truly random fashion. This way, even if the data is deliberately biased, bias might be minimized. However, using random rounding with randomly distributed data might result in a larger bias than banker's rounding. Random rounding could result in two different totals on the same data.

CitectVBA does not have a function for random rounding.

#### Alternate rounding

Alternate rounding is rounding between .5 up and .5 down on successive calls. CitectVBA does not have a function for alternate rounding.

**See Also**

Numbers

## Date and Time Handling

CitectVBA has several pre-defined date and time functions that return date/time values. There are three functions enabling you to determine the current date and time set in Windows. The `Now` function, `Date` function, and `Time` function check your system clock and return all or part of the current setting.

To retrieve the date portion of a date/time value, use the pre-defined `DateValue` function. This function takes in either a string or a date value and returns only the date portion.

Using `DateValue`, you can compare the date portion of a date variable to a specific date value, like this:

```
If DateValue(dtmSomeDate) = #5/14/70# Then
' You know the date portion of dtmSomeDate is 5/14/70
End If
```

If you need just the time portion of a date variable, use the `TimeValue` function. Using this function, you could write code that checks the time portion of a date variable against a particular time, like this:

```
If TimeValue(dtmSomeDate) > #1:00 PM# Then
' You know the date variable contained a date portion
' with a time after 1:00 PM.
End If
```

You can perform arithmetic or mathematics (math) on date/time values because CitectVBA stores dates internally as serial values. Adding or subtracting integers adds or subtracts days, while adding or subtracting fractions adds or subtracts time. Therefore, adding 20 to a date value in CitectVBA adds 20 days, while subtracting 1/24 subtracts one hour. For example, to get tomorrow's date, you could just add 1 to today's date, like this:

```
dtmTomorrow = Date()+ 1
```

`Date` is a built-in CitectVBA function that returns the date portion (the integer part) of the current date and time retrieved from the Windows operating system. Adding 1 to that value returns a date that represents the next day.

The same mechanism works for subtracting two dates. Although CitectVBA supplies the `DateDiff` function for finding the interval spanned by two date/time values, if you just need to know the number of days between the two dates, you can simply subtract one from the other.

**See Also**

Date and Time Functions

Date Constants

Formatting Date Values

CitectVBA Function Reference

## Date Constants

You can use date/time literals in CitectVBA code by enclosing them with the hash sign (#), in the same way you enclose string literals with double quotation marks (""). This is commonly known as declaring date constants.

For example, #2/6/10# represents the Australian date value of 2nd June, 2010 if the short date setting for the locale was set to d/MM/yyyy. The same date constant would represent the American date value of February 6, 2010 if the short date setting for the locale was set to MM/d/yyyy. See Formatting Date Values.

**Note:** The system locale settings are determined using Regional Settings in Windows Control Panel.

Similarly, you can compare a date/time value with a complete date/time literal:

```
If SomeDate > #3/6/99 1:20pm# Then
```

If you don't include a time in a date/time literal, CitectVBA sets the time part of the value to midnight (the start of the day). If you don't include a date in a date/time literal, CitectVBA sets the date part of the value to December 30, 1899.

CitectVBA accepts a wide variety of date and time formats in literals. These are all valid date/time values:

```
SomeDate = #3-6-93 13:20#
SomeDate = #March 27, 1993 1:20am#
SomeDate = #Apr-2-93#
SomeDate = #4 April 1993#
```

In the same way that you can use the IsNumeric function to determine if a Variant variable contains a value that can be considered a valid numeric value, you can use the `IsDate` function to determine if a variant contains a value that can be considered a valid date/time value. You can then use the `CDate` function to convert the value into a date/time value.

For example, the following code tests the Text property of a text box with `IsDate`. If the property contains text that can be considered a valid date, CitectVBA converts the text into a date and computes the days left until the end of the year:

```
Dim SomeDate, daysleft
If IsDate(Text1.Text) Then
SomeDate = CDate(Text1.Text)
daysleft = DateSerial(Year(SomeDate) + _
1, 1, 1) - SomeDate
Text2.Text = daysleft & " days left in the year."
Else
MsgBox Text1.Text & " is not a valid date."
End If
```

**See Also**

Date and Time Functions

Formatting Date Values

CitectVBA Function Reference

Date Handling

## Formatting Date Values

Date values in CitectVBA can be formatted and displayed as strings containing any combination of the following lists of values, all of which are case-sensitive.

Most CitectVBA date and time functions are locale-aware and recognise the order for day, month, and year according to the short date format in the Regional Settings section of Windows Control Panel.

> **Note:** Always use long date format whenever possible. Also, please ensure that you enter and display dates in an unambiguous format. For example, dates in short date format might be misinterpreted in queries if the year or the day of the month are 12 or less (for example, 3/11/10). Dates in medium date format display only the first few characters of the month name, which can create ambiguity or an undesirable appearance.

### Text

All strings should be surrounded by single quotes, and any single quotes should be entered as four single quotes in a row:

| Value | Example |
|---|---|
| it''''s | it's |
| 'Today is 'M/dd/yy' and it''''s 'h:mm | Today is 01/22/99 and it's 8:18 |

### Day

The day can be displayed in one of four formats using a lowercase "d".

| Value | Meaning | Example |
|---|---|---|
| d | Day of the month as digits without leading zeros for single digit days. | 5 |
| dd | Day of the month as digits with leading zeros for single digit days. | 05 |
| ddd | Day of the week as a three letter abbreviation. | Mon |
| dddd | Day of the week as its full name. | Monday |

**Month**

The month can be displayed in one of four formats using capital "M". The letter "M" must be uppercase to distinguish months from minutes.

| Value | Meaning | Example |
|---|---|---|
| M | Month as digits without leading zeros for single digit months. | 1 |
| MM | Month as digits with leading zeros for single digit months. | 01 |
| MMM | Month as a three letter abbreviation. | Jan |
| MMMM | Month as its full name. | January |

**Year**

The year can be displayed in one of three formats using lowercase "y"..

| Value | Meaning | Example |
|---|---|---|
| y | Year represented only by the last digit, if the year is less than 10. Years greater than 10 will be given the value of yy. | 9 |
| yy | Year represented only by the last two digits. | 09 |
| yyyy | Year represented by the full 4 digits. | 1909 |

**Period/Era**

The period/era string can be displayed in a single format using the letter "g". The letter "g" must be lowercase. If you include the gg in a date string that does not have any associated Era string, the gg is ignored.

| Value | Meaning |
|---|---|
| (Null) | Gregorian dates are used. Does nothing if Gregorian is value of iCalendarType |
| gg | Period/era string. This is used by Windows to calculate the year when an optional calendar is selected. See iCalendarType for optional Calendars. |

**Time**

The time can be displayed in one of many formats using the letter "h" or "H" to denote hours, the letter "m" to denote minutes, the letter "s" to denote seconds and the letter "t" to denote the time marker. The lowercase "h" denotes the 12 hour clock and uppercase "H" denotes the 24 hour clock. The "m" must be lowercase to denote minutes as opposed to Months. The "s" for seconds and "t" for the time marker string must also be lowercase.

| Value | Meaning | Example |
|---|---|---|
| h | Hours without leading zeros for single digit hours (12 hour clock). | 1 |
| hh | Hours with leading zeros for single digit hours (12 hour clock). | 01 |
| H | Hours without leading zeros for single digit hours (24 hour clock). | 1 |
| HH | Hours with leading zeros for single digit hours (24 hour clock). | 01 |
| m | Minutes without leading zeros for single digit minutes. | 9 |
| mm | Minutes with leading zeros for single digit minutes. | 09 |
| s | Seconds without leading zeros for single digit seconds. | 5 |
| ss | Seconds with leading zeros for single digit seconds. | 05 |
| t | One character time marker string. This will be the first letter of the values in the AM symbol or PM symbol boxes in Regional Options | A |
| tt | Multi-character time marker string. This will be values in the AM symbol or PM symbol boxes in Regional Options | AM |

**See Also**

Date Handling

Date Constants

Date and Time Data Constraints

## Date and Time Data Constraints

CitectVBA has several constraints on date and time values based upon the Gregorian Calendar:

- The value of the month field must be between 1 and 12, inclusive.
- The value of the day field must be in the range from 1 through the number of days in the month. The number of days in the month is determined from the values of the year and months fields and can be 28, 29, 30, or 31. (The number of days in the month can also depend on whether it is a leap year.)
- The value of the hour field must be between 0 and 23, inclusive.
- The value of the minute field must be between 0 and 59, inclusive.
- For the trailing seconds field of interval data types, the value of the seconds field must be between 0 and 59.9(*n*), inclusive, where *n* is the number of digits in the fractional seconds precision.
- For the trailing seconds field of datetime data types, the value of the seconds field must be between 0 and 61.9(*n*), inclusive, where *n* specifies the number of "9" digits and the value of *n* is the fractional seconds precision. (The range of seconds allows as many as two leap seconds to maintain synchronisation of sidereal time.)

**See Also**

Date Handling

Date Constants

Formatting Date Values

Date Data Type Structure

Dates in Databases Using Different Calendars

## Date Data Type Structure

The date data type structure in CitectVBA is a junction of two very different methods of data storage. It is formed from the serial concatenation of a date-value followed by a time-value, separated by a floating point, and stored in an 8-byte floating-point date data type value.

- The integer (whole) portion of the value represents the number of days elapsed since December 30, 1899.
- The remainder (fractional) portion of the value represents the elapsed portion of the day since midnight.

The integer (date-value) portion (to the left of the floating point) and the remainder (time-value) portion (to the right of the floating point) must be handled differently as they are structured very differently. CitectVBA has a number of pre-defined date and time functions to convert between the internal floating-point date data type format and visibly recognisable dates and times.

**Note:** Dates in CitectVBA are based upon the Gregorian Calendar.

For example, the date and time of 5/22/97 at 3:00 p.m. would be stored in CitectVBA as 35572.625 representing the 35572 days since 12/30/1899, and 3:00 p.m. as 625/1000 of a full day.

**Note:** Don't confuse Date data types used in CitectVBA with date and time values used in Windows, DLLs, Vijeo Citect, or in Cicode. For instance, Vijeo Citect stores time/date-related variables as a single integer representing the number of seconds since 01/01/1970.

## Date-values

A date-value in CitectVBA is a count of the number of days from December 30, 1899. December 31, 1899 has the date-value of 1, and the 1st January 1900 is 2. December 30, 1899 has the date value of zero. Negative date-values represent dates prior to December 30, 1899.

A date-value in CitectVBA can actually range from January 1, 0100, to December 31, 9999 inclusive, which is a integer value ranging from -657434 up to +2958465 respectively. Using values outside this range will cause compilererrors in CitectVBA.

The pre-defined CitectVBA `Year`, `Month`, and `Day` functions calculate and return the appropriate year, month or day value (as an integer) from a date-value.

### Time-values

A time-value in CitectVBA represents the fractional time of day since the previous midnight. Unlike a date-value which is simply a count of the number of days, the time-value is a decimal fraction of a day.

As every day invariably consists of 24 hours, or 1440 minutes, or 86,400 seconds, the time of day can be readily determined from a time-value using simple math. An hour has the time-value of one twenty-fourth of one day (0.0416'), one minute has the time-value of 0.000694', and a second has the time-value of 0.000011574'0'7'. Midday has the time-value of 0.50. 1AM has the time-value of 0.0416'. 1PM has the time value of 0.5416'.

The pre-defined CitectVBA `Hour` function, `Minute` function, and `Second` function calculates and returns the appropriate hour, minute or second value (as an integer) from a time-value.

**See Also**

Date Handling

Date Constants

Formatting Date Values

Dates in Databases Using Different Calendars

Date and Time Data Constraints

## Dates in Databases Using Different Calendars

If you use an existing database with date references of a different calendar type than your current operation system locality settings, CitectVBA could report a variety of errors or perform in an unexpected manner at runtime. For example, if you have used Hijri Calendar dates in your database, a syntax error message will occur if CitectVBA makes reference to Gregorian dates that are invalid Hijri dates (for example, the 31st of any month will produce a syntax error because no Hijri month has 31 days).

**⚠ WARNING**

**UNINTENDED EQUIPMENT OPERATION**

Always confirm that calendar types in existing databases are compatible with the locality (regional and language) options of the operating system.

**Failure to follow these instructions can result in death, serious injury, or equipment damage.**

To avoid problems of this sort, all date references in an external database should be based on the Gregorian Calendar, or the database tables can be exported to text files before use in Vijeo Citect. Dates in Microsoft Access database tables exported as text files are always stored as Gregorian values. If the database calendar is set to Hijri for example, automatic Hijri to Gregorian conversion is performed during the export process.

You can't set a database calendar programmatically using CitectVBA.

**Note:** When you want to use characters for Baltic, Central European, Cyrillic, Greek, Turkish, and Asian languages, or right-to-left languages (Arabic, Hebrew, Farsi, and Urdu) the operating system would need to have the corresponding language version of Windows, or have installed system support for that language.

**See Also**

Date Handling

## Operators

Variables can be manipulated in CitectVBA using assignment, arithmetic, relational, and logical operators.

- The assignment operator is used to assign a value to a variable or constant (that equals this).
- Arithmetic operators are used to mathematically manipulate numeric variables and numbers (addition, subtraction, multiplication, division, etc.).
- Relational Operators are used to compare the relationship between variables (less than, greater than, not equal to, etc.).
- Logical operators are used to perform digital logic operations on variables ( AND, OR, NOT, etc.).

When using multiple operators in a CitectVBA statement, you need to ensure the proper execution of your code by observing order of precedence rules.

The string concatenation operator is used to join strings together.

**See Also**

Constants

Variables

Numbers

Strings

Date Handling

CitectVBA Function Reference

### Assignment Operator

The CitectVBA assignment operator uses the equals character ( = ) in a CitectVBA statement. The variable named to the left side of the assignment operator is assigned the operand value from the right side of the assignment operator, as shown here:

```
' declares integer variable named X
Dim X As Integer
' declares integer variable named Y
Dim Y as Integer
X = 123    ' assigns numeric value 123 to variable X
Y = X    ' assigns value of variable X to variable Y
```

Only one variable can be assigned at any one time with the assignment operator.

There must be a space on either side of the assignment operator, or the equals character may be confused with either the variable name or the value being assigned, and a compile error may occur.

Unless the variable is a variant data type, the value being assigned must be the same data type as the variable receiving the assigned value. For instance, if you assign a text string into a long data type variable, you'll cause an error to occur.

The variable must be previously declared before being assigned a value. The value of a variable can be changed any number of times in a later statements, as in the following CitectVBA example:

```
' declare integer variable named X
' and assign an initial numeric value of 123 to it
Dim X = 123 As Integer
' <statement>
' <statement>
' <statement>
' reassign X to store the numeric value 456
X = 456
' <statement>
' <statement>
' <statement>
' reassign X to store the numeric value 789
X = 789
```

**See Also**

Operators

## Arithmetical (Math) Operators

CitectVBA arithmetic operators are used in CitectVBA statements to mathematically manipulate numeric variables and numbers. The resultant is often assigned to a third variable using the assignment operator.

The arithmetic operation as determined by the arithmetic operator is performed between the values of the operands (variables or numbers positioned immediately on either side of the arithmetic operator).

| Operator | Function | Usage |
|---|---|---|
| ^ | Exponentiation | X = Y ^ 2 |
| - | Negation | X = - 2 |
| * | Multiplication | X = 2 * 3 |

| | | |
|---|---|---|
| / | Division | X = 10 / 2 |
| Mod | Modulo | X = Y MOD Z |
| + | Addition | X = 2 + 3 |
| - | Addition | X = 6 - 4 |

**See Also**

Operators

## Relational Operators

CitectVBA Relational Operators are used in CitectVBA statements to compare the relationship between operands (values positioned immediately on either side of the Relational Operator). The boolean result is either True or False. .

| Operator | Function | Usage |
|---|---|---|
| < | Less than | X < Y |
| <= | Less than or equal to | X <= Y |
| = | Equals | X = Y |
| >= | Greater than or equal to | X >= Y |
| > | Greater than | X > Y |
| <> | Not equal to | X <> Y |

**See Also**

Operators

## Logical Operators

Logical (boolean) operators are used to perform digital logic operations on variables. All logical operations result in either a boolean True or False.

| Operator | Function | Usage |
|---|---|---|
| Not | logical negation | if not X |
| And | logical And | If (X> Y) **And** (X < Z) |
| Or | logical Or | If (X = Y) **Or** (X = Z) |

**See Also**

Operators

## Operator Precedence

The operator precedence in CitectVBA runs like this:

| Operator | Description | Order |
|---|---|---|
| ( ) | Parenthesis | Highest |
| ^ | Exponentiation | |
| - | Unary minus | |
| /, * | division/multplication | |
| Mod | Modulo | |
| +, -, & | addition, subtraction, concatenation | |
| =, <>, <, >,<=,>= | Relational | |
| Not | Logical negation | |
| And | Logical conjunction | |
| Or | logical disjunction | |
| Xor | logical exclusion | |
| Eqv | logical equivalence | |

| | | |
|---|---|---|
| Imp | logical implication | Lowest |

**See Also**

Operators

## Strings

Strings can be stored in variables of `string` and `variant`. When using `variant` strings, be aware of type coercion in CitectVBA. Strings can be compared with each other in CitectVBA to determine whether they contain the same characters or not. Strings can be joined together to create longer strings in CitectVBA using the concatenation operator.

Strings can be searched using the:

- `InStr()` function, which returns the character position of the first occurrence of a string within another;
- `Left()` function or `Right()` function which return a copy of the left or right most characters of a string respectively; and
- `Mid()` function, which returns the copy of a substring from within another string.

To determine the length of a string, use the `Len()` function which returns a Long variable containing the number of characters in the string.

String characters can be converted to ASCII values using the `Asc()` function, and ASCII values can be converted to their string characters using the `Chr()` Function.

String characters can be converted to all lowercase or all uppercase using the `Lcase()` Function or the `Ucase()` Function respectively.

Leading or trailing spaces can be stripped off strings using the `Ltrim()` function or the `Rtrim()` function respectively.

Strings can be created consisting of a specified number of spaces or characters using the `Space()` function or the `String()` function respectively.

For syntax details of using string functions, see String Functions.

**See Also**

Operators

Numbers

Control Structures

## String Comparisons

CitectVBA compares ANSI values of characters when comparing strings. For example, the character capital 'A' has the ANSI value of 65, and the character lowercase 'a' has the ANSI value of 97. For a listing of ANSI characters values, see ASCII/ANSI Character Code Listings.

You can use CitectVBA relational operators (less than, greater than, equal to, not equal to, and so on) to compare string variables. All relational operators return either `true` or `false`.

With comparisons made using relational operators, the result depends on the `option compare` string-comparison option setting of the CitectVBA file. Consider the following example:

```
"Citectvba" > "CitectVBA"
```

If the file Option string-comparison setting is Option Compare Binary (or not set at all), the comparison returns `true`. CitectVBA compares the binary (ANSI) values for each corresponding position in the string until it finds two that differ. In this example, the lowercase letter "v" corresponds to the ANSI value 118, while the uppercase letter "V" corresponds to the ANSI value 86. Because 118 is greater than 86, the comparison returns `true`.

If the file Option string-comparison setting is Option Compare Text, "Citectvba" > "CitectVBA" returns `false`, because the strings are equivalent apart from case.

The built-in CitectVBA `StrComp()` Function returns a variant containing a value representing the result of the comparison of two strings. It has an optional third argument Comp which can override the file Option string-comparison setting.

### See Also

Option Compare statement

Strings

## String Concatenation

To concatenate strings in CitectVBA, use the ampersand ( & ) concatenation operator between the strings. Multiple concatenations can occur in the same CitectVBA statement.

```
Dim strFirstName As String
Dim strLastName As String
Dim strFullName As String
Const strSpaceChar = " "
' note the space character between the quotes
```

```
strFirstName = "Colin"
strLastName = "Ramsden"
strFullName = strFirstName &strSpaceChar &strLastName
' concatenates string values
```

The & concatenation operator does not perform arithmetic, and will convert variable data types to strings for concatenation. For instance, if a variant string and a variant number are concatenated, the result is a string. For more details of variant data types, see Variant Declaration and CitectVBA Data Types.

**See Also**

Strings
Operators
Control Structures

## Control Structures

CitectVBA provides conditional control functionality, which can be used to conditionally perform CitectVBA statements or blocks of statements dependant upon the result of the condition tested. This is known as logical decision making.

The logical decision making control structures available in CitectVBA consist of three conditional looping or repetitive statements (Do Statement, While Statement, and For Statement), and two conditional flow control sequence statements (Select Case Statement, and variations of the If Statement). In addition, CitectVBA provides one unconditional branching GoTo Statement.

**Note:** In the control structure syntax examples, every placeholder shown inside arrow brackets ( <placeholder> ) should be replaced in any actual code with the value of the item that it describes. The arrow brackets and the word they contain should not be included in the statement, and are shown here only for your information. Also, statements shown between square brackets ( [ ]) are optional. The square brackets should not be included in the statement, and are shown here only for your information.

**See Also**

Operators

GoTo statement

Do statement

While statement

For statement

If statement

Select case statement

End statement

Exit statement

OnError statement

Stop statement

With statement

CitectVBA Function Reference

## GoTo statement

The GoTo conditional statement branches unconditionally and without return to the label specified in the GoTo statement. The label must be located in the same subroutine or function as the Goto statement.

```
<statement/s>
If <condition> then
    GoToLabel1
Else
    GoToLabel2
End If
Label1:
<statement/s>
    GoToLabel3
Label2:
<statement/s>
    GoToLabel3
Label3:
<statement/s>
```

In this example, CitectVBA tests the If condition, and jumps to the part of the script that begins with the label "Label1:" if the condition was true, or jumps to the part of the script that begins with the label "Label2:" if the condition was false. This could be anywhere in the same subroutine or function.

### See Also

Control Structures

## Do statement

The Do...Loop conditional statement allows you to execute a block of statements an indefinite number of times. The variations of the Do...Loop are Do While, Do Until, Do Loop While, and Do Loop Until.

```
Do While|Until <condition>
    <statement/s>
Loop

Do Until <condition>
    <statement/s>
Loop

Do
    <statement/s>
Loop While <condition>

Do
    <statement/s>
Loop Until <condition>
```

Do While and Do Until check the condition before entering the loop, thus the block of statements inside the loop are only executed when those conditions are met. Do Loop While and Do Loop Until check the condition after having executed the block of statements so that the block of statements is executed at least once.

Any Do statement can be exited using the Exit Do statement.

**See Also**

Control Structures

## While statement

The While...Wend loop conditional statement is similar to the Do While loop statement. The condition is checked before executing the block of statements comprising the loop.

```
While <condition>
    <statement/s>
Wend
```

**See Also**

Control Structures

## For statement

The For...Next loop conditional statement repeats its block of statements a set number of times as determined by the values used with the To clause.

```
For <CounterName> = <BeginValue> To <EndValue> [Step <StepValue>]
    <statement/s>
Next
```

The counter variable is increased or decreased (by the value stated in the optional Step parameter), with each reiteration of the loop. The counter default is to increment by one if the Step parameter is omitted.

**See Also**

Control Structures

## If statement

The If statement in CitectVBA tests an initial condition and then either performs or omits to perform the statements it contains, dependant upon the logical result of the test condition. The condition can be a comparison or an expression, and must logically evaluate to either True or False. The If statement has both single line and multiple line syntax structure.

The single line syntax uses the If <TestCondition> Then <StatementToPerformIfTrue> structure, however, can only perform a single statement if and only if the test condition result is True. No 'End If' statement is required:

```
If <Condition> Then <Statement>
```

If the result of the If test condition was True, the program flow continues into and performs the statement following the Then statement, until it reaches the end of the line.

To perform a single statement conditionally upon a False result, use the NOT logical operator:

```
If NOT <Condition> Then <Statement>
```

To perform multiple statements, use the multiple line syntax structure which ends with the 'End If' statement:

```
If <Condition> Then
' Then statement block
' perform only if true
<Statement/s>
End If
```

If the result of the If test condition was True, the program flow continues into the Then statement block, and performs the statements following the Then statement, until it reaches the End If statement.

If the result of the If test condition was False, the program flow jumps over the Then statement block, which in this case exits the If structure (without performing any statements other than the initial test condition).

The mutiple line If structure can perform different blocks of statements dependant upon EITHER a True OR a False result to the test condition, through the use of the Else statement block:

```
If <Condition> Then
' Then statement block
' perform only if true
<Statement/s>
Else
' Else statement block
' perform only if false
<Statement/s>
End If
```

If the result of the If test condition was True, the program flow performs the Then block statements, until it reaches the Else statement. It then jumps over the Else statement block and exits the If structure (without performing any of the Else statement block statements).

If the result of the If test condition was False, the program flow jumps over the Then statement block (without performing any of those statements) to the Else statement to perform the statements in the Else statement block until it reaches the End If statement.

Further test conditions can be placed into an If structure through the use of the optional Else If <Condition> statement block. ElseIf statement blocks can only be positioned within an If structure before the Else statement block.

```
If <Condition> Then
' Then statement block
' perform only if true
<Statement/s>
ElseIf <Condition>
' Else If statement block
' perform only if true
<Statement/s>
Else
' Else statement block
' perform only if false
<Statement/s>
End If
```

The ElseIf test condition is only evaluated after the initial If structure test condition results in False.

If the result of the ElseIf test condition was True, the statements within the ElseIf statement block are performed. The program flow then jumps over the Else statement block and exits the If structure (without performing any of the Else statement block statements).

If the result of the ElseIf test condition was False, the program flow jumps over the ElseIf statement block (without performing any of those statements) to the Else statement to perform the statements in the Else statement block until it reaches the End If statement.

There is no apparent limit to the number of Else If statement blocks that any one If structure can hold, however, the Select Case Statement structure handles multiple condition result alternatives much more efficiently.

**See Also**

Control Structures

## Select case statement

The Select Case statement tests the same variable for many different conditions. The test value provided with the initial Select Case statement is logically tested against the Case test condition.

The Select Case structure can perform different blocks of statements dependant upon whichever Case statement test condition (if more than one) first results as True, through the use of the Case statement block:

```
Select Case <TestValue>
Case <Condition>
' Case statement block
' perform only if case true
<Statement/s>
Case Else
' Else statement block
' perform only if all cases false
<Statement/s>
End Select
```

If the result of the Case test condition was True, the program flow performs the statements contained within that Case statement block, and will then exit the Select Case structure (without performing any of the Else statement block statements).

If the result of the Case test condition was False, the program flow jumps over the Case statement block (without performing any of those statements) to the Case Else statement to perform the statements in the Else statement block until it reaches the End Select statement.

Further test conditions can be placed into a Select Case structure through the optional use of further Case statement blocks. Case statement blocks can only be positioned within a Select Case structure before the Case Else statement block.

```
Select Case <TestValue>
Case <Condition>
```

```
' Case statement block
' perform only if case true
<Statement/s>
Case <Condition>
' Case statement block
' perform only if case true
<Statement/s>
Case Else
' Else statement block
' perform only if all cases false
<Statement/s>
End Select
```

Each Case statement block is evaluated in order until the test condition of one results as True. The program flow performs the statements contained within that Case statement block, and will then exit the Select Case structure (without performing any other statements).

The statements of ONLY one Case statement block are ever performed, unless all result in False and there is no Case Else block declared, in which case no Case statement blocks are performed at all.

The following example may help clarify the logic testing being performed in a Select Case structure. Lets say that we have a variable named (intDayOfWeek) containing an integer (ranging from 1 to 7) representing the day of the week, and we wished to display that value as a string (named strDayOfWeek) containing the name of the day of the week, assuming in this example, that Sunday is the first day of the week (1). The Select Case structure would look like this:

```
Dim strDayOfWeek As String
Select Case intDayOfWeek
Case = 1
StrDayOfWeek = "Sunday"
Case = 2
StrDayOfWeek = "Monday"
Case = 3
StrDayOfWeek = "Tuesday"
Case = 4
StrDayOfWeek = "Wednesday"
Case = 5
StrDayOfWeek = "Thursday"
Case = 6
StrDayOfWeek = "Friday"
Case = 7
StrDayOfWeek = "Saturday"
Case Else
StrDayOfWeek = "Invalid"
End Select
```

The Select Case structure tends to be easier to read, understand, and follow and should be used in place of a complicated multi-nested If...ElseIf structure.

**See Also**

Control Structures

## End statement

The End statement Ends a block of statements such as a Sub procedure or Function.

```
End[{Function | If | Sub}]
```

**Example**

```
Dim Var1 as String
Var1 = "hello"
' Calling Test
Test Var1
MsgBox Var1

Sub Test(wvar1 as string)
MsgBox wvar1
wvar1 = "goodbye"
End
End Sub
```

**See Also**

Control Structures

## Exit statement

Exits a loop or procedure

```
Exit {Do | For | Function | Sub }
```

**Example**

```
' This sample shows Do ... Loop with Exit Do to get out.
Dim Value, Msg    ' Declare variables
Do
Value = InputBox("Enter a value from 5 to 10.")
If Value >= 5 And Value <= 10 Then    ' Check range
Exit Do    ' Exit Do...Loop
Else
Beep    ' Beep if not in range
End If
```

```
Loop
```

**See Also**

Control Structures

## OnError statement

CitectVBA's error-handling routine and specifies the line label of the error-handling routine. The line parameter refers to a label. That label needs to be present in the code or an error is generated.

**Syntax**

**On Error { GoTo line | Resume Next | GoTo 0 }**

**Example**

```
On Error GoTo errHandler
Dim x as object
x.draw     ' Object not set
..
Exit Sub
errHandler:
Print Err.Number, Err.Description
Resume Next
```

**See Also**

Control Structures

## Stop statement

Ends execution of the program. The Stop statement can be placed anywhere in your code.

**Example**

```
Dim x,y,z
For x = 1 to 5
For y = 1 to 5
For z = 1 to 5
Print "Looping",z,y,x
Next z
Next y
Stop
Next x
```

**See Also**

Control Structures

### With statement

The With Statement is not supported in CitectVBA. When performing a series of commands on an object, you must explicitly refer to the name of the object with each command.

**See Also**

Control Structures

## Subroutines and Functions

Commonly used sequences of CitectVBA statements can be grouped together into functions and subroutines, so that they can be keyed in once, and used many times in many places, by 'calling' the name of the subroutine or function in code.

A subroutine or function is a group or list of sequential statements that CitectVBA can perform (execute) in the logical order that they exist within the subroutine or function from top to bottom in the order they are listed within the function or subroutine.

If the group of statements returns a value, it must be declared as a function. If it does not return a value, it must be declared as a subroutine. A subroutine or function is called by placing the name of the subroutine or function in a code statement where you want the action of the subroutine or function to occur.

**Note:** Subroutines and functions can contain statements that call other subroutines or functions (to perform, before returning to the following statements within the calling subroutine or function).

Both subroutines and functions can similarly be passed values as arguments when they are called:

- Arguments are passed to **subroutines** in CitectVBA code following the subroutine name and separated by space characters.
- Arguments are passed to **functions** enclosed within parentheses in CitectVBA code, similarly following the subroutine name and separated by space characters.

**Note:**Vijeo Citect tag values must be declared by value when passed as argument

values to a CitectVBA procedure from within a Vijeo Citect command or expression field (see Passing variables Byref and Byval).

**See Also**

Subroutines

Functions

Arguments

## Subroutines

A CitectVBA subroutine starts with the SUB statement and finishes with the END SUB statement. All other statements that lie between the SUB and END SUB statements, will be executed by the subroutine, when called to do so.

**Note:** In the following subroutine syntax example:

- Every placeholder shown inside arrow brackets ( **<placeholder>**) should be replaced in any actual code with the value of the item that it describes. The arrow brackets and the word they contain should not be included in the statement, and are shown here only for your information.
- Statements shown between square brackets ( **[ ]**) are optional. The square brackets should not be included in the statement, and are shown here only for your information.

In CitectVBA, Subroutines are created with the SUB statement in the following format.

```
Sub <SubName> ( [ Byval ] [ <Argument/s> ] [ <As Data Type> ])
<statement>
<statement>
<statement>
End Sub
```

where:

- **[Byval]** is the optional parameter for the argument;
- **Sub** is the required subroutine statement basic keyword
- **<SubName>** represents the required name of the subroutine being created
- **<Argument/s>** represents the optional argument/s of the subroutine
- **<statement>** represents the executable CitectVBA script statement/s
- **End Sub** is the subroutine terminating statement

The name given to the subroutine immediately follows the SUB keyword, and is used to identify the subroutine to CitectVBA. This name is referred to when the subroutine is called upon (called) to be executed (perform the statements it contains) by some other procedure in CitectVBA.

Subroutine names can contain the letters 'A' to 'Z' and 'a' to 'z', the underscore '_' and digits '0' to '9'. The subroutine name must begin with a letter, be no longer than 40 characters, cannot contain the space character, and cannot be a reserved word. Subroutine names (once declared), become a keyword in CitectVBA. Like most keywords in CitectVBA, these names are not case sensitive.

The subroutine name always ends with a pair of parentheses ( ) which may or may not contain one or more arguments required by (necessary for use in) the subroutine . Multiple arguments if used, are separated by commas ( , ). See Arguments for more details and argument syntax.

All the lines located between the SUB and the END SUB statements, contain the statements that will be executed when the subroutine is called in CitectVBA. These statements will be executed one at a time in logical order from top to bottom within the subroutine.

**See Also**

Subroutines and Functions

Functions

Arguments

## Functions

A CitectVBA function starts with the FUNCTION statement and finishes with the END FUNCTION statement. All other statements that lie between the FUNCTION and END FUNCTION statements, will be executed by the function, when called to do so.

**Note:** In the following function syntax example:

- Every placeholder shown inside arrow brackets ( **<placeholder>**) should be replaced in any actual code with the value of the item that it describes. The arrow brackets and the word they contain should not be included in the statement, and are shown here only for your information.
- Statements shown between square brackets ( **[ ]**) are optional. The square brackets should not be included in the statement, and are shown here only for your information.

A typical CitectVBA function is structured like in the following example:

```
Function <FunctionName> ( [ Byval ] [ <Argument/s> ] ) [ As <ReturnDataType> ]
```

```
<statement>
<statement>
<statement>
[ <FunctionName> = <value> ]
End Function
```

where:

- Function' is the required function statement basic keyword
- [ Byval ] is the optional parameter for the argument;
- <FunctionName> represents the required name of the function being created
- ( <Argument/s> ) represents the optional argument/s of the function
- <ReturnDataType> represents the optional return data type of the function
- <statement> represents the executable CitectVBA script statement/s
- = <value> represents the optional assignment of the return value for the function
- 'End Function' is the function terminating statement

The name given to the function, immediately follows the FUNCTION keyword, and is used to identify the function to CitectVBA. This name is referred to when the function is called upon (called) to be executed (perform the statements it contains) by some other procedure in CitectVBA.

Function names can contain the letters 'A' to 'Z' and 'a' to 'z', the underscore '_' and digits '0' to '9'. The function name must begin with a letter, be no longer than 40 characters, cannot contain the space character, and cannot be a reserved word. Function names (once declared), become a keyword in CitectVBA. Like most keywords in CitectVBA, these names are not case sensitive.

The function name always ends with a pair of parentheses ( ) which may or may not contain one or more arguments required by (necessary for use in) the function. Multiple arguments if used, are separated by commas ( , ). See the section titled 'Arguments in CitectVBA' for more details and argument syntax.

All the lines located between the FUNCTION and the END FUNCTION statements, contain the statements that will be executed when the function is called in CitectVBA. These statements will be executed one at a time in logical order from top to bottom within the function.

The return value of the function is optionally assigned within the function in a statement using the function name. This value is often used within the calling procedure to determine the status of the function. Commonly, this value may be a Boolean True or False to indicate the successful completion or not of the function.

### See Also

Subroutines and Functions
Arguments
Subroutines
Accessing Functions in DLLs

## Arguments

Arguments are used in CitectVBA to pass values into subroutines and functions when they are being called. Arguments are positioned between parentheses '( )' immediately after the subroutine or function name in the subroutine or function declaration. If no arguments are required for the subroutine or function, the parentheses must be included and left empty in the declaration.

Arguments are optional in the sense that subroutines and functions do not require them. However, if arguments are to be used in a subroutine or function, the arguments must first be declared with the subroutine or function declaration, before they can be used. If declared, they must be used whenever the subroutine or function is called.

CitectVBA does NOT support named arguments so all arguments must be used in declaration order. If omitted, strings default to an empty string (""), and numeric values default to zero (0). Boolean values in CitectVBA are represented with -1 for TRUE, and 0 for FALSE.

Multiple arguments must be separated by a comma ( , ) placed between the arguments. The number of arguments that can be used in any single subroutine or function is not stated, (but likely limited to something like 255). If you are declaring a subroutine or function with that many arguments, you should probably split your subroutine or function into smaller separate logical routines with less arguments for each routine. If an argument is omitted, its place must be declared by the use of a comma in the call.

If you want to use the value in a Vijeo Citect tag as an argument to a function or subroutine, you must assign the value of the tag to a CitectVBA variable, and then pass the variable as the argument. You cannot pass a Vijeo Citect tag name as an argument to a function or subroutine.

Each argument declaration in a subroutine or function must be structured using the proper CitectVBA argument syntax as described below.

CitectVBA argument structure syntax in the declaration of functions or subroutines is as follows:

```
( [ Byval ] <Argument/s> [ As <DataType> ] )
```

where:

- [ Byval ] is the optional parameter for the argument.
- <Argument/s> represents the argument/s required by the function or subroutine.
- [ As <DataType> ] represents the optional data type declaration of the argument.

The optional 'Byval' parameter specifies that the variable is passed by value instead of by reference (see the section titled 'Passing Variables Byref and Byval with CitectVBA').

**Note:**Vijeo Citect tag values MUST be declared by value when passed as argument values to a CitectVBA procedure from within a Vijeo Citect command or expression field. This is best done by declaring a variable, assigning it the tag value, then passing the variable by value.

The function or subroutine name always ends with a pair of parentheses ( ) which may or may not contain one or more arguments required by (necessary for use in) the function or subroutine. Multiple arguments if used, are separated by commas ( , ).

The optional 'As <DataType>' parameter is used to specify the data type of the argument variable. The argument data types must be individually declared, or will be of Variant data type by default. Valid data types for arguments in CitectVBA are: String, Integer, Double, Long, and Variant (see the section titled 'CitectVBA_Data_Types' for descriptions of data types in CitectVBA).

**Example**

```
' Arguments are declared with the function or subroutine
' The function is called from the subroutine highlighted below
Function longArea(Byval longLength As Long, _
Byval longWidth As Long) As Long
' multiplies arguments and
' assigns result to return value
longArea = longLength * longWidth
End Function
Sub FindArea
' declare long variables X Y and Z
Dim longX As Long
Dim longY As Long
Dim longZ As Long

' assign numeric value 12 to variable X
X = 12
' assign numeric value 34 to variable Y
Y = 34

' call function named longArea,
' passing in values of X and Y variables
' as arguments
'store result in variable Z
Z = longArea(X, Y)
' copy result Z to tag
```

```
TestTag_1 = Z
End Sub
```

Granted, that's not likely the way you'd actually calculate an area given two fixed values in a subroutine that calls a function. You could just as easily do the calculation within the subroutine. However, this example does demonstrate the passing of values from a subroutine to a function, and the retrieval of a return value from the function back to the calling subroutine.

Note in the previous example, that the argument names ('longLength' and 'longWidth') are only used within the function in which they were declared. The values they represented were passed in with the call to the function in the statement line:

```
Z = longArea(X, Y)
```

The values of the variables 'X' and 'Y' were passed into the function 'longArea' and were handled within the function as its argument names 'longLength' and 'longWidth'. The result was returned and stored in the variable named 'Z'.

**See Also**

Subroutines and Functions

Subroutines

Functions

## DLLs and APIs

Dynamic Linked Libraries (DLLs) are files that contain functions which can be called from any application running under Microsoft Windows. At run time, a function in a DLL is dynamically linked into an application that calls it. No matter how many applications call a function in a DLL, that function exists in only a single file on the computer, and the DLL is loaded only once in memory.

An application programming interface (API) is a set of functions you can use to work with a component, application, or the operating system. Typically an API consists of one or more DLLs that provide some specific functionality.

For example, the Windows API includes the DLLs that make up the Windows Operating System (OS). Every Windows application interacts with the Windows API directly or indirectly. The Windows API is provided so that all applications running under Windows can behave in a consistent manner.

**Note:** Vijeo Citect itself provides an API for external access to Vijeo Citect I/O

variable tags via a DLL interface.

APIs are traditionally written for C and C++ programmers who are building Windows applications, however, the functions in a DLL can also be called by other programming languages, including CitectVBA. Because most DLLs are written and documented primarily for C/C++ programmers, calling a DLL function may differ somewhat from calling a CitectVBA function. In order to work with an API, you need to understand how to pass arguments from CitectVBA to a DLL function. See Passing Arguments to DLL Functions from CitectVBA.

**See Also**

Passing variables Byref and Byval

CitectVBA Function Reference

## Accessing Functions in DLLs

To be able to call and use an external DLL function using CitectVBA, you must have previously provided CitectVBA with details about the function being called. CitectVBA requires information like the name of the function, where that function is located, what arguments it expects, and what type of data it returns. CitectVBA uses the Declare statement to detail that information.

The Declare statement must be positioned in the CitectVBA file in your project above and before any code that calls that declared function of the DLL.

### Declare statement structure

The Declare statement consists of the required Declare keyword, followed by the required Function statement, the required Lib statement, the optional Alias statement, the optional Argument statement(s) contained within braces, and the optional return data type statement.

**Note:** The use of the OPTIONAL components of the **Declare** statement syntax indicates that they may not be required in all DLL functions. It is not up to you whether you can optionally use them or not. If included in a DLL function, they MUST be used when declaring that function to CitectVBA.

The Declare statement in CitectVBA details the name, file location, arguments, intrinsic constants, and type definitions that the DLL function requires. Here's an example of the Declare statement for the Windows API GetTempPathA function, which returns the path to the Windows system temporary folder:

```
Declare Function GetTempPathA Lib "kernel32" _
(Byval nBufferLength As Long, _
Byval lpBuffer As String) As Long
```

The Declare keyword indicates to CitectVBA that you intend to call a function belonging to an external DLL. The Declare keyword must be used first in the declaration statement.

**Declare - Function Statement**

The Function statement consists of the Function keyword, followed by the name that you will use when calling this function from CitectVBA.

**Declare - Lib Statement**

The Lib statement specifies which DLL contains the function you wish to use. The Lib statement consists of the Lib keyword, followed by the name of the DLL contained within string double quotes. Some commonly used DLLs in the Windows API for example, are Kernel32.dll - which performs low level OS functions like memory management and resource handling, the User32.dll - which performs Windows message handling, timers, menus and communication functions, and the GDI32.dll - which performs the graphics display and font management functions.

### Declare - Alias Statement

In the previous Declare statement example, the name of the declared function in CitectVBA is the same as the name of the actual function within the DLL. This does not necessarily have to be the case. There are some instances where the name of the function in the DLL is incompatible with the naming structure of CitectVBA, and cannot be used as a declared function name in CitectVBA. An example would be those DLL function names that start with an underscore.

To overcome such incompatibilities, the CitectVBA Declare statement supports the use of an alias name for the DLL function, through the use of the optional Alias statement . The Alias statement consists of the Alias keyword, followed by the actual name of the DLL function contained within string double quotes. The Alias statement must be positioned within the Declare statement between the Lib statement and the Argument statement.

Here's an example of the Declare statement for the Windows API GetTempPathA function as used above, however, this time using the optional Alias statement:

```
Declare Function GetWinTempPath Lib "kernel32" _
(Byval nBufferLength As Long, _    Alias "GetTempPathA" _
Byval lpBuffer As String) As Long
```

In this example, the name of the API function in the DLL is GetTempPathA, and the name by which you would call this function from CitectVBA is GetWinTempPath. Note that the actual name of the DLL function appears contained within string double quotes positioned after the Alias keyword. This instructs CitectVBA to use the alias function name when calling the DLL.

Because an alias allows you to name a declared DLL function anything you want in CitectVBA, you can make the function name conform to your own naming standards.

> **Note:** DLL functions are case sensitive; CitectVBA function names are not. When declaring DLL functions in CitectVBA, be careful to accurately remain case sensitive in the declaration.

**See Also**

Functions

Passing variables Byref and Byval

Passing Arguments to DLL Functions from CitectVBA

DLLs and APIs

### Passing variables Byref and Byval

Passing an argument by reference (using the Byref parameter) passes a pointer to the memory location of that argument. A pointer is just a memory address that indicates where the value is stored. If the procedure modifies that argument's value, it modifies the source of that argument, so when execution returns to the calling procedure, the source contains the modified value.

Passing an argument to a function by value (using the Byval parameter), on the other hand, passes a copy of the value as the argument. This prevents that function from modifying the source of the argument. When execution returns to the calling procedure, the source contains the same value it did before the function was called.

The **Byref** parameter is the default in CitectVBA and does not need to be used explicitly within CitectVBA. **Byref** gives other subroutines and functions permission to make changes to the source of the values that are passed in **Byref**. The keyword **Byval** denies this permission so the argument source cannot be altered.

There are two possible methods for indicating to CitectVBA that you wish to pass an argument by value :

- When declaring the argument in the subroutine or function declaration statement, by using the **Byval** keyword placed immediately before the argument name. This forces the subroutine or function to use a copy of the argument passed in and not modify the source. For example, the following function TestPassArg has declared its first

argument intVal as being requested **Byval**.

```
Function TestPassArg(Byval intVal As Integer, varVal, strVal as String)
```

- When passing an argument to a subroutine or function, by enclosing the individual argument within parentheses. Only the value of the argument, and not its address in memory, is passed to the subroutine or function, so that the source of the argument is not modified. For example, only the variable var3 is passed by value to the subroutine TestPassArg (because only that argument is enclosed within parentheses in the subroutine call).

```
TestPassArg var1, var2,(var3)
```

- In the next example, the parameter iVar is passed by value to the function TestFunction. Since arguments passed to functions must be enclosed in parentheses, an extra pair is used to force the argument to be passed by value.

```
TestFunction((iVar))
```

> **Note:**Vijeo Citect does not support passing by reference, so Vijeo Citect tag values MUST be declared by value when passed as arguments to a CitectVBA procedure from within a Vijeo Citect command or expression field. This is best done by declaring the variable, assigning it the tag value, then passing the variable by value. (See the Example below.)

**Example**

Suppose you had a variable tag of integer type named "iTag1" and you need to pass it to a function. From within a CitectVBA script, or Vijeo Citect command or expression field, you would use the following code example to pass the variable tag value to a function named `TagArgumentTest`:

```
CiVBA
Dim iVar1 as Integer
iVar1 = iTag1
TagArgumentTest(iVar1)
```

> **Note:** Cicode does not support passing by reference, so CitectVBA variables passed to Cicode functions using the `CicodeCallOpen` function must be enclosed in brackets to force the passing of those variables by value.

**See Also**

Passing Arguments to DLL Functions from CitectVBA

DLLs and APIs

Arguments

## Passing Arguments to DLL Functions from CitectVBA

Many arguments to DLL functions are passed by value. By default, arguments in CitectVBA are passed by reference, so it's important that you include the Byval keyword in the function definition when the DLL function requires that an argument be passed by value. See Passing variables Byref and Byval.

> **Note:** Although the Byval keyword appears in front of some arguments of type String, strings are always passed to Windows API functions by reference, therefore any DLL function can always modify a string source directly.

DLL functions don't return strings in the same way that CitectVBA functions do. Because strings are always passed to DLL functions by reference, the DLL function can modify the value of the string argument. Rather than returning a string as the return value for the function, as you would probably do in CitectVBA, a DLL function returns a string into an argument of type String that was passed to the function. The actual return value for the function is often a long integer specifying the number of bytes that were written into the string argument.

To call a DLL function that writes to a String variable, you need to take additional steps to format the string properly. First of all, the String variable must be a null-terminated string. A null-terminated string ends in a special null character. Secondly, a DLL function can't change the size of a string once it has been created.

Therefore, you need to make sure that the string that you pass to a function is large enough to hold the entire return value, and that it terminates with a Null character. When you pass a string to a DLL function, you'll usually need to specify the size of the string that you've passed in another argument. Windows keeps track of the length of the string so that it doesn't overwrite any memory that the string is using.

> **Note:** It's only necessary to pass in a null-terminated string and its size if you're returning a string from a function. If the function does not return a string into a string argument, but instead takes a string that provides information to the function, you can simply pass in a normal CitectVBA String variable.

A Nullstring is a string of value 0 [no Character code]; note that this is not the same as an empty string ("").

### See Also

DLLs and APIs

Arguments

Passing variables Byref and Byval

# OLE Services

OLE (Object Linking and Embedding) services is the term used to generally describe the integrated use of separate software components (applications) working together to provide custom software solutions based upon the Microsoft Component Object Model (COM) architecture.

**Note:** When considering the use of OLE services, you should be aware that there are different uses of OLE which have developed over the years and which may be confused with one another. Examples of different OLE services include: object linking, object embedding, visual editing, drag-and-drop, ActiveX Controls, OLE Automation, OLE DB, OLE Messaging, and OLE Networking services. See OLE terminology.

Vijeo Citect supports linked and embedded OLE objects in its graphics pages with the use of ActiveX Controls. See Accessing ActiveX Objects with CitectVBA.

Vijeo Citect can use CitectVBA to perform as an OLE Automation controller. See OLE automation objects. Vijeo Citect can also exchange data with other applications using other data transfer technologies.

## OLE terminology

OLE superceded the Dynamic Data Exchange protocol. Network DDE was introduced to afford the same data transfer facility between Windows applications connected across the same network. Vijeo Citect supports both DDE and Network DDE connectivity.

### OLE Linking and Embedding

The differences between linked objects and embedded objects which may affect you, concern where the data is stored, and how it is updated after you place it in the destination file. With linked OLE objects, the source of the OLE object data remains in the original data file of the application that was used to create it, and only a copy of the data is ever displayed in the destination document. The data is updated only when the source file is modified. Embedded OLE objects duplicate and store a local copy of the source file data within the destination document data file, and are not linked to the source file. That is, the data copy in the destination file does not change when you modify the source file.

With both linked and embedded OLE objects, when the OLE object in the destination document is double-clicked, the original application (that was used to create the data) of the OLE object is launched to permit editing of the data using that source program's editor. Linked OLE objects store their data back in the original source data files, while embedded OLE objects store their data in the destination program data files.

### OLE Automation

'OLE Automation' was developed to permit the (remote) control of other applications on the same computer. Applications which expose their functionality using OLE Automation are known as OLE Automation servers, and could be automated by code running in a completely separate application, known as OLE Automation clients or controllers.

OLE Automation servers exposed their functionality through structured object models, which are listings of the internal functions, methods and properties of the application object. All Microsoft Office applications are OLE Automation servers to some extent, and can be subsequently controlled by any OLE Automation compliant controller, using the appropriate syntax to manipulate and control the relevant application object model.

Not all applications that support OLE services support OLE Automation. For example, many products support drag-and-drop, and object linking and embedding, but do not support OLE Automation. Linking and embedding allow the user to access the object, whereas OLE Automation allows one application to control another application, possibly with minimal or no user interaction.

**See Also**

OLE Services

## OLE automation objects

CitectVBA supports the referencing and control of OLE Automation objects of external applications, permitting you to use the properties, methods and events of those objects from within Vijeo Citect.

To access an OLE Automation object using CitectVBA, you must first declare an object variable in your CitectVBA code, then assign an OLE Automation reference to the variable. See Declaration of OLE Automation objects in CitectVBA, and Assigning references to OLE Automation objects in CitectVBA.

Objects declared in a CitectVBA Sub or Function procedure are local to that procedure, and their lifetime ends along with the end of the procedure. An object declared outside a procedure has modular scope to all procedures within the same CitectVBA file module and lasts for the lifetime of the variable that retains the reference to the object.

All object references must be deleted when they are no longer required, to release the memory they were using.

When considering the use of OLE Automation, you should be aware that there are different uses of OLE which have developed over the years and which may be confused with one another.

**See Also**

OLE Services

## Declaration of OLE automation objects

CitectVBA objects can only be declared and referenced within CitectVBA file modules. CitectVBA modular objects have modular scope and cannot be referenced (accessed and used) from outside their CitectVBA module (file).

> **Note:** CitectVBA objects cannot be used directly in Vijeo Citect command or expression fields.

Once declared within a CitectVBA module, CitectVBA objects can be referenced and used in any procedure within the same code module. An object declared outside of a procedure has modular scope to all procedures within that same CitectVBA module (file). Objects declared within a Sub or Function procedure have local scope only within that procedure.

The object variable must be declared before it can be assigned an object reference. Object variables are declared by the Dim Statement with the As Object CitectVBA data type using the following syntax:

```
Dim <VariableName> As Object
```

where:

- Dim is the required Variable declaration statement BASIC keyword
- <VariableName> represents the required name of the variable being declared (dimensioned)
- As Object declares the variable as a CitectVBA 'object' data type

> **Note:** The placeholder shown inside arrow brackets (<`placeholder`>) should be replaced in any actual code with the value of the item that it describes. The arrow brackets and the word they contain should not be included in the statement, and are shown here only for your information.

For example:

```
' create local variables to store object references
Dim objExcelApp As Object
Dim objWordApp As Object
```

Once declared, you can then assign an OLE Automation reference to the object variable in CitectVBA.

**See Also**

Deleting OLE automation objects

Using OLE automation objects

## Assigning references to OLE automation objects

An OLE Automation object MUST be defined before it can be used. Once defined (see Declaration of OLE Automation objects in CitectVBA), you assign an OLE Automation reference to the object variable in CitectVBA using the CitectVBA CreateObject function within a CitectVBA Set statement in the following syntax:

```
Set <objVarName> = CreateObject(<objClassName>)
```

where:

- Set is the required reference assignment statement keyword
- <objVarName> represents the required name of the variable receiving the reference
- CreateObject() function creates the object of the class type specified in the argument
- <objClassName> represents the required name of the class providing the object

The object class name passed as the argument to the CreatObject function usually consists of the fully qualified class name of the object being created, for example "Word.Application" or "Excel.Application".

**Example**

```
' create variable to store object reference
Dim objExcelApp As Object
' create the app object and assign the reference
Set objExcelApp = CreateObject("Excel.Application")
' or
' create variable to store object reference
Dim objWordApp As Object
' create the app object and assign the reference
Set objWordApp = CreateObject("Word.Application")
```

Once assigned, you can then use that object variable in your CitectVBA code to manipulate the referenced object model. See Using OLE automation objects.

Dependant objects (which cannot be created independantly) can be "drilled-down" to and subsequently assigned from existing (externally creatable) independant object s, by using a method of the higher level object. See Understanding object models in OLE Automation.

For examples of independant objects, Microsoft Excel provides the "Excel.Application", "Excel.Sheet ", and "Excel.Chart" externally creatable objects amongst others, (two of which are demonstrated in OLE Automation example using the Microsoft Excel object), and Microsoft Word provides the "Word.Application", "Word.Document", and "Word.Picture" externally creatable objects amongst others (and is demonstrated in OLE Automation example using the Microsoft Word object).

**See Also**

OLE automation objects

## Using OLE automation objects

The trick with successfully using OLE Automation is determining what you can and can't do with it. In theory, you can do anything the OLE Automation server application can do. However, in practice, not every OLE Automation server application exposes all of its functionality through its OLE Automation interface.

You have to be able to use the native programming language of the OLE Automation server application in your code. You also need to know about the limitations imposed by the Vijeo Citect operating environment, and its implementation of the CitectVBA programming language.

CitectVBA does not support early binding of OLE Automation objects, as there is no mechanism for providing a reference to the object type library (like you can do in Microsoft Visual Studio) until runtime. So, CitectVBA compile errors can occur with valid VBA code which may work well in other VBA supporting applications. Most ported VBA code will require some modification to compile and perform as expected in CitectVBA. For example, CitectVBA does not support the use of "With" statements concerning properties or methods of an object, yet does support the use of "For Each" statements with objects in a collection.

CitectVBA does not support the use of named arguments using the ":=" named argument operator (colon followed by an equal sign). Nor does it support the use of missing arguments using placeholder commas, however, CitectVBA does support the use of the "empty" keyword in place of missing arguments.

CitectVBA does not support the passing of SCADA variable tags by reference, however, the tag value can be copied to a CitectVBA variable, and it can be passed by value. See Passing Variables Byref and Byval with CitectVBA.

To help manage these considerations, you should know how to access the object model of the OLE Automation server applications. CitectVBA does not support the use of application-defined object types nor intrinsic constants due to late-binding of the object model. CitectVBA supports only 10 data-types, so be aware of the possibility of data being lost

due to rounding when converting between different data types. See Rounding Numbers in CitectVBA.

To make full use of the OLE Automation object models, you should make yourself familiar with Object related terms. See Understanding object models in OLE Automation.

**See Also**

OLE automation objects

## Accessing the object model of OLE automation server applications

During the development stage of your project, to access the object model of any OLE Automation server applications, you must have a copy of the appropriate application program installed on the computer you are developing the OLE Automation controller with.

Equally, during Vijeo Citect runtime, there must be a copy of the appropriate application program installed on the computer you are running the OLE Automation controller from. If, for example, you were calling the code which creates the object from say a button on a graphics page on a Vijeo Citect Client machine, the appropriate application program must be installed on every Client machine with access to that graphics page, for the code to work (if called) on that Client machine.

All of the Microsoft Office suite of products support the VBA language in some manner, and export an OLE Automation object type library which you can view and use. See How to view an OLE Automation object type library from a Microsoft Office product.

Also, the VB programming IDE within Visual Studio can be referenced to load the appropriate type library as required. See How to view an OLE Automation object type library in VB.

Both these suites provide an object browser which you can use to explore the object models. You use the structure of the object model to access, manipulate and control the OLE Automation object using CitectVBA. See Understanding object models in OLE Automation.

**See Also**

OLE automation objects

## Understanding object models in OLE automation

Objects are the fundamental building blocks of OLE Automation, and object models are a roadmap to the object structure. OLE Automation using CitectVBA involves creating and modifying the objects provided by other applications (external to the Vijeo Citect application). For instance, every element of Microsoft Word ( documents, tables, paragraphs, bookmarks, fields and so on) can be represented by an object in CitectVBA using OLE automation with the Word object model.

### What are objects and collections?

An object represents an element of the OLE Automation application. A collection is an object that contains several other objects, usually of the same type. For example, all the bookmark objects in a document are contained in a single Bookmarks collection object of the Word application. Using appropriate properties and methods, OLE Automation permits the modification of a single object or an entire collection of objects.

### What is a property?

A property is an attribute of an object or an aspect of its behavior. For example, properties of a Word document include its name, its content, and its save status, as well as whether change tracking is turned on. To change the characteristics of any referenced object, you change the values of its properties.

To set the value of a property, follow the reference to an object with a period, the property name, an equal sign, and the new property value. The following example turns on change tracking in the Word document named "MyDoc.doc."

```
objWordApp.Documents("MyDoc.doc").TrackRevisions = True
```

In this example, Documents refers to the collection of open documents, and the name "MyDoc.doc" identifies a single document in the collection. The TrackRevisions property is set for that single document.

You can also return information about an object by returning the value of one of its properties. The following example returns the name of the active Word document.

```
docName = objWordApp.ActiveDocument.Name
```

In this example, ActiveDocument refers to the document in the active window in Word. The name of that document is assigned to the variable "docName".

**Note:** Some properties cannot be set. The Help topic for each property indicates

whether you can set that property (read-write), only read the property (read-only), or only write the property (write-only). Also the Object Browser in the Visual Basic Editor displays the read-write status at the bottom of the browser window when the property is selected.

**What is a method?**

A method is an action that an object can perform. For example, just as a Word document can be printed, the Document object has a PrintOut method. Methods often have arguments that qualify how the action is performed. The following example prints the first three pages of the active Word document.

```
objWordApp.ActiveDocument.PrintOut From:=1, To:=3
```

In most cases, methods are actions and properties are qualities. Using a method causes something to happen to an object, while using a property returns information about the object or it causes a quality about the object to change.

**Returning an object**

Most objects return a single object from the collection. For example, the Documents collection contains the currently open Word documents. You use the Documents property of the Application object (the object at the top of the Word object hierarchy) to return the Documents collection.

After you've accessed the collection, you can return a single object by using an index value in parentheses (this is similar to how you work with VBA arrays). The index value can be either a number or a name.

The following example uses the Documents property to access the Document collection. The index number is used to return the first document in the Documents collection. The Close method is then applied to the Document object to close the first document in the Documents collection.

```
objWordApp.Documents(1).Close
```

The following example uses a name (specified as a string) to identify a Document object within the Documents collection.

```
objWordApp.Documents("Sales.doc").Close
```

Collection objects often have methods and properties which you can use to modify the entire collection of objects. The Documents object has a Save method that saves all the documents in the collection. The following example saves the open documents by applying the Save method.

```
objWordApp.Documents.Save
```

The Document object also has a Save method available for saving a single document. The following example saves the document named Report.doc.

```
objWordApp.Documents("Report.doc").Save
```

To return an object that is further down in the Word object hierarchy, you must "drill down" to it by using properties and methods to return objects.

To see how this is done, in Word, open the Visual Basic Editor and click Object Browser on the View menu. Click Application in the Classes list on the left. Then click ActiveDocument from the list of members on the right. The text at bottom of the Object Browser indicates that ActiveDocument is a read-only property that returns a Document object. Click Document at the bottom of the Object Browser; the Document object is automatically selected in the Classes list, and the Members list displays the members of the Document object. Scroll through the list of members until you find Close. Click the Close method. The text at the bottom of the Object Browser window shows the syntax for the method. For more information about the method, press F1 or click the Help button to jump to the Close method Help topic.

Given this information, you can write the following instruction to close the active document.

```
objWordApp.ActiveDocument.Close SaveChanges:=wdSaveChanges
```

The following example maximizes the active document window.

```
objWordApp.ActiveDocument.ActiveWindow.WindowState =
wdWindowStateMaximize
```

The ActiveWindow property returns a Window object that represents the active window. The WindowState property is set to the maximize constant (wdWindowStateMaximize).

The following example creates a new document and displays the Save As dialog box so that a name can be provided for the document.

```
objWordApp.Documents.Add.Save
```

The Documents property returns the Documents collection. The Add method creates a new document and returns a Document object. The Save method is then applied to the Document object.

As you can see, you use methods or properties to drill down to an object. That is, you return an object by applying a method or property to an object above it in the object hierarchy. After you return the object you want, you can apply the methods and control the properties of that object.

**See Also**

OLE Services

## Using the Microsoft Word object model

You should use the associated online help documentation that came with the object application to obtain details of the object model.

The help is quite easy to use. Each of the classes and collections can be clicked to jump to its page.

In CitectVBA, you must use the full Application object qualifier when referencing the properties and methods of the object. For example, you must use the full syntax "Application.ActiveDocument.PrintOut", instead of "ActiveDocument.PrintOut".

**See Also**

OLE Services

## OLE automation example using the Microsoft Word object

All commands in Word are directed to the active document, which may be changed in code. It is recommended to use named arguments, as the argument sequences are recorded incorrectly in some documentation, including the type library and what the recorder writes to macros.

```
Sub runWord()
' demonstrating the use of OLE Automation
' to manipulate Word

' create local variables
Dim objWordApp As Object
' create the app object and assign the reference
Set objWordApp = CreateObject("Word.Application")

' manipulate the app object
' insert appropriate VBA code here to manipulate Word object

' close Word
```

```
objWordApp.Quit

' delete the object
Set objWordApp= Nothing

End Sub
```

**See Also**

OLE automation example using the Microsoft Word object

Using OLE automation objects

## Using the Microsoft Excel object model

You should use the associated online help documentation that came with the object application to obtain details of the object model.

**See Also**

Using the Microsoft Excel object model

Using OLE automation objects

## Deleting OLE automation objects

All object references must be deleted when they are no longer required, to release the memory they were using.

You delete an OLE Automation reference to the object variable in CitectVBA using the CitectVBA Nothing keyword within a CitectVBA Set statement in the following syntax:

```
Set <objVarName> = Nothing
```

where:

- `Set` is the required reference assignment/release statement keyword.
- `<objVarName>` represents the required name of the variable holding the reference.
- `Nothing` is the keyword used to release the object reference.

When several object variables refer to the same object, they also refer to the memory and system resources associated with the object. These resources are released only after all of them have been set to Nothing, either explicitly using Set, or implicitly after the last object variable set to Nothing goes out of scope.

**Example**

```
' Word example
' create variable to store object reference
Dim objWord as Object
' create object and assign reference to variable
Set objWord = CreateObject( "Word.Document" )
' insert appropriate VBA code here to manipulate Word object
' release reference
Set objWord = Nothing
' Excel example
' create local variables
Dim objExcelApp As Object
Dim objExcelCht As Object
' create the app object and assign the reference
Set objExcelApp = CreateObject("Excel.Application")
' create a chart and assign the reference
Set objExcelCht = objExcelApp.Charts.Add()
' insert appropriate VBA code here to manipulate Excel objects
' delete the objects
Set objExcelApp = Nothing
Set objExcelCht = Nothing
```

**See Also**

Using OLE automation objects

## File Input/Output with CitectVBA

CitectVBA supports full sequential and binary file Input/Output (I/O).

Files stored on disk, can contain text (ASCII) characters or binary (ones and zeros) digits. All CitectVBA files that contain CitectVBA code are stored as text files. However, you can use CitectVBA to store and retrieve files in either format, using CitectVBA file I/O functions and statements.

The File I/O functions predefined in CitectVBA are:

```
ChDir, ChDrive, Close, CurDir, Dir, EOF, FileCopy, FileLen,
FreeFile, Get #, GetAttr, Input #, Kill, Line Input #, Loc, LOF,
MkDir, RmDir, Name, Open, Print #, Put, Seek, SetAttr, Write #.
```

For details of all predefined CitectVBA functions, see CitectVBA Function Reference.

# Chapter 5: CitectVBA Function Reference

CitectVBA includes the following function categories.

| | |
|---|---|
| Array Functions | File I/O Functions |
| Conditional Statements | Math/Trigonometry Functions |
| Conversion Functions | Miscellaneous Functions |
| Declarations | Procedural Statements |
| Date and Time Functions | String Functions |

## Array Functions

CitectVBA array functions are provided to allow you to declare, resize, initialize, populate, and erase arrays and their elements.

The array functions predefined in CitectVBA are:

| | |
|---|---|
| Dim | Allocates storage for, and declares the data type of, variables and arrays in a module. |
| Erase | Reinitializes the elements of a fixed array. |
| Lbound | Returns the smallest available subscript for the dimension of the indicated array. |
| Option Base | Declares the default lower bound for array subscripts. |
| ReDim | Used to size or resize a dynamic array that has already been declared using the Dim statement with empty parentheses. |
| Ubound | Returns the value of the largest usable subscript for the specified dimension of an array. |

### Dim

The Dim statement allocates storage for, and declares the data type of, variables and arrays in a module.

The To clause in the array subscript range of a Dim statement provides a more flexible way to control the lower bound of an array. If you don't explicitly set the lower bound with a To clause, the Option Base setting (if used) comes into affect, or defaults to zero (if not used).

**Syntax**

**Dim** *VariableName*[(*Subscripts*)] [As *DataType*]

*VariableName*:

The name of the variable or array being declared (dimensioned).

*Subscripts*:

The optional subscript range (dimensions) for an array in parentheses.

*DataType*:

The optional data type declaration for the variable or array.

**Related Functions**

Const | ReDim | Static

**Example**

```
Dim bytVar As Byte
Dim binVar As Boolean
Dim strVar As String
Dim intVar As Integer
Dim lngVar As Long
Dim sngVar As Single
Dim dblVar As Double
Dim vntVar As Variant
Dim objVar As Object
Dim dtmVar As Date

Dim daysOfWeek() As String ' declares an array variable to hold strings

Dim monthsOfYear(12) As Date ' declares an array variable to hold 12 strings
Dim users(,) As String ' declares a two dimensional array to hold strings
Dim usernames(5,5) As String ' declares a two dimensional 5 x 5 array to hold
strings

Dim MyArray(1 To 10, 5 To 15, 10 To 20) ' declares the three dimensional array
 MyArray and specifies the upper and lower bounds of each dimension
```

## Erase

Reinitialises the elements of a fixed array specified in the ArrayList parameter.

**Syntax**

**Erase**(*Arraylist*)

*Arraylist:*

A comma delimited list of valid variable array names.

**Related Functions**

Dim | ReDim

**Example**

```
Option Base 1
Dim Num(10) As Integer    ' Integer array.
Dim StrVarArray(10) As String    ' Variable-string array.
Dim StrFixArray(10) As String * 10    ' Fixed-string array.
Dim VarArray(10) As Variant     ' Variant array.
Dim DynamicArray() As Integer    ' Dynamic array.
ReDim DynamicArray(10)    ' Allocate storage space.
Erase Num    ' Each element set to 0.
Erase StrVarArray    ' Each element set to zero-length string ("").
Erase StrFixArray    ' Each element set to 0.
Erase VarArray    ' Each element set to Empty.
Erase DynamicArray    ' Free memory used by array.
Erase StrVarArray,StrFixArray,VarArray ' Reset three arrays at the same time.
```

## Lbound

Determines the value of the lower bound for the dimension of the array specified in the arguments.

Lbound expects the required argument *ArrayName* to be a valid variable array name. The optional argument *ArrayDimension* must be a whole long number indicating which dimension's lower bound is to be returned. Use 1 for the first dimension, 2 for the second, and so on.

**Syntax**

**Lbound**(*ArrayName*, *ArrayDimension*)

*ArrayName*:

The name of the array.

*ArrayDimension*:

The dimension of the array for which you want to the lower bound. If *ArrayDimension* is omitted, 1 is assumed.

**Return Value**

Returns a number of Long data type.

**Related Functions**

Ubound

**Example**

```
Dim Lower
Dim MyArray(1 To 10, 5 To 15, 10 To 20)  ' Declare array variables.
Dim AnyArray(10)
Lower = LBound(MyArray, 1)  ' Returns 1.
Lower = LBound(MyArray, 2)  ' Returns 5.
Lower = LBound(AnyArray)  ' Returns 1.
```

## Option Base

Declares the default lower bound for array subscripts.

The Option Base statement is optional. If used, it can appear only once in a CitectVBA file, and must be used before you declare the dimensions of any arrays.

The To clause in the array subscript range of a Dim statement provides a more flexible way to control the lower bound of an array. If you don't explicitly set the lower bound with a To clause, the Option Base setting (if used) comes into affect, or defaults to zero (if not used).

**Syntax**

**Option Base***Num*

*Num*:

An Integer or expression representing a valid numeric value. The value of the 'number' parameter must be either 0 or 1. The default is 0.

**Related Functions**

Dim | ReDim

**Example**

The example below uses the Option Base statement to override the default base array subscript value of 0.

```
' Module level statement
Option Base 1

' Create the array
Dim Arr(20)
' Declare message variables
Dim Msg As String
```

```
Dim NL as String
' Define newline
NL = Chr(10) & Chr(13)
' Create message
Msg = "The lower bound is " & LBound(Arr) & "."
Msg = Msg & NL & "The upper bound is " & UBound(Arr) & "."
' Display message
MsgBox Msg
```

## ReDim

Used to size or resize a dynamic array that has already been declared using the Dim statement with empty parentheses.

Use the ReDim statement to change the number of elements in an array, but not to change the number of dimensions in an array or the type of the elements in the array.

### Syntax

**ReDim***VariableName*(*Subscripts*)

*VariableName*:

The name of the variable or array being redimensioned.

*Subscripts*:

An Integer or expression representing a valid To numeric value range when declaring the dimensions of an variable array. Up to 60 multiple dimensions may be declared.

The subscripts argument uses the following syntax:

```
[lower To] upper [,[lower To] upper] . . .
```

When not explicitly stated in **lower**, the **lower** bound of an array is controlled by the Option Base statement. The lower bound is zero if no Option Base statement is present in the CitectVBA file.

### Related Functions

Dim | Const | Static

### Example

```
Dim TestArray() As Integer
Dim I
ReDim TestArray(10)
For I = 1 To 10
    TestArray(I) = I + 10
    Print TestArray(I)
```

```
Next I
```

## Ubound

Determines the value of the largest subscript for the *ArrayDimension* of the *ArrayName* provided in the argument. Ubound expects the required argument *ArrayName* to be a valid variable array name.

The optional argument *ArrayDimension* must be a whole long number indicating which dimension's lower bound is to be returned. Use 1 for the first dimension, 2 for the second, and so on. If *ArrayDimension*is omitted, 1 is assumed.

### Syntax

**Ubound**(*ArrayName, ArrayDimension*)

*ArrayName*:

A string or expression that can represent a valid variable array name.

*ArrayDimension*:

A numeric value or expression that can represent a valid long data type value.

### Return Value

Returns a number of Long data type.

### Related Functions

Lbound

### Example

```
Dim Upper
Dim MyArray(1 To 10, 5 To 15, 10 To 20)  ' Declare array variables.
Dim AnyArray(10)
Upper = UBound(MyArray, 1)  ' Returns 10.
Upper = UBound(MyArray, 3)  ' Returns 20.
Upper = UBound(AnyArray)  ' Returns 10.
```

# Conditional Statements

| | |
|---|---|
| Do Loop | Allows you to execute a block of statements an indefinite number of times. |
| End Func- | Ends a block of statements such as a Sub procedure or function. |

| tion | |
|---|---|
| Exit | Exits a loop or procedure. |
| For | Repeats its block of statements a set number of times as determined by the values used with the To clause. |
| Goto | Branches unconditionally and without return to the label specified in the GoTo statement. |
| If | Tests an initial condition and then either performs or omits to perform the statements it contains, dependant upon the logical result of the test condition. |
| OnError | CitectVBAs error-handling routine and specifies the line label of the error-handling routine. |
| Select | Tests the same variable for many different conditions. |
| Stop | Ends execution of the program. |
| While...W-end | Similar to the Do While loop statement. |
| With | Not supported in CitectVBA. |

## Do Loop

The Do...Loop conditional statement allows you to execute a block of statements an indefinite number of times. The variations of the Do...Loop are Do...While, Do...Until, Do...Loop While, and Do...Loop Until.

```
Do While <condition>
    <statement/s>
Loop
```

```
Do Until <condition>
    <statement/s>
Loop
```

```
Do
    <statement/s>
```

```
Loop While <condition>
```

```
Do
    <statement/s>
Loop Until <condition>
```

Do...While and Do...Until check the condition before entering the loop, thus the block of statements inside the loop are only executed when those conditions are met. Do...Loop While and Do...Loop Until check the condition after having executed the block of statements so that the block of statements is executed at least once.

Any Do statement can be exited using the Exit Do statement.

## End Function

The End Function statement ends a program or a block of statements within a function. A CitectVBA function starts with the FUNCTION statement and finishes with the END FUNCTION statement. All other statements that lie between the FUNCTION and END FUNCTION statements will be executed by the function when called to do so.

### Syntax

**End {Function | Sub | If}**

### Related Functions

Call | Sub | End Sub | Exit

### Example

```
Function GetColor2( c% ) As Long
    GetColor2 = c% * 25
    If c% > 2 Then
        GetColor2 = 255        ' 0x0000FF - Red
    End If
    If c% > 5 Then
        GetColor2 = 65280    ' 0x00FF00 - Green
    End If
    If c% > 8 Then
        GetColor2 = 16711680    ' 0xFF0000 - Blue
    End If
End Function

Sub TestColor2
    Dim I as integer
    For I = 1 to 10
        Print GetColor2(I)
    Next I
End Sub
```

## Exit

Exits a loop or procedure.

**Syntax**

**Exit {Do | For | Function | Sub}**

**Example**

```
' This sample shows Do ... Loop with Exit Do to get out.
Dim Value, Msg    ' Declare variables
Do
    Value = InputBox("Enter a value from 5 to 10.")
    If Value >= 5 And Value <= 10 Then    ' Check range
        Exit Do    ' Exit Do...Loop
    Else
        Beep    ' Beep if not in range
    End If
Loop
```

## For

Repeats its block of statements a set number of times as determined by the values used with the To clause.

**Example**

```
For <CounterName> = <BeginValue> To <EndValue> [Step <StepValue>]
    <statement/s>
Next
```

## Goto

The GoTo conditional statement branches unconditionally and without return to the label specified in the GoTo statement. The label must be located in the same subroutine or function as the GoTo statement.

**Example**

```
<statement/s>
If <condition> then
    GoTo Label1
Else
    GoTo Label2
End If

Label1:
```

```
<statement/s>
GoTo Label3

Label2:
<statement/s>
GoTo Label3

Label3:
<statement/s>
```

In this example, CitectVBA tests the If condition, and jumps to the part of the script that begins with the label "Label1:" if the condition was true, or jumps to the part of the script that begins with the label "Label2:" if the condition was false. This could be anywhere in the same subroutine or function.

## If

Tests an initial condition and then either performs or omits to perform the statements it contains, dependant upon the logical result of the test condition. The condition can be a comparison or an expression, and must logically evaluate to either True or False. The If statement has both single line and multiple line syntax structure.

The single line syntax uses the If <TestCondition> Then <StatementToPerformIfTrue> structure, however, can only perform a single statement if and only if the test condition result is True. No 'End If' statement is required:

**Example**

**If**<*Condition*>**Then**<*Statement*>

If the result of the If test condition was True, the program flow continues into and performs the statement following the Then statement, until it reaches the end of the line.

To perform a single statement conditionally upon a False result, use the NOT logical operator:

```
If NOT <Condition> Then <Statement>
```

To perform multiple statements, use the multiple line syntax structure which ends with the 'End If' statement:

```
If <Condition> Then
    ' Then statement block
    ' perform only if true
    <Statement/s>
End If
```

If the result of the If test condition was True, the program flow continues into the Then statement block, and performs the statements following the Then statement, until it reaches the End If statement.

If the result of the If test condition was False, the program flow jumps over the Then statement block, which in this case exits the If structure (without performing any statements other than the initial test condition).

The mutiple line If structure can perform different blocks of statements dependant upon EITHER a True OR a False result to the test condition, through the use of the Else statement block:

```
If <Condition> Then
    ' Then statement block
    ' perform only if true
    <Statement/s>
Else
    ' Else statement block
    ' perform only if false
    <Statement/s>
End If
```

If the result of the If test condition was True, the program flow performs the Then block statements, until it reaches the Else statement. It then jumps over the Else statement block and exits the If structure (without performing any of the Else statement block statements).

Further test conditions can be placed into an If structure through the use of the optional Else If <Condition> statement block. ElseIf statement blocks can only be positioned within an If structure before the Else statement block. If the result of the If test condition was False, the program flow jumps over the Then statement block (without performing any of those statements) to the Else statement to perform the statements in the Else statement block until it reaches the End If statement.

```
If <Condition> Then
    ' Then statement block
    ' perform only if true
    <Statement/s>
ElseIf <Condition>
    ' Else If statement block
    ' perform only if true
    <Statement/s>
Else
    ' Else statement block
    ' perform only if false
    <Statement/s>
End If
```

The ElseIf test condition is only evaluated after the initial If structure test condition results in False.

If the result of the ElseIf test condition was True, the statements within the ElseIf statement block are performed. The program flow then jumps over the Else statement block and exits the If structure (without performing any of the Else statement block statements).

If the result of the ElseIf test condition was False, the program flow jumps over the ElseIf statement block (without performing any of those statements) to the Else statement to perform the statements in the Else statement block until it reaches the End If statement.

There is no apparent limit to the number of Else If statement blocks that any one If structure can hold, however, the Select Case Statement structure handles multiple condition result alternatives much more efficiently.

## OnError

CitectVBA's error-handling routine and specifies the line label of the error-handling routine. The line parameter refers to a label. That label needs to be present in the code or an error is generated.

**Syntax**

**On Error{GoTo line | Resume Next | GoTo 0}**

**Example**

```
On Error GoTo errHandler
Dim x as object
x.draw     ' Object not set
..
Exit Sub
errHandler:
Print Err.Number, Err.Description
Resume Next
```

## Select

The Select Case statement tests the same variable for many different conditions. The test value provided with the initial Select Case statement is logically tested against the Case test condition.

The Select Case structure can perform different blocks of statements dependant upon whichever Case statement test condition (if more than one) first results as True, through the use of the Case statement block:

```
Select Case <TestValue>
```

```
Case <Condition>
    ' Case statement block
    ' perform only if case true
    <Statement/s>
Case Else
    ' Else statement block
    ' perform only if all cases false
    <Statement/s>
End Select
```

If the result of the Case test condition was True, the program flow performs the statements contained within that Case statement block, and will then exit the Select Case structure (without performing any of the Else statement block statements).

If the result of the Case test condition was False, the program flow jumps over the Case statement block (without performing any of those statements) to the Case Else statement to perform the statements in the Else statement block until it reaches the End Select statement.

Further test conditions can be placed into a Select Case structure through the optional use of further Case statement blocks. Case statement blocks can only be positioned within a Select Case structure before the Case Else statement block.

```
Select Case <TestValue>
Case <Condition>
    ' Case statement block
    ' perform only if case true
    <Statement/s>
Case <Condition>
    ' Case statement block
    ' perform only if case true
    <Statement/s>
Case Else
    ' Else statement block
    ' perform only if all cases false
    <Statement/s>
End Select
```

Each Case statement block is evaluated in order until the test condition of one results as True. The program flow performs the statements contained within that Case statement block, and will then exit the Select Case structure (without performing any other statements).

The statements of ONLY one Case statement block are ever performed, unless all result in False and there is no Case Else block declared, in which case no Case statement blocks are performed at all.

The following example may help clarify the logic testing being performed in a Select Case structure. Lets say that we have a variable named (intDayOfWeek) containing an integer (ranging from 1 to 7) representing the day of the week, and we wished to display that value as a string (named strDayOfWeek) containing the name of the day of the week, assuming in this example, that Sunday is the first day of the week (1). The Select Case structure would look like this:

```
Dim strDayOfWeek As String

Select Case intDayOfWeek
Case = 1
    StrDayOfWeek = "Sunday"
Case = 2
    StrDayOfWeek = "Monday"
Case = 3
    StrDayOfWeek = "Tuesday"
Case = 4
    StrDayOfWeek = "Wednesday"
Case = 5
    StrDayOfWeek = "Thursday"
Case = 6
    StrDayOfWeek = "Friday"
Case = 7
    StrDayOfWeek = "Saturday"
Case Else
    StrDayOfWeek = "Invalid"
End Select
```

The Select Case structure tends to be easier to read, understand, and follow and should be used in place of a complicated multi-nested If...ElseIf structure.

## Stop

Ends execution of the program. The Stop statement can be placed anywhere in your code.

**Example**

```
Dim x,y,z

For x = 1 to 5
    For y = 1 to 5
        For z = 1 to 5
            Print "Looping",z,y,x
        Next z
    Next y
    Stop
Next x
```

## While...Wend

The `While...Wend`loop conditional statement is similar to the `Do While`loop statement. The condition is checked before executing the block of statements comprising the loop.

**Example**

```
While <condition>
    <statement/s>
Wend
```

### With

**Note:** The With statement is not supported in CitectVBA.

When performing a series of commands on an object, you must explicitly refer to the name of the object with each command.

## Conversion Functions

CitectVBA conversion functions are provided to assist with data manipulation and calculation in your formulas. Conversion functions can be used in CitectVBA statements, and will (like all other functions), return a value to the caller.

### ASCII character code conversion

Vijeo Citect uses the following character code conversion functions:

| | |
|---|---|
| Asc | Returns the numeric ASCII value of a string. |
| Chr | Returns the string ASCII value of a number. |

### Asc

Converts a text string character to its numeric ASCII code value. The Asc function expects the argument *Str* to be a valid string expression. If *Str*contains no characters, a runtime error occurs. The Asc function performs the opposite of the Chr function, which converts a number into its string character ASCII code value.

**Syntax**

**Asc**(*Str*)

*Str*:

A string or expression that can represent a valid text value.

**Return Value**

Returns the numeric ASCII code value of the first character in *Str* provided in the argument.

**Related Functions**

Chr

**Example**

```
Dim vntVar ' declare result holder variable
vntVar = Asc("A")' returns 65
vntVar = Asc("Z")' returns 90
vntVar = Asc("a")' returns 97
vntVar = Asc("z")' returns 122
vntVar = Asc("Apple")' returns 65
vntVar = Asc("Zoe")' returns 90
```

## Chr

Converts a number into its string character ASCII code value.

The Chr function expects the argument *Num* to be a valid numeric integer (whole positive number within the range 0 to 255 inclusive). If *Chr*contains no number, a runtime error occurs.

**Note:** Values 8, 9, 10, and 13 convert to backspace, tab, linefeed, and carriage return characters respectively.

The Chr function performs the opposite of the Asc function, which converts a text string character to it's numeric ASCII code value.

**Syntax**

**Chr**(*Num*)

*Num*:

An integer or expression representing a valid numeric value.

**Return Value**

Returns a single character string representing the ASCII character code value of the number *Num* provided in the argument.

**Related Functions**

Asc

**Example**

```
Dim vntVar     ' declare result holder variable
vntVar = Chr(65)    ' returns "A"
vntVar = Chr(97)    ' returns "a"
vntVar = Chr(90)    ' returns "Z"
vntVar = Chr(122)    ' returns "z"
```

## Date conversion

Vijeo Citect uses the following date conversion functions:

| | |
|---|---|
| CDate | Converts an expression to a variant of date data type. |
| CDbl | Converts an expression to a double data type. |
| CInt | Converts an expression to a integer data type. |
| CLng | Converts an expression to a long data type. |
| CSng | Converts an expression to a single data type. |
| CStr | Converts an expression to a string data type. |
| CVar | Converts an expression to a variant data type. |

## CDate

Converts any valid date expression to a Date data type.

The CDate function expects the argument *Date* to be a date expression (limited to numbers or strings in any combination) that can represent a date from January 1, 100 through December 31, 9999.

**Syntax**

**CDate**(*Date*)

*Date*:

A string or expression that can represent a date value. This includes any combination of date literals, numbers that look like dates, strings that look like dates, and dates from functions.

**Return Value**

Returns the value of the expression *Date* provided in the argument as a variant with a vartype of 7 (date data type).

**Related Functions**

CDbl | CInt | CLng | CSng | CStr | CVar

**Example**

```
Dim MybDate, MDate, MTime, MSTime
' Define date.
MybDate = "May 29, 1959"
' Convert to Date data type.
MDate = CDate(MybDate)
' Define time.
MTime = "10:32:27 PM"
' Convert to Date data type.
MSTime = CDate(MTime)
```

## CDbl

Converts expressions to a double data type.

**Syntax**

**CDbl**(*Exp*)

*Exp*:

A valid string, number or Variant containing a value recognizable as a string or a number.

**Return Value**

Returns the value of the expression *Exp* provided in the argument as a double data type.

**Related Functions**

CDate | CInt | CLng | CSng | CStr | CVar

**Example**

```
Dim x as integer
Dim z as double
z = CDbl(x)'Converts the integer value of x to a double value in z
```

## CInt

Converts expressions to an integer data type.

**Syntax**

**CInt**(*Exp*)

*Exp*:

A valid string, number or Variant containing a value recognizable as a string or number.

**Return Value**

Returns the value of the expression *Exp* provided in the argument as an integer data type.

**Related Functions**

CDate | CDbl | CLng | CSng | CStr | CVar

**Example**

```
Dim x as integer
Dim y as long
x = CInt(y)    'Converts the long value of y to an integer value in x
```

## CLng

Converts expressions to a long data type.

**Syntax**

**CLng**(*Exp*)

*Exp*:

A valid string, number or Variant containing a value recognizable as a string or number.

**Return Value**

Returns the value of the expression *Exp* provided in the argument as a long data type.

**Related Functions**

CDate | CDbl | CInt | CSng | CStr | CVar

**Example**

```
Dim x as integer
Dim y as long
y = CLng(x)    'Converts the integer value of x to an long value in y
```

## CSng

Converts expressions to a single data type.

**Syntax**

**CSng**(*Exp*)

*Exp:*

A valid string, number or Variant containing a value recognizable as a string or number.

**Return Value**

Returns the value of the expression *Exp* provided in the argument as a single data type.

**Related Functions**

CDate | CDbl | CInt | CLng | CStr | CVar

**Example**

```
Dim x as integer
Dim s as single
s = CSng(x)    'Converts the integer value of x to a single value in s
```

## CStr

Converts expressions to a string data type.

**Syntax**

**CStr**(*Exp*)

*Exp*:

A valid string, number or Variant containing a value recognizable as a string or number.

**Return Value**

Returns the value of the expression *Exp* provided in the argument as a string data type.

**Related Functions**

CDate | CDbl | CInt | CLng | CStr | CVar | CSng

**Example**

```
Dim x as integer
Dim t as string
t = CStr(x)    'Converts the integer value of x to a string value in t
```

## CVar

Converts expressions to a variant data type.

**Syntax**

**CVar**(*Exp*)

*Exp*:

A valid string, number or Variant containing a value recognizable as a string or number.

**Return Value**

Returns the value of the expression (*Exp*) provided in the argument as a variant data type.

**Related Functions**

CDate | CDbl | CInt | CLng | CSng | CStr

**Example**

```
Dim x as integer
Dim v as variant
v = CVar(x)    'Converts the integer value of x to a variant value in v
```

## Date and time formatting/conversion

Vijeo Citect uses the following formatting/conversion functions:

| | |
|---|---|
| DateSerial | Constructs a date value. |
| TimeSerial | Constructs an time value. |

## DateSerial

Constructs a date value from the given Year, Month, and Day arguments passed to the function. The DateSerial function expects all three parameters to be valid. Date values in CitectVBA are evaluated using the Gregorian Calendar.

**Syntax**

**DateSerial**(*year,month,day*)

*year, month, day*:

The year, month and day as integers.

**Return Value**

Returns a Variant (of date data type) containing a date value corresponding to the Year, Month and Day values that were passed in to the function.

**Related Functions**

TimeSerial

**Example**

```
Dim varMyBDate
varMyBDate = DateSerial(1958, 7, 08)
' constructs and stores date value.
```

## TimeSerial

Constructs a time value serially from the given Hrs, Mins, and Secs arguments passed to the function. The TimeSerial Function expects all three arguments to be valid.

**Syntax**

**TimeSerial**(*hours,minutes,seconds*)

*hours, minutes, seconds*:

The hours, minutes and seconds to be converted to serial form as integers.

**Return Value**

Returns a Variant (of date data type) containing a time value corresponding to the Hrs, Mins, and Secs values that were passed in to the function.

**Related Functions**

DateSerial

**Example**

```
Dim varMyTime
varMyTime = TimeSerial(14, 35, 17)
' stores time as 2:35:17 PM
```

## Number and string conversion

Vijeo Citect uses the following functions for converting and formatting numbers and strings:

| | |
|---|---|
| Format | Formats a string, number, or variant to the format expression (fmt ). |
| Hex | Converts a value to a string representing the hex value. |
| Oct | Converts a value to a string representing the octal value. |
| Str | Converts a value to a string containing numeric characters. |
| Val | Converts a string containing numeric characters to a numeric value. |

## Format

Formats a string, number, or variant to the format expression *fmt*. The Format function expects the argument *Exp* to be a valid expression to be formatted.

The Format function expects the argument *fmt* to be a string of characters that specify how the expression is to displayed, or the name of a commonly used format that has been predefined in CitectVBA. Do not mix different type format expressions in a single *fmt* parameter.

If the fmt parameter is omitted or is zero-length and the expression parameter is a numeric, Format[$] provides the same functionality as the Str[$] function by converting the numeric value to the appropriate return data type. Positive numbers convert to strings using Format[$] lack the leading space reserved for displaying the sign of the value, whereas those converted using Str[$] retain the leading space.

To format numbers, you can use the commonly used formats that have been predefined in CitectVBA, or you can create user-defined formats with standard characters that have special meaning when used in a format expression.

### Syntax

**Format**(*Exp [,fmt]*)

*Exp*:

A valid string, number or Variant containing a value recognizable as a string or number.

### Return Value

Returns a formatted string.

**Predefined numeric format names**

- *General* - Display the number as is, with no thousand Separators
- *Fixed* - Display at least one digit to the left and two digits to the right of the decimal separator.

- *Standard* - Display number with thousand separator, if appropriate; display two digits to the right of the decimal separator.
- *Percent* - Display number multiplied by 100 with a percent sign (%) appended to the right' display two digits to the right of the decimal separator.
- *Scientific* - Use standard scientific notation.
- *True/False* - Display False if number is 0, otherwise display True.

**User-defined number formats**

- *Null string* - Display the number with no formatting.
- *0* - Digit placeholder.

**Display a digit or a zero**

If the number being formatted has fewer digits than there are zeros (on either side of the decimal) in the format expression, leading or trailing zeros are displayed. If the number has more digits to the right of the decimal separator than there are zeros to the right of the decimal separator in the format expression, the number is rounded to as many decimal places as there are zeros. If the number has more digits to left of the decimal separator than there are zeros to the left of the decimal separator in the format expression, the extra digits are displayed without modification.

- **Digit placeholder**(#). Displays a digit or nothing. If there is a digit in the expression being formatted in the position where the # appears in the format string, displays it; otherwise, nothing is displayed.
- **Decimal placeholder**(.). The decimal placeholder determines how many digits are displayed to the left and right of the decimal separator.
- **Percentage placeholder.**(%) The percent character (%) is inserted in the position where it appears in the format string. The expression is multiplied by 100.
- **Thousand separator**(,). The thousand separator separates thousands from hundreds within a number that has four or more places to the left of the decimal separator. Use of this separator as specified in the format statement contains a comma surrounded by digit placeholders(0 or #). Two adjacent commas or a comma immediately to the left of the decimal separator (whether or not a decimal is specified) means "scale the number by dividing it by 1000, rounding as needed."
- **Scientific format**(E-E+e-e+). If the format expression contains at least one digit placeholder (0 or #) to the right of E-,E+,e- or e+, the number is displayed in scientific formatted E or e inserted between the number and its exponent. The number of digit placeholders to the right determines the number of digits in the exponent. Use E- or e- to place a minus sign next to negative exponents. Use E+ or e+ to place a plus sign next to positive exponents.
- **Time separator**(:). The actual character used as the time separator depends on the Time Format specified in the International section of the Control Panel.

- **Date separator**(/). The actual character used as the date separator in the formatted out depends on Date Format specified in the International section of the Control Panel.
- Display a literal character (- + $ ( )). To display a character other than one of those listed, precede it with a backslash (\).
- Display the next character in the format string (\). The backslash itself isn't displayed. To display a backslash, use two backslashes (\ \).

**Note:** Examples of characters that can't be displayed as literal characters are the date- and time- formatting characters (a,c,d,h,m,n,p,q,s,t,w,y, and /:), the numeric formatting characters(#,0,%,E,e,comma, and period), and the string- formatting characters (@,&,<,>, and !).

- Display the string inside the double quotation marks ("String"). To include a string in fmt from within CitectVBA, you need to use the ANSI code for a double quotation mark Chr(34) to enclose the text.
- Display the next character as the fill character (*). Any empty space in a field is filled with the character following the asterisk.

Unless the *fmt* argument contains one of the predefined formats, a format expression for numbers can have from one to four sections separated by semicolons.

| If you use | The result is |
|---|---|
| One section | The format expression applies to all values. |
| Two | The first section applies to positive values, the second to negative sections values. |
| Three | The first section applies to positive values, the second to negative sections values, and the third to zeros. |
| Four | The first section applies to positive values, the second to negative section values, the third to zeros, and the fourth to Null values. |

The following example has two sections: the first defines the format for positive values and zeros; the second section defines the format for negative values.

"$#,##0; ($#,##0)"

If you include semicolons with nothing between them. the missing section is printed using the format of the positive value. For example, the following format displays positive and negative values using the format in the first section and displays "Zero" if the value is zero.

```
"$#,##0;;\Z\e\r\o"
```

Some sample format expressions for numbers are shown below. (These examples assume the Country is set to United States in the International section of the Control Panel.) The first column contains the format strings. The other columns contain the output the results if the formatted data has the value given in the column headings.

| Format (fmt) | Positive 3 | Negative 3 | Decimal .3 | Null |
|---|---|---|---|---|
| Null string | 3 | -3 | 0.3 | |
| 0 | 3 | -3 | 1 | |
| 0.00 | 3.00 | -3.00 | 0.30 | |
| #,##0 | 3 | -3 | 1 | |
| #,##0.00;;;Nil | 3.00 | -3.00 | 0.30 | Nil |
| $#,##0;($#,##0) | $3 | ($3) | $1 | |
| $#,##0.00;($#,##0.00) | $3.00 | ($3.00) | $0.30 | |
| 0% | 300% | -300% | 30% | |
| 0.00% | 300.00% | -300.00% | 30.00% | |
| 0.00E+00 | 3.00E+00 | -3.00E+00 | 3.00E-01 | |
| 0.00E-00 | 3.00E00 | -3.00E00 | 3.00E-01 | |

Numbers can also be used to represent date and time information. You can format date and time serial numbers using date and time formats or number formats because date/time serial numbers are stored as floating-point values.

To format dates and times, you can use either the commonly used format that have been predefined or create user-defined time formats using standard meaning of each:

| | |
|---|---|
| General | Display a date and/or time. for real numbers, display a date and time. (e.g. 4/3/93 03:34 PM); If there is no fractional part, display only a date (e.g. 4/3/93); if there is no integer part, display time only (e.g. 03:34 PM). |

| | |
|---|---|
| Long Date | Display a Long Date, as defined in the International section of the Control Panel. |
| Medium | Display a date in the same form as the Short Date, as defined in the international section of the Control Panel, except spell out the month abbreviation. |
| Short Date | Display a Short Date, as defined in the International section of the Control Panel. |
| Long Time | Display a Long Time, as defined in the International section of the Control panel. Long Time includes hours, minutes, seconds. |
| Medium | Display time in 12-hour format using hours and minuets and the Time AM/PM designator. |
| Short Time | Display a time using the 24-hour format (e.g. 17:45) |
| c | Display the date as dddd and display the time as ttttt. in the order. |
| d | Display the day as a number without a leading zero (1-31). |
| dd | Display the day as a number with a leading zero (01-31). |
| ddd | Display the day as an abbreviation (Sun-Sat). |
| ddddd | Display a date serial number as a complete date (including day , month, and year). |
| w | Display the day of the week as a number (1- 7 ). |
| ww | Display the week of the year as a number (1-53). |
| m | Display the month as a number without a leading zero (1-12). If m immediately follows h or hh, the minute rather than the month is displayed. |
| mm | Display the month as a number with a leading zero (01-12). If mm immediately follows h or hh, the minute rather than the month is displayed. |
| mmm | Display the month as an abbreviation (Jan-Dec). |
| mmmm | Display the month as a full month name (January-December). |
| q | display the quarter of the year as a number (1-4). |

| | |
|---|---|
| y | Display the day of the year as a number (1-366). |
| yy | Display the day of the year as a two-digit number (00-99) |
| yyyy | Display the day of the year as a four-digit number (100-9999). |
| h | Display the hour as a number without leading zeros (0-23). |
| hh | Display the hour as a number with leading zeros (00-23). |
| n | Display the minute as a number without leading zeros (0-59). |
| nn | Display the minute as a number with leading zeros (00-59). |
| s | Display the second as a number without leading zeros (0-59). |
| ss | Display the second as a number with leading zeros (00-59). |
| ttttt | Display a time serial number as a complete time (including hour, minute, and second) formatted using the time separator defined by the Time Format in the International section of the Control Panel. A leading zero is displayed if the Leading Zero option is selected and the time is before 10:00 A.M. or P.M. The default time format is h:mm:ss. |
| AM/PM | Use the 12-hour clock and display an uppercase AM/PM |
| am/pm | Use the 12-hour clock display a lowercase am/pm |
| A/P | Use the 12-hour clock display a uppercase A/P |
| a/p | Use the 12-hour clock display a lowercase a/p |
| AMPM | Use the 12-hour clock and display the contents of the 11:59 string (s1159) in the WIN.INI file with any hour before noon; display the contents of the 2359 string (s2359) with any hour between noon and 11:59 PM. AMPM can be either uppercase or lowercase, but the case of the string displayed matches the string as it exists in the WIN.INI file. The default format is AM/PM. |
| m/d/yy | 2/26/65 |
| d-mmmm-yy | 26-February-65 |
| d-mmmm | 26 February |

| | |
|---|---|
| mmmm-yy | February 65 |
| hh:nn | AM/PM 06:45 PM |
| h:nn:ss a/p | 6:45:15 p |
| h:nn:ss | 18:45:15 |
| m/d/yy/h:nn | 2/26/65 18:45 |

Strings can also be formatted with Format[$]. A format expression for strings can have one section or two sections separated by a semicolon.

| If you use | The result is |
|---|---|
| One section only | The format applies to all string data. |
| Two sections | The first section applies to string data, the second to Null values and zero-length strings. |

The following characters can be used to create a format expression for strings:

| | |
|---|---|
| @ | Character placeholder. Displays a character or a space. Placeholders are filled from right to left unless there is an !character in the format string. |
| & | Character placeholder. Display a character or nothing. |
| < | Force lowercase. |
| > | Force uppercase. |
| ! | Force placeholders to fill from left to right instead of right to left. |

**Related Functions**

Hex | Oct | Str | Val

**Example**

```
' Format Function Example
' This example shows various uses of the Format function to format values
' using both named and user-defined formats. For the date separator (/),
```

```
' time separator (:), and AM/ PM literal, the actual formatted output
' displayed by your system depends on the locale settings on which the code
' is running. When times and dates are displayed in the development
' environment, the short time and short date formats of the code locale
' are used. When displayed by running code, the short time and short date
' formats of the system locale are used, which may differ from the code
' locale. For this example, English/United States is assumed.

' MyTime and MyDate are displayed in the development environment using
' current system short time and short date settings.
MyTime = "08:04:23 PM"
MyDate = "03/03/95"
MyDate = "January 27, 1993"
MsgBox Now
MsgBox MyTime
MsgBox Second( MyTime ) & " Seconds"
MsgBox Minute( MyTime ) & " Minutes"
MsgBox Hour( MyTime ) & " Hours"
MsgBox Day( MyDate ) & " Days"
MsgBox Month( MyDate ) & " Months"
MsgBox Year( MyDate ) & " Years"
' Returns current system time in the system-defined long time format.
MsgBox Format(Time, "Short Time")
MyStr = Format(Time, "Long Time")
' Returns current system date in the system-defined long date format.
MsgBox Format(Date, "Short Date")
MsgBox Format(Date, "Long Date")
MyStr Format(MyTime, "h:n:s")    ' Returns "17:4:23".
MyStr Format(MyTime, "hh:nn:ss")' Returns "20:04:22 ".
MyStr Format(MyDate, "dddd, mmm d yyyy")' Returns "Wednesday, Jan 27 1993".
' If format is not supplied, a string is returned.
MsgBox Format(23)            ' Returns "23".
' User-defined formats.
MsgBox Format(5459.4, "##,##0.00")   ' Returns "5,459.40".
MsgBox Format(334.9, "###0.00")     ' Returns "334.90".
MsgBox Format(5, "0.00%")        ' Returns "500.00%".
MsgBox Format("HELLO", "<")       ' Returns "hello".
MsgBox Format("This is it", ">")    ' Returns "THIS IS IT".
```

## Hex

Converts a numeric value to a text string representing the hexadecimal value of the number.

The Hex function expects the argument *Num* to be a valid numeric value. It is rounded to nearest whole number before evaluation.

### Syntax

**Hex**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

Returns a text string containing the hexadecimal value of the numeric *Num* value provided in the argument.

**Related Functions**

Format | Oct | Str | Val

**Example**

```
Dim MyHex as String
MyHex = Hex(5)    'returns "5"
MyHex = Hex(10)    'returns "A"
MyHex = Hex(459)    'returns "1CB"
```

## Oct

Converts a numeric value to a text string representing the octal value of the number.

The Oct function expects the argument *Num* to be a valid numeric value. It is rounded to nearest whole number before evaluation.

**Syntax**

**Oct**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

Returns a text string containing the octal value of the numeric *Num* value provided in the argument.

**Related Functions**

Format | Hex | Str | Val

**Example**

```
Dim MyOct as String
MyOct = Oct(4)        'returns "4"
MyOct = Oct(8)        'returns "10"
MyOct = Oct(459)        'returns "713"
```

## Str

Converts a numeric value to a text string containing numeric characters. The Str function expects the argument *Num* to be a valid numeric value.

The Str function is often used to prepare a numerical value for display as a string in a caption, label, string field, or string expression.

The Str function performs the opposite of the Val function, which converts a text string containing numeric characters to a numeric value.

> **Note:** Please remember the data type coercion considerations with variant data types. See Variants.

**Syntax**

**Str**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

Returns a string containing the numeric character representation of the numeric *Num* value provided in the argument.

The Str function reserves the first return string character for the sign of *Num*. If *Num* is positive, a leading space is used and the plus sign is implied.

**Related Functions**

Format | Hex | Oct | Val

**Example**

```
Dim vntVar         ' declare result holder variable

vntVar = Str()             ' returns " "
vntVar = Str(65)         ' returns " 65"
vntVar = Str(97.578)         ' returns " 97.578"
vntVar = Str(-97.578)     ' returns "-97.578"
```

## Val

Converts a text string containing numeric characters to a numeric value. The Val function expects the argument *Str* to be a valid string expression. The Val function stops reading the string when it reaches a non numeric character.

Symbols such as dollar signs and commas are not recognised; however, radix prefixes for octal (&0) and hexadecimal (&H) are. Blanks, tabs and linefeeds are stripped out from the return.

The Val function performs the opposite of the Str function, which converts a numeric value to a text string containing numeric characters.

**Syntax**

**Val**(*Str*)

*Str*:

A string or expression that can represent a valid text value.

**Return Value**

Returns the numeric value of a string of characters extracted from the *Str* provided in the argument.

**Related Functions**

Format | Hex | Oct | Str

**Example**

```
Dim vntVar          ' declare result holder variable
vntVar = Val("65")        ' returns 65
vntVar = Val("90 Main St.")     ' returns 90
vntVar = Val("12+34+56")     ' returns 12
vntVar = Val(" 12 34 56 ")     ' returns 123456
vntVar = Val("&0FF")        ' returns
vntVar = Val("Zoe")        ' returns 0
```

## Declarations

CitectVBA declarations allow you to manipulate and control variables and constants. The Declaration functions and statements predefined in CitectVBA are:

| | |
|---|---|
| CreateObject function | Creates an OLE Automation object reference |
| Const statement | Assigns a symbolic name to a constant value. |
| Declare statement | Declare references to external procedures in a DLL. |

| | |
|---|---|
| Dim statement | Allocates storage for, and declares the data type of, variables and arrays. |
| IsDate | Determines if a Variant parameter can be converted to a date. |
| IsEmpty | Determines if a Variant parameter has been initialized. |
| IsNull | Determines if a Variant contains NULL. |
| IsNumeric | Determines if a Variant can be converted to a numeric data type. |
| Nothing keyword | Releases an OLE Automation object reference from a variable of object type. |
| Option Base statement | Declares the default lower bound for array subscripts. |
| Option Compare statement | Determines the default string comparison method. Forces explicit declaration of all variables. |
| ReDim statement | Used to size or resize a dynamic array. |
| Set statement | Assigns an OLE Automation object reference to a variable of object type. |
| Static statement | Allocates storage for, and declares the data type of, variables and arrays. |
| VarType | Indicates the data type used within the Variant. |

## CreateObject

Creates a new OLE Automation object and assigns a reference to the object.

**Syntax**

Set *objVarName* = CreateObject(*objClassName)*

*objVarName*:

The required name of the variable receiving the reference.

*objClassName*:

The required class name of the object to be created.

The object variable *objVarName* must be declared before it can be set to reference an OLE Automation object.

**Related Functions**

Dim | Set

**Example**

```
' create variable to store object reference
Dim objWord as Object
' create object and assign reference to variable
Set objWord = CreateObject( "Word.Document" )
' insert appropriate VBA code here to manipulate Word object
' release reference
Set objWord = Nothing.
```

## Const

Assigns a symbolic name to a constant value using the following syntax:

```
Const VarName [As DataType] = Expression
```

A constant must be defined before it is used. Unlike variables, constants are assigned values when initialized and retain that same value during the life of the constant.

Constant statements can only be declared and assigned using simple expressions. Constants can NOT be assigned values from variables, user-defined functions, intrinsic CitectVBA functions (such as Chr), or from any expression that involves an operator.

Constants declared in a Sub or Function procedure are local to that procedure. A constant declared outside a procedure has modular scope to all procedures within the same CitectVBA file module. See Scope of CitectVBA.

Constants can be used anywhere in your CitectVBA code where you could use a CitectVBA expression.

If you use Const outside a procedure its scope becomes global.

A type declaration character may also be used. However if none is used, CitectVBA will automatically assign one of the following data types to the constant: long (if it is a long or integer); Double (if a decimal place is present); or String (if it is a string).

**Syntax**

**Const**(*VarName*, *Exp*)

*VarName*:

A string representing a valid variable name.

*Exp:*

A valid string, number or Variant containing a value recognizable as a string or number.

**Related Functions**

Dim | ReDim | Static

**Example**

```
' Correct declaration examples
' long assignment
Const Seven = 7
' double assignment
Const Pi = 3.14159
' string assignment
Const Lab = "Laboratory"

' Incorrect declaration examples
' NOTE that the following declarations demonstrate incorrect
assignments
' because each contains an operator
Const conPi = 4 * Atn(1)
' will cause a CitectVBA compile error
Const conDegToRad = (conPi / 180)
' will cause a CitectVBA compile error
```

## Declare

The Declare statement is used at module (file) level to declare references to external procedures in a dynamic-link library (DLL).

**Syntax**

**Declare Function**<*FunctionName*> *Lib "<LibName>" [Alias "<AliasName>"] [([<ArgList>])] [As <ReturnType>]*

*FunctionName*:

The required name of the function being declared.

*LibName*:

The required DLL filename containing the function being called.

*AliasName*:

The optional function name within the DLL being called.

*ArgList*:

The optional argument/s of the function.

*ReturnType*:

The optional return data type.

**Related Functions**

Dim

**Example**

```
Declare Function GetWinTempPath Lib "kernel32" _
Alias "GetTempPathA" _
(ByVal nBufferLength As Long, _
ByVal lpBuffer As String) As Long
```

## Dim

The Dim statement allocates storage for, and declares the data type of, variables and arrays in a module.

The To clause in the array subscript range of a Dim statement provides a more flexible way to control the lower bound of an array. If you don't explicitly set the lower bound with a To clause, the Option Base setting (if used) comes into affect, or defaults to zero (if not used).

**Syntax**

**Dim** *VariableName*[(*Subscripts*)] [As *DataType*]

*VariableName*:

The name of the variable or array being declared (dimensioned).

*Subscripts*:

The optional subscript range (dimensions) for an array in parentheses.

*DataType*:

The optional data type declaration for the variable or array.

**Related Functions**

Const | ReDim | Static

**Example**

```
Dim bytVar As Byte
Dim binVar As Boolean
Dim strVar As String
Dim intVar As Integer
Dim lngVar As Long
Dim sngVar As Single
Dim dblVar As Double
Dim vntVar As Variant
Dim objVar As Object
Dim dtmVar As Date
```

```
Dim daysOfWeek() As String ' declares an array variable to hold strings

Dim monthsOfYear(12) As Date ' declares an array variable to hold 12 strings
Dim users(,) As String ' declares a two dimensional array to hold strings
Dim usernames(5,5) As String ' declares a two dimensional 5 x 5 array to hold
strings

Dim MyArray(1 To 10, 5 To 15, 10 To 20) ' declares the three dimensional array
 MyArray and specifies the upper and lower bounds of each dimension
```

## IsDate

Determines if an expression can be converted to a date.

The required *Date* argument is a Variant containing a date expression or string expression recognizable as a date or time value.

### Syntax

**IsDate**(*Date*)

*Date*:

A string or expression that can represent a date value. This includes any combination of date literals, numbers that look like dates, strings that look like dates, and dates from functions.

### Return Value

Returns a Boolean **True** or **False**.

### Related Functions

IsEmpty | IsNull | IsNumeric | VarType

### Example

```
Dim x As String
Dim MArray As Integer, MCheck
MArray = 345
x = "January 1, 1987"
MCheck = IsDate(MArray)
MChekk = IsDate(x)
MArray1 = CStr(MArray)
MCheck1 = CStr(MCheck)
Print MArray1 & " is a date " & Chr(10) & MCheck
Print x & " is a date" & Chr(10) & MChekk
```

## IsEmpty

Determines if a variant parameter has been initialised.

The required *Exp*argument is a variant containing a numeric or string expression. However, because `IsEmpty` is used to determine if individual variables are initialised, the *Exp*argument is most often a single variable name.

`IsEmpty` returns **True** if the variable is un-initialised, or is explicitly set to **Empty**; otherwise, it returns **False**. **False** is returned if *Exp*contains more than one variable.

> **Note:**`IsEmpty` only returns meaningful information for variants.

**Syntax**

**IsEmpty**(*Exp*)

*Exp*

A valid string, number or Variant containing a value recognizable as a string or number.

**Related Functions**

Returns a Boolean **True** or **False**.

**Related Functions**

IsDate | IsNull | IsNumeric | VarType

**Example**

```
Dim x    ' Empty
x = 5    ' Not Empty - Long
x = Empty    ' Empty
y = x    ' Both Empty
MsgBox "x" & " IsEmpty: " & IsEmpty(x)
```

## IsNull

Determines if a Variant contains **Null**.

IsNull returns **True** if expression is **Null**; otherwise, IsNull returns **False**. If *Exp* consists of more than one variable, **Null** in any constituent variable causes **True** to be returned for the entire expression.

The **Null** value indicates that the Variant contains no valid data. **Null** is not the same as **Empty**, which indicates that a variable has not yet been initialised. It is also not the same as a zero-length string (" "), which is sometimes referred to as a null string.

> **Note:** Use the IsNull function to determine whether *VarName* contains a **Null** value. Expressions that you might expect to evaluate to **True** under some circumstances,

such as If Var = Null and If Var <> Null, are always **False**. This is because any expression containing a **Null** is itself **Null** and, therefore, **False**.

**Syntax**

**IsNull**(*Exp*)

*Exp*

A valid string, number or Variant containing a value recognizable as a string or number.

**Return Value**

Returns a Boolean **True** or **False**.

**Related Functions**

IsDate | IsEmpty | IsNumeric | VarType

**Example**

```
Dim MyVar, MyCheck
MyCheck = IsNull(MyVar)        ' Returns False.

MyVar = ""
MyCheck = IsNull(MyVar)        ' Returns False.

MyVar = Null
MyCheck = IsNull(MyVar)        ' Returns True.
```

## IsNumeric

Determines if a variant can be evaluated as a number.

The required *Exp* argument is a variant containing a numeric expression or string expression that can be evaluated as a number.

IsNumeric returns **False** if *Exp* is a date expression.

**Syntax**

**IsNumeric**(*Exp*)

*Exp*

A valid string, number or Variant containing a value recognizable as a string or number.

**Return Value**

Returns a Boolean **True** or **False**.

**Related Functions**

IsDate | IsEmpty | IsNull | VarType

**Example**

```
Dim TestVar    ' Declare variable.
TestVar = InputBox("Please enter a number, letter, or symbol.")
If IsNumeric(TestVar) Then    ' Evaluate variable.
    MsgBox "Entered data is numeric."    ' Message if number.
Else
    MsgBox "Entered data is not numeric."    ' Message if not.
End If
```

## Nothing

Releases an OLE Automation object reference from a variable of object type. The Nothing keyword is used in a Set statement.

In the following declaration syntax example, each placeholder shown inside arrow brackets ( <placeholder> ) should be replaced in any actual code with the value of the item that it describes. The arrow brackets and the word they contain should not be included in the statement, and are shown here only for your information.

**Syntax**

**Set***objVarName* = **Nothing**

*objVarName*:

The required name of the variable receiving the reference.

The **nothing** keyword should be used when you are finished with an object, to clear any variables that reference the object, so the object can be released from memory.

**Related Functions**

CreateObject | Function | Set

**Example**

```
' create variable to store object reference
Dim objWord as Object
' create object and assign reference to variable
Set objWord = CreateObject( "Word.Document" )
' insert appropriate VBA code here to manipulate Word object
' release reference
Set objWord = Nothing
```

## Option Base

Declares the default lower bound for array subscripts.

The Option Base statement is optional. If used, it can appear only once in a CitectVBA file, and must be used before you declare the dimensions of any arrays.

The To clause in the array subscript range of a Dim statement provides a more flexible way to control the lower bound of an array. If you don't explicitly set the lower bound with a To clause, the Option Base setting (if used) comes into affect, or defaults to zero (if not used).

**Syntax**

**Option Base***Num*

*Num*:

An Integer or expression representing a valid numeric value. The value of the 'number' parameter must be either 0 or 1. The default is 0.

**Related Functions**

Dim | ReDim

**Example**

The example below uses the Option Base statement to override the default base array subscript value of 0.

```
' Module level statement
Option Base 1

' Create the array
Dim Arr(20)
' Declare message variables
Dim Msg As String
Dim NL as String
' Define newline
NL = Chr(10) & Chr(13)
' Create message
Msg = "The lower bound is " & LBound(Arr) & "."
Msg = Msg & NL & "The upper bound is " & UBound(Arr) & "."
' Display message
MsgBox Msg
```

## Option Compare

Determines how strings are compared within a CitectVBA module. The optional Option Compare statement if used, must be placed at the top of the CitectVBA file along with any other Option declarations.

If an Option Compare statement is not included, the default text comparison method is Binary.

**Syntax**

**Option Compare {Binary | Text}**

**Related Functions**

InStr | StrComp

**Example**

```
Option Compare Binary
Dim vntResult as Variant
vntResult = StrComp("CitectVBA rules!", "Citectvba Rules!")
' returns 1 (strings unequal)
```

**Example**

```
Option Compare Text
Dim vntResult as Variant
vntResult = StrComp("CitectVBA rules!", "Citectvba Rules!")
' returns 0 (strings equal)
```

## ReDim

Used to size or resize a dynamic array that has already been declared using the Dim statement with empty parentheses.

Use the ReDim statement to change the number of elements in an array, but not to change the number of dimensions in an array or the type of the elements in the array.

**Syntax**

**ReDim***VariableName*(*Subscripts*)

*VariableName*:

The name of the variable or array being redimensioned.

*Subscripts*:

An Integer or expression representing a valid To numeric value range when declaring the dimensions of an variable array. Up to 60 multiple dimensions may be declared.

The subscripts argument uses the following syntax:

```
[lower To] upper [,[lower To] upper] . . .
```

When not explicitly stated in **lower**, the **lower** bound of an array is controlled by the Option Base statement. The lower bound is zero if no Option Base statement is present in the CitectVBA file.

**Related Functions**

Dim | Const | Static

**Example**

```
Dim TestArray() As Integer
Dim I
ReDim TestArray(10)
For I = 1 To 10
    TestArray(I) = I + 10
    Print TestArray(I)
Next I
```

## Set

Assigns an OLE Automation object reference to a variable of object type.

**Syntax**

**Set** *objVarName* **= CreateObject(***objClassName***)** | **Nothing**

*objVarName*:

The required name of the variable receiving the reference.

*objClassName*:

The required class name of the object to be created.

Use the **Nothing** keyword to release the object reference.

The object variable *objVarName* must be declared before it can be set to reference an OLE Automation object.

**Related Functions**

CreateObject | Nothing

**Example**

```
' create variable to store object reference
Dim objWord as Object
' create object and assign reference to variable
Set objWord = CreateObject( "Word.Document" )
' insert appropriate VBA code here to manipulate Word object
' release reference
Set objWord = Nothing
```

## Static

The Static statement allocates storage for-and declares the data type of-variables and arrays that will retain their values between subsequent references. Static variables are more commonly used within procedures (subroutines and functions), and have local scope.

**Syntax**

**Static** *VariableName*[(*Subscripts*)] [As *DataType*]

*VariableName*:

The required name of the variable being declared (dimensioned).

*Subscripts*:

The optional subscript range for an array.

*DataType*:

The optional CitectVBA data type declaration for the variable.

**Related Functions**

Const | Dim | ReDim

**Example**

```
Static bytVar As Byte
Static binVar As Boolean
Static strVar As String
Static intVar As Integer
Static lngVar As Long
Static sngVar As Single
Static dblVar As Double
Static vntVar As Variant
Static objVar As Object
Static dtmVar As Date
Static udtVar As <UserDefinedTypeName>
```

## VarType

Determines the data type of a Variant variable.

The required *VarName* argument is a Variant containing any variable (except user-defined type).

**Syntax**

**VarType**(*VarName*)

*VarName:*

A string representing a valid variable name.

**Return Value**

These are the return values:

| Return Value | Data Type |
|---|---|
| 0 | Empty |
| 1 | Null |
| 2 | Integer |
| 3 | Long |
| 4 | Single |
| 5 | Double |
| 6 | Not Applicable |
| 7 | Date/Time |
| 8 | String |

**Related Functions**

IsDate | IsEmpty | IsNull | IsNumeric

**Example**

```
Dim IntVar, StrVar, DateVar, MyCheck
' Initialize variables.
IntVar = 459
StrVar = "Hello World"
DateVar = #2/12/69#
MyCheck = VarType(IntVar)    ' Returns 2.
MyCheck = VarType(DateVar)    ' Returns 7.
MyCheck = VarType(StrVar)    ' Returns 8.
```

## Date and Time Functions

CitectVBA date and time functions let you make use of your computer's system time and date.

The date and time functions predefined in CitectVBA are:

| | |
|---|---|
| Date function | Determines the current system date according to the setting of the computer's system time. |
| Date statement | Sets the current system date. |
| DateSerial function | Constructs a date value. |
| DateValue function | Calculates a date. |
| Day function | Calculates the day. |
| Hour function | Extracts the hours value from an expression (Time ). |
| Minute function | Extracts the minutes value from an expression (Time ). |
| Month function | Calculates the month. |
| Now function | Determines the current date and time according to the setting of the computer's system date and time. |
| Second function | Extracts the seconds value from an expression (Time ). |
| Time function | Determines the current time according to the setting of the computer's system time. |
| Time (statement) | Sets the current system time. |
| Timer event | Used to track elapsed time. |
| TimeSerial function | Constructs an time value. |
| TimeValue function | Calculates a time. |
| WeekDay function | Calculates the weekday value of a date. |
| Year function | Calculates the year. |

## Date

Gets the current date in string format.

Time/Date functions can only be used with dates between 1980 and 2035. You should check that the date you are using is valid with Cicode similar to the following:

```
IF StrToDate(Arg1) > 0 THEN
    ...
ELSE
    ...
END
```

### Syntax

**Date**(*[Format]*)

*Format:*

The format required:

*2* - Short date format, dd/mm/yy

*3* - Long date format, day month year

*9* - Extended date format, dd/mm/yyyy

If omitted, the default Format is 2. All of these formats follow the Regional Settings found in the Windows Control Panel.

### Return Value

The current date (in string format).

### Related Functions

Time | TimeToStr | TimeCurrent

### Example

```
/* If the current system date is 3rd November 1991 and the Windows date format is
 dd/mm/yy; */
str = Date();
! Sets str to "3/11/91".
str = Date(2);
! Sets str to "3/11/91".
str = Date(3);
! Sets str to "3rd November 1991".
```

### See Also

Time/Date Functions

## Date statement

Sets the current system date.

The DateValue literal is displayed in short date format using the locale settings of your development system. To view the locale settings for your system, in Windows, select Start, Settings, Control Panel, Regional Options, Date. For example: in Australia, the short date format is represented as d/MM/yyyy.

**Syntax**

**Date =** *dateVariable*

*dateVariable*:

You must enclose a Date literal within number signs (# #), for example #31/5/1993#.

**Related Functions**

Time (statement)

**Example**

```
Dim varCitectVBAReleaseDate
varCitectVBAReleaseDate = #01/07/2001#
Date = varCitectVBAReleaseDate
' sets system date to CitectVBA Release Date
```

## DateSerial

Constructs a date value from the given Year, Month, and Day arguments passed to the function. The DateSerial function expects all three parameters to be valid. Date values in CitectVBA are evaluated using the Gregorian Calendar.

**Syntax**

**DateSerial**(*year,month,day*)

*year, month, day*:

The year, month and day as integers.

**Return Value**

Returns a Variant (of date data type) containing a date value corresponding to the Year, Month and Day values that were passed in to the function.

**Related Functions**

TimeSerial

**Example**

```
Dim varMyBDate
varMyBDate = DateSerial(1958, 7, 08)
' constructs and stores date value.
```

### DateValue

Calculates a date from the given date argument passed to the function. Date values in CitectVBA are evaluated using the Gregorian Calendar. The DateValue function expects the argument value (*Date)*to be a string or any expression that can represent a date.

#### Syntax

**DateValue**(*Date*)

*Date*:

A string or expression that can represent a date value. This includes any combination of date literals, numbers that look like dates, strings that look like dates, and dates from functions.

#### Return Value

Returns a variant (of date data type) corresponding to the string date expression that was passed in.

#### Related Functions

TimeValue

#### Example

```
Dim varMyBDate
varMyBDate = DateValue("1958/07/08")
' stores date value.
```

## Day

Calculates the day from the given date argument passed to the function using the Gregorian Calendar.

#### Syntax

**Day**(*Date*)

*Date*:

A string or expression that can represent a date value. This includes any combination of date literals, numbers that look like dates, strings that look like dates, and dates from functions.

#### Return Value

Returns a variant date corresponding to the date expression that was passed in.

**Related Functions**

Date | Year | Month | WeekDay

**Example**

```
Dim varMyBDate, varMyDay
varMyBDate = #July 8, 1958#
varMyDay = Day(varMyBDate)
' stores 8 for day value.
```

## Hour

Calculates the hour value from the given time argument passed to the function.

**Syntax**

**Hour**(*Time*)

*Time*:

A string or expression that can represent a time value. This includes and combination of time literals, numbers that look like times, strings that look like times, and times from functions.

**Return Value**

Returns an integer between 0 and 23 that is the hour of the parameter (*Time*).

**Related Functions**

Minute | Second

**Example**

```
Dim varMyHour, varMyTime
varMyTime = "08:04:23 PM"
varMyHour = Hour(varMyTime)
' stores hours value.
```

## Minute

Calculates the minute value from the given time argument passed to the function.

**Syntax**

**Minute**(*Time*)

*Time*:

A string or expression that can represent a time value. This includes and combination of time literals, numbers that look like times, strings that look like times, and times from functions.

**Return Value**

Returns an integer between 0 and 59 representing the minute of the parameter (*Time*).

**Related Functions**

Hour | Second

**Example**

```
Dim varMyMin, varMyTime
varMyTime = "08:04:23 PM"
varMyMin = Minute(varMyTime)
' stores minutes value.
```

## Month

Calculates the month from the given date argument passed to the function using the Gregorian Calendar.

**Syntax**

**Month**(*Date*)

*Date:*

A string or expression that can represent a date value. This includes any combination of date literals, numbers that look like dates, strings that look like dates, and dates from functions.

**Return Value**

Returns an integer between 1 and 12 inclusive, that represents the month of the year.

**Related Functions**

Date | Year | WeekDay | Day

**Example**

```
Dim varMyBDate, varMyMonth
varMyBDate = "08 July 1958"
varMyMonth = Month(varMyBDate)
' returns 7 for July
```

## Now

Determines the current date and time according to the setting of the computer's system date and time using the Gregorian Calendar. Unlike other functions, Now does not require trailing parentheses.

**Syntax**

**Now**()

**Return Value**

The Now function returns a Variant data type containing a date and time value that is stored internally as a double data type.

The number represents a date and time from January 1, 100 through December 31, 9999. Numbers to the left of the decimal point represent the date and numbers to the right represent the time.

**Related Functions**

Date | Time | Timer

**Example**

```
Dim vntToday
vntToday = Now
' stores current system date and time.
```

## Second

Calculates the second value from the given time argument passed to the function.

**Syntax**

**Second**(*Time*)

*Time*:

A string or expression that can represent a time value. This includes and combination of time literals, numbers that look like times, strings that look like times, and times from functions.

**Return Value**

Returns an integer that is the second portion of the parameter (*Time*).

**Related Functions**

Hour | Minute

**Example**

```
Dim varMySec, varMyTime
varMyTime = "08:04:23 PM"
varMySec = Second(varMyTime)
' stores seconds value.
```

## Time

Gets the current time in string format.

Time/date functions can only be used with dates from 1980 to 2035. You should check that the date you are using is valid with Cicode similar to the following:

```
IF StrToDate(Arg1) > 0 THEN
    ...
ELSE
    ...
END
```

### Syntax

**Time**(*[Format]*)

*Format:*

The format of the time:

*0* - Short time format, hh:mm

*1* - Long time format., hh:mm:ss

If omitted, the default *Format* is 0.

### Return Value

The current time (as a string).

### Related Functions

Date

### Example

```
! If the current time is 10:45:30;
Variable=Time();
! Sets Variable to "10:45".
Variable=Time(0);
! Sets Variable to "10:45".
Variable=Time(1);
! Sets Variable to "10:45:30".
```

### See Also

Time/Date Functions

## Time (statement)

Sets the system time.

**Syntax**

**Time** = *timeVariable*

*timeVariable*:

You must enclose a Time literal within number signs (# #), for example #12:14:00 PM#.

**Related Functions**

Date statement

**Example**

```
' Time statement example
Dim varMyTime
' Assign a time.
varMyTime = #4:35:17 PM#
' Set system time to variant varMyTime.
Time = varMyTime
```

## Timer

The Timer event is used to track elapsed time or can be displayed as a stopwatch in a dialog.

**Syntax**

**Timer**()

**Return Value**

The number of seconds since midnight.

**Related Functions**

Date | Time | Now

**Example**

```
Dim TS As Single
Dim TE As Single
Dim TEL As Single
```

```
TS = Timer
MsgBox "Starting Timer"
TE = Timer
TT = TE - TS
Print TT
```

## TimeSerial

Constructs a time value serially from the given Hrs, Mins, and Secs arguments passed to the function. The TimeSerial Function expects all three arguments to be valid.

### Syntax

**TimeSerial**(*hours,minutes,seconds*)

*hours, minutes, seconds*:

The hours, minutes and seconds to be converted to serial form as integers.

### Return Value

Returns a Variant (of date data type) containing a time value corresponding to the Hrs, Mins, and Secs values that were passed in to the function.

### Related Functions

DateSerial

### Example

```
Dim varMyTime
varMyTime = TimeSerial(14, 35, 17)
' stores time as 2:35:17 PM
```

## TimeValue

Calculates a time. The TimeValue function expects the argument value (*Time*) to be a string or any expression that can represent a time value.

### Syntax

**TimeValue**(*Time*)

*Time:*

A string or expression that can represent a time value. This includes and combination of time literals, numbers that look like times, strings that look like times, and times from functions.

### Return Value

Returns a variant (of date data type) corresponding to the parameter (*Time*).

**Related Functions**

DateValue

**Example**

```
Dim varMyTime
varMyTime = TimeValue("2:35:17 PM")
' stores time as 14:35:17
```

## WeekDay

Calculates the weekday value of the given date argument passed to the function. Date values in CitectVBA are evaluated using the Gregorian Calendar.

**Syntax**

**WeekDay**(*Date*)

*Date*:

A string or expression that can represent a date value. This includes any combination of date literals, numbers that look like dates, strings that look like dates, and dates from functions.

**Return Value**

Returns an integer between the range of 1-7 inclusive representing the whole number for the weekday:

| Return Value | Description |
|---|---|
| 1 | Sunday |
| 2 | Monday |
| 3 | Tuesday |
| 4 | Wednesday |
| 5 | Thursday |
| 6 | Friday |
| 7 | Saturday |

**Related Functions**

Date | Year | Month | Day

**Example**

```
Dim varMyBDate, varMyWeekDay
varMyBDate = #8/07/1958#
varMyWeekDay = WeekDay(varMyBDate)
' returns 3 (Tuesday)
```

## Year

Calculates the year from the given date argument passed to the function. Date values in CitectVBA are evaluated using the Gregorian Calendar.

**Syntax**

**Year**(*Date*)

*Date*:

A string or expression that can represent a date value. This includes any combination of date literals, numbers that look like dates, strings that look like dates, and dates from functions.

**Return Value**

Returns an integer representing a year 1930-2029 inclusive.

**Related Functions**

Date | Month | WeekDay | Day

**Example**

```
Dim varMyBDate, varMyYear
varMyDate = "08/07/58"
varMyYear = Year(varMyBDate)
' returns 1958
```

# File I/O Functions

CitectVBA file Input/Output (I/O) functions are provided to enable read and write disk file functionality.

The file I/O functions predefined in CitectVBA are:

| | |
|---|---|
| ChDir | Changes the system environment current directory on the specified drive. |

| | |
|---|---|
| ChDrive | Changes the system environment current drive to the specified drive. |
| Close | Closes the file/s previously opened with the Open statement. |
| CurDir, CurDir$ | Returns the current system environment path for the specified drive (*Drv*). |
| Dir | Returns a file or directory name that matches the given *File*and *Attrib* arguments. |
| EOF | Returns a boolean True or False value during file access that indicates whether the current position of an open file has reached the end of the file. |
| FileCopy | Copies a file from *Src* to *Dest*. |
| FileLen | Determines the byte length of a file. |
| FreeFile | Retrieves the next sequential system file number available for association with a file. |
| Get # | Reads data from a disk file into a variable. |
| GetAttr | Returns an Integer representing the attribute settings of a file, directory, or volume. |
| Input | Reads data from a Sequential file and assigns that data to variables. Input function returns characters from a file opened in Input or Binary mode. |
| Kill | Deletes files from disk. |
| Line Input # | Reads a single line from an open sequential file and assigns it to a String variable. |
| Loc | Returns a number indicating the current position within a file opened using the Open statement. |
| LOF | Returns a number indicating the byte length of a sequential file opened using the Open statement. |
| MkDir | Creates the directory specified in the *Path* parameter. |
| Name | Renames the disk file specified in the *OldFileName*parameter, to the name specified in the *NewFileName* parameter. |
| Open | Enables input/output (I/O) to a disk file. |

| | |
|---|---|
| Print (function) | Displays a message in the Vijeo Citect Kernel and the Cicode Editor output window. |
| Print # | Reads data from *OutputList* and writes that data to a sequential file. |
| Put # | Writes data from a variable to a disk file. |
| RmDir | Deletes the directory specified in the *Path* parameter. |
| Seek | Sets the current position within a file opened using the Open statement, ready for the next read or write action. |
| Write # | Writes data to a Sequential file opened in output or append mode and reads that data from a list of variables. |

## ChDir

ChDir statement changes the system environment current directory on the specified drive.

The parameter Path must be a string or expression that can represent a valid DOS file structure path value. The parameter Dir must be a string or expression that can represent a valid DOS file structure directory name. The Path and Dir parameters together, must be limited to less than 128 characters. The Path drive letter is optional, unless the directory is on another drive. The required Dir parameter must be a valid directory name.

**Note:** The file system keeps track of the current drive, and the current directory of every drive. Use the CurDir statement to determine the current directory. The current drive letter can be extracted from the Left character returned in the CurDir statement.

The ChDir statement changes the current directory but not the current drive. To change the current drive, use the ChDrive statement.

### Syntax

**ChDir***Path Dir*

*Path*:

A string or expression that can represent a valid DOS file structure path value. This includes a directory name, and may include a relative or static directory or folder structure and drive letter, in the order:

```
[<driveletter>:][\<rootdirectoryname>][\<subdirectory> ...
```

```
\<subdirectory>\] directoryname
```

Note that the path can be relative to the current directory. A single period represents the current directory (.), and two periods represent the parent directory of the current directory (..). For example:

- chdir .. ' changes to the parent directory of the current directory
- chdir ..\test ' changes to the test subdirectory of the parent directory

*Dir*:

A string or expression that can represent a valid DOS file structure directory name. Dir is not case sensitive. Dir is often used with the Path parameter.

**Related Functions**

ChDrive | CurDir, CurDir$ | Dir | MkDir | RmDir

**Example**

```
Dim strPath as String
strPath = CurDir()' store current path
ChDir "\"' change to root dir on current drive
<statements>' do stuff in root directory
ChDir strPath' change back to previous path
```

## ChDrive

Changes the system environment current drive to the specified drive.

The parameter *Drv* must be a string or expression that can represent a valid DOS file structure drive letter. The *Drv* may be local to the computer, or mapped from anywhere on the network connected to the computer. If *Drv* contains more than one letter, only the first character is used.

**Note:** The file system keeps track of the current drive, and the current directory of every drive. Use the CurDir statement to determine the current directory. The current drive letter can be extracted from the Left character returned in the CurDir statement.

The ChDrive statement changes the current drive but not the current directory on any drive. To change the current directory, use the ChDir statement.

**Syntax**

**ChDrive***Drv*

*Drv*:

A string or expression that can represent a valid DOS file structure drive letter. *Drv* is case insensitive and must end with a colon (:). The Drv may be local to the computer, or mapped from anywhere on the network connected to the computer. Drv is often included as part of the Path parameter.

**Related Functions**

ChDir | CurDir, CurDir$ | Dir | RmDir | MkDir

**Example**

```
Dim strCurPath as String
strCurPath = CurDir$()' store current path as string
ChDir "\"' change to root directory of current drive
<statements>' do stuff in root directory
ChDrive "C"' change to C drive (if not already there)
<statements>' do stuff in current directory on C drive
ChDrive strCurPath' change back to previous drive
ChDir strCurPath' change back to previous path
```

## Close

Closes the file(s) previously opened with the Open statement.

The optional *FileNumList* parameter can contain one or more valid file associated reference numbers using the following syntax:

```
[[#]FileNum] [, [#]FileNum] ...
```

where *Filenum* is any valid number associated with an open file.

If the Close statement is used without any arguments it closes all open files. When the Close statement is executed, the association of a file with its file number ends.

**Syntax**

**Close***FileNumList*

*FileNumList*:

Must contain one or more valid integer or numeric expression values representing associated file numbers using the following syntax:
[[#]filenumber] [, [#]filenumber] ... where filenumber is any valid number associated with an open file.

**Related Functions**

FileCopy | FreeFile | Kill | Name | Open

**Example**

```
Dim strFileContents As String
Dim strTemp As String
Open "c:\test.txt" For Input As #1    ' open file.
Do While Not EOF(1)        ' loop until end of file
    strTemp = Input(10, #1)    ' read next ten characters
    strFileContents = strFileContents & strTemp    ' join strings
Loop
Close #1
```

## CurDir, CurDir$

Both CurDir and CurDir$ functions return the current system environment path for the specified drive (*Drv*).

The parameter *Drv* must be a string or expression that can represent a valid DOS file structure drive letter. The *Drv* may be local to the computer, or mapped from anywhere on the network connected to the computer. If *Drv* contains more than one letter, only the first character is used.

> **Note:** The file system keeps track of the current drive, and the current directory of every drive. Use the CurDir statement to determine the current directory. The current drive letter can be extracted from the Left character returned in the CurDir statement.

If no *Drv* is specified or if *Drv* is a zero-length string (" "), CurDir functions return the system environment path for the current drive.

### Syntax

**CurDir**(*Drv*)

*Drv*:

A string or expression that can represent a valid DOS file structure drive letter. Drv is case insensitive and must end with a colon (:). The Drv may be local to the computer, or mapped from anywhere on the network connected to the computer. Drv is often included as part of the Path parameter.

### Return Value

CurDir returns a Variant containing a string; CurDir$ returns a String.

### Related Functions

ChDir | ChDrive | Dir | MkDir | RmDir

### Example

```
Dim vntCurPath As Variant
```

```
Dim strCurPath As String
vntCurPath = CurDir()    ' store current path as variant
strCurPath = CurDir$()    ' store current path as string
```

## Dir

Dir function returns a file or directory name that matches the given *File* and *Attrib* arguments.

- The *File* argument is optional, and represents a string expression that specifies a valid file name, and may include a DOS path structure including directory or folder names, and a drive letter. You must specify *File* the first time you call the Dir function, or an error occurs.
- The *Attrib* argument is optional, and can be a constant or numeric expression whose sum specifies file attribute values. If you specify file attributes in the function call, *File* must be included. If the Volume attribute value (8) is specified, all other attribute values are ignored.

Dir supports the use of multiple-character (*) and single-character (?) wildcards to specify multiple files.

Dir returns the first file name that matches both *File* and *Attrib*. To get any additional file names that match, call Dir again with no arguments. When no more file names match, Dir returns a zero-length string (" "). Once a zero-length string is returned, you must specify argument/s in the next call (to reset the function), or an error occurs.

Calling Dir with any argument will reset the function, and it will treat the call as a new call. Previous arguments passed to the Dir function are overwritten and forgotten (reset). You can reset the function (by supplying arguments in the function call) at any time, even if it has not yet returned every possible argument match result.

Calling Dir with the Directory attribute (16) does not continually return Directory names. You will need to check the attribute value of every return result to determine if the return is a valid directory name. To do so, use the GetAttr function. Because file names are retrieved in no particular order, you may want to store returned file names in an array and then sort the array.

**Note:** The file system keeps track of the current drive, and the current directory of every drive. Use the CurDir statement to determine the current directory. The current drive letter can be extracted from the Left character returned in the CurDir statement.

### Syntax

**Dir**(*File*, *Attrib*)

*File*:

A string or expression that can represent a valid file name, and may include a DOS path structure including directory or folder names, and a drive letter.

*Attrib*:

A number or expression that can represent a sum of the attribute values of a file . This can be a constant or a numeric expression which includes any combination of attribute numeric values, whose sum specifies all relevant attributes of a file.

where:

- 0 = Normal
- 1 = Read Only
- 2 = Hidden
- 4 = System
- 8 = Volume
- 16 = Directory or Folder
- 32 = Archive

Possible combinations of values can sum to 0, 1, 2, 3, in fact every number from 0 to 64, each representing a unique combination of attribute values. For example, a file attribute value of 6 represents that the file has both System (4) and Hidden (2) attributes set.

**Return Value**

Returns a String representing the name of a file, directory, or folder that matches a specified pattern or file attribute, or the volume label of a drive. If *File* is not found, a zero-length string (" ") is returned. If *Attrib* is omitted, all files are returned that match *File.*

**Related Functions**

ChDir| ChDrive| CurDir, CurDir$| MkDir| RmDir

**Example**

```
Dim strCurPath As String        ' declare string to store current path
Dim strFileName As String    ' declare string to store retrieved file name
Dim intFileCount As Integer    ' declare integer to keep count of retrieved files
Dim arrFileList() As String         ' declare string array to store file names
strCurPath = CurDir$()                ' store current path for later restoration
ChDir "\"                        ' change to root directory of current drive
strFileName = Dir(*.dat)            ' retrieve file name with .dat extension
Do                             ' initialize loop
    If strFileName = "" Then             ' check to see if valid filename returned
        exit do                    ' exit from loop
    Else
        intFileCount = intFileCount + 1           ' increment file counter variable
        arrFileList(intFileCount) = strFileName    ' store file name in array
Redim
        arrFileList(intFileCount)    ' increase array size to count value
        strFileName = Dir()       ' retrieve next file name to match original Dir
```

```
call
    EndIf
Loop Until strFileName = ""            ' loop again
ChDir strCurPath                    'restore previous current directory
```

## EOF

EOF function returns a Boolean True or False value during file access that indicates whether the current position of an open file has reached the end of the file. The required *FileNum* argument must contain an Integer representing any valid system file number associated with an open file.

**Note:** The file system keeps track of all open files and the current position of access within every file. Every statement or function that accesses the data within a file, alters the current position within that file. The Loc function can be used to determine the current position within an open file.

Use the LOF and Loc functions instead of EOF when reading binary files with the Input function, or use Get when using the EOF function.

**Note:** An error occurs with files opened for Binary access, when the file is read using the Input function until EOF returns True.

### Syntax

**EOF**(*FileNum*)

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

### Return Value

Returns an Integer containing the Boolean value False until the end of the file has been reached. Returns True when the end of a file opened for Random or sequential Input has been reached.

### Related Functions

FileLen | Loc | LOF | Seek

### Example

```
Dim strFileContents as String, strTemp as String
```

```
Open "c:\test.txt" For Input As #1        ' open file
Do While Not EOF(1)                     ' loop until end of file
    strTemp = Input(10, #1)                ' read next ten characters
    strFileContents = strFileContents & strTemp    ' join strings
Loop
Close #1
```

## FileCopy

Copies a file from *Src* to *Dest*.

The required source file (*Src*) and destination file (*Dest*) arguments must be valid string expressions representing valid file names. *Src* is the file name of the file to copy from. *Dest* is the file name to be copied to. Both *Src* and *Dest* arguments may contain a DOS path structure including directory or folder names, and a drive letter.

If the *Dest* file does not exist, it will be created by the FileCopy statement. If the *Dest* file already exists, it will be overwritten.

The FileCopy statement does not work on a currently open file. Both the *Src* and *Dest* files must be closed before using the FileCopy statement. To close an open file, use the Close statement.

> **Note:** The file system keeps track of the current drive, and the current directory of every drive. Use the CurDir statement to determine the current directory. The current drive letter can be extracted from the Left character returned in the CurDir statement.

### Syntax

**FileCopy***Src*, *Dest*

*Src*:

A string or expression that can represent a valid DOS file structure FileName. Src is case insensitive. This may include a relative or static Path including directory or folder structure and drive letter. To specify multiple files, the FileName may consist of multiple-character ( * ) and single-character ( ? ) wildcards in the file name.

*Dest*:

A string or expression that can represent a valid DOS file structure FileName. Dest is case insensitive. This may include a relative or static Path including directory or folder structure and drive letter. To specify multiple files, the FileName may consist of multiple-character ( * ) and single-character ( ? ) wildcards in the file name.

### Related Functions

Close | FreeFile | Kill | Name | Open

### Example

```
Dim SourceFile as String, DestinationFile as String
SourceFile = "SRCFILE.Dat"            ' Define source file name.
DestinationFile = "DESTFILE.Dat"        ' Define target file name.
FileCopy SourceFile, DestinationFile    ' Copy source to target.
```

## FileLen

FileLen function determines the byte length of a file. The required *File* argument must be valid string expression representing a valid file name. *File* may contain a DOS path structure including directory or folder names, and a drive letter.

The FileLen function returns the size of a file immediately before it was most recently opened. To obtain the length of a file that is already open, use the LOF function.

### Syntax

**FileLen**(*File*)

*File*:

A string or expression that can represent a valid file name, and may include a DOS path structure including directory or folder names, and a drive letter.

### Return Value

Returns a Long value representing the length of the file measured in bytes.

### Related Functions

EOF | Loc | LOF | Seek

### Example

```
Dim lonFileSize As Long
lonFileSize = FileLen("C:\TESTFILE.txt")    ' returns length of file
in bytes
```

## FreeFile

Retrieves the next sequential system file number available for association with a file. Use the FreeFile function to retrieve an unassociated file number from the file system. This number can be used by the Open statement to be associated with a file.

### Syntax

**FreeFile**

### Return Value

Returns an Integer reference number ready for being associated with a file.

**Related Functions**

Close | FileCopy | Kill | Name | Open

**Example**

```
Dim intFileNum as Integer
intFileNum = FreeFile    'retrieve next free file number
Open "c:\TEST.txt" For Output As #intFileNum
Write #intFileNum, "This is a sample line of text."
Close #intFileNum
```

## Get #

Get statement reads data from a disk file into a variable.

The required *FileNum* argument is a system reference number associated with an open file. The optional *RecNum* argument is the byte position where the read starts for files opened in Binary mode. If you omit *RecNum*, the next record or byte following the last Get or Put statement (or pointed to by the last Seek function) is read. You must include delimiting commas.

The required *VarName* is the name of the variable where the file data is read (copied) to.

**Random mode**

For files opened in Random mode, the following rules apply:

- If the length of the data being read is less than the length specified in the *Len*clause of the Open statement, Get reads subsequent records on record-length boundaries. The space between the end of one record and the beginning of the next record is padded with the existing contents of the file buffer. Because the amount of padding data can't be determined with any certainty, it is generally a good idea to have the record length match the length of the data being read.
- If the variable being read into is a variable-length string, Get reads a 2-byte descriptor containing the string length and then reads the data that goes into the variable. Therefore, the record length specified by the *Len*clause in the Open statement must be at least 2 bytes greater than the actual length of the string.
- If the variable being read into is a Variant of numeric type, Get reads 2 bytes identifying the VarType of the Variant and then the data that goes into the variable. For example, when reading a Variant of VarType 3, Get reads 6 bytes: 2 bytes identifying the Variant as VarType 3 (Long) and 4 bytes containing the Long data. The record length specified by the *Len*clause in the Open statement must be at least 2 bytes greater than the actual number of bytes required to store the variable.

> **Note:** You can use the Get statement to read a Variant array from disk, but you can't use Get to read a scalar Variant containing an array. You also can't use Get to read objects from disk.

- If the variable being read into is a Variant of VarType 8 (String), Get reads 2 bytes identifying the VarType, 2 bytes indicating the length of the string, and then reads the string data. The record length specified by the *Len*clause in the Open statement must be at least 4 bytes greater than the actual length of the string.
- If the variable being read into is a dynamic array, Get reads a descriptor whose length equals 2 plus 8 times the number of dimensions, that is, 2 + 8 * NumberOfDimensions. The record length specified by the *Len*clause in the Open statement must be greater than or equal to the sum of all the bytes required to read the array data and the array descriptor. For example, the following array declaration requires 118 bytes when the array is written to disk.
- If the variable being read into is a fixed-size array, Get reads only the data. No descriptor is read.
- If the variable being read into is any other type of variable (not a variable-length string or a Variant), Get reads only the variable data. The record length specified by the *Len*clause in the Open statement must be greater than or equal to the length of the data being read.

Get reads elements of user-defined types as if each were being read individually, except that there is no padding between elements. On disk, a dynamic array in a user-defined type (written with Put) is prefixed by a descriptor whose length equals 2 plus 8 times the number of dimensions, that is, 2 + 8 * NumberOfDimensions. The record length specified by the *Len*clause in the Open statement must be greater than or equal to the sum of all the bytes required to read the individual elements, including any arrays and their descriptors.

**Binary mode**

For files opened in Binary mode, all of the Random rules apply, except:

- The *Len*clause in the Open statement has no effect. Get reads all variables from disk contiguously; that is, with no padding between records.
- For any array other than an array in a user-defined type, Get reads only the data. No descriptor is read.

Get reads variable-length strings that aren't elements of user-defined types without expecting the 2-byte length descriptor. The number of bytes read equals the number of characters already in the string.

**Syntax**

**Get** #(*FileNum, RecNum, VarName*)

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

*RecNum*:

The byte position where the read starts for files opened in Binary mode. If you omit RecNum, the next record or byte following the last Get or Put statement (or pointed to by the last Seek function) is read.

*VarName*:

A string representing a valid variable name.

**Related Functions**

GetAttr | Input | Line Input # | Print # | Put # | Write #

**Example**

```
Type Record                    ' Define user-defined type.
ID As Integer
Name As String * 20
End Type

Dim recRecord As Record
Dim intPosition As Integer
Dim intFileNum as Integer
intFileNum = FreeFile    'retrieve next free file number
' Open sample file for random access.
Open "TESTFILE.txt" For Random As #intFileNum
' Read the sample file using the Get statement.
intPosition = 3                ' Define third record number.
Get #intFileNum, intPosition, recRecord    ' Read third record.
Close #intFileNum                ' Close file.
```

## GetAttr

GetAttr function returns an Integer representing the attribute settings of a file, directory, or volume.

The required *File* argument must be valid string expression representing a valid file name. *File* may contain a DOS path structure including directory or folder names, and a drive letter.

To determine which attributes are set, use the **AND** operator to perform a bitwise comparison of the value returned by the GetAttr function and the value of the individual file attribute you want. If the result is not zero, that attribute is set for the named file. For example, the return value of the following **AND** expression is zero if the Archive attribute is not set:

```
Const AttrArchive = 32
Result = GetAttr(FileName) And AttrArchive    ' A nonzero value is
returned if the Archive attribute is set.
```

**Syntax**

**GetAttr**(*File*)

*File*:

A string or expression that can represent a valid file name, and may include a DOS path structure including directory or folder names, and a drive letter.

**Return Value**

Returns an Integer number indicating the sum Attribute value of a file, directory, or folder for the *File*argument, where:

- 0 = Normal
- 1 = Read Only
- 2 = Hidden
- 4 = System
- 8 = Volume
- 16 = Directory or Folder
- 32 = Archive

**Related Functions**

Get # | Line Input # | Print # | Put #

**Example**

```
Dim intAttrVal
' Assume file TESTFILE has hidden attribute set.
intAttrVal = GetAttr("TESTFILE.txt")    ' Returns 2.
' Assume file TESTFILE has hidden and read-only attributes set.
intAttrVal = GetAttr("TESTFILE.txt")     Returns 3.
' Assume MYDIR is a directory or folder.
intAttrVal = GetAttr("MYDIR")    ' Returns 16.
```

## Input

Input # statement reads data from a Sequential file and assigns that data to variables. Input function returns characters from a file opened in Input or Binary mode.

The Input # statement has two parameters *FileNum* and *VarList*. The required *FileNum* argument is the associated file number used in the Open statement when the file was opened. The required *VarList* argument is a comma delimited list of variables that are assigned values read from the file.

The Input function has two parameters: *Num* and *FileNum*. The required *Num* argument is a number or valid numeric expression specifying the number of characters (bytes) to be read from the file. *FileNum* is the associated file number used in the Open statement when the file was opened.

The file system tracks all open files and the current position of access within every file. Every statement or function that accesses the data within a file, alters the current position within that file. The Loc function can be used to determine the current position within an open file.

Use the LOF and Loc functions instead of EOF when reading binary files with the Input function, or use Get when using the EOF function.

An error occurs with files opened for Binary access, when the file is read using the Input function until EOF returns True.

Data read with the Input # statement has usually been written to a file with the Write # statement. Data read with the Input function has usually been written to a file with the Print # or Put statements.

When saving data to a file for future reading with the Input # statement, use the Write # statement instead of the Print # statement to write the data to the file. Using Write # properly delimits each separate data field, so it can be read back in using Input #. Using Write # also formats the data in a manner that will allow correct read operations in most locales.

**Syntax**

**Input** #(*FileNum*, *VarList*)

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

*VarList*:

A predefined valid CitectVBA variable name or comma delimited list of valid variable names.

**Return Value**

Input # statement returns data record by record from a file opened in Input or Binary mode. Data items in a file must appear in the same order as the variables in *VarList* and match variables of the same data type. If a variable is numeric and the data is not numeric, a value of zero is assigned to the variable.

Input function returns a String containing characters from a file opened in Input or Binary mode. The Input function returns all of the characters it reads, including commas, carriage returns, linefeeds, quotation marks, and leading spaces.

**Related Functions**

Get # | GetAttr | Line Input # | Print # | Put # | Write #

**Example**

```
Dim strFileContents As String
Dim strTemp As String
Dim strString As String
Dim intFileNum as Integer
Dim intNumber as Integer
intFileNum = FreeFile    'retrieve next free file number
Open "c:\test.txt" For Input As #intFileNum        ' open file.
Do While Not EOF(intFileNum)        ' loop until end of file
    strTemp = Input(10, #intFileNum)    ' read next ten characters
    strFileContents = strFileContents & strTemp    ' join strings
Loop
Input #intFileNum, strString, intNumber        ' Read data into two variables.
Close #intFileNum
```

## Kill

Kill statement deletes files from disk.

The required *File* argument must be valid string expression representing a valid file name. *File*may contain a DOS path structure including directory or folder names, and a drive letter.

Kill supports the use of multiple-character (*) and single-character (?) wildcards to specify multiple files. The Kill statement does not work on a currently open file. To remove a directory use the RmDir statement.

The file system tracks the current drive and the current directory of every drive. Use the CurDir statement to determine the current directory. The current drive letter can be extracted from the Left character returned in the CurDir statement.

**Syntax**

**Kill***File*

*File*:

A string or expression that can represent a valid file name, and may include a DOS path structure including directory or folder names, and a drive letter.

**Related Functions**

Close | FileCopy | FreeFile | Name | Open

**Example**

```
' Assume TESTFILE is a file containing some data.
Kill "TestFile"

' Delete all Dat files in current directory.
Kill "*.Dat"
```

## Line Input #

Line Input # statement reads a single line from an open sequential file and assigns it to a String variable.

The required *FileNum* argument is a system reference number associated with an open file. The required *VarName* is the name of the variable where the file data is read (copied) to.

**Note:** The number sign (# ) preceding *FileNum* is not optional.

The Line Input # statement reads from a file one character at a time until it encounters a carriage return (Chr(13)) or carriage return-linefeed (Chr(13) + Chr(10)) sequence. Carriage return - linefeed sequences are skipped rather than appended to the character string.

**Note:** The file system keeps track of all open files and the current position of access within every file. Every statement or function that accesses the data within a file, alters the current position within that file. The Loc function can be used to determine the current position within an open file.

Data read with the Line Input # statement has usually been written to a file with the Print # statement.

**Syntax**

**Line Input** # FileNum, *VarName*

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

*VarName:*

A string representing a valid variable name.

**Related Functions**

Get # | GetAttr | Input | Print # | Put # | Write #

**Example**

```
Dim strTextLine As String
Dim intFileNum As Integer
Open "c:\TEST.txt" For Input As #intFileNum    intFileNum = FreeFile
'retrieve next free file number
Do While Not EOF(intFileNum)            ' Loop until end of file.
    Line Input #intFileNum, strTextLine    ' Read line into variable.
    Print TextLine               ' Print line.
Loop
Close #intFileNum
```

## Loc

Loc function returns a number indicating the current position within a file opened using the Open statement.

The required *FileNum* argument must contain an Integer representing any valid number associated with an open file.

**Syntax**

**Loc**(*FileNum*)

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

**Return Value**

Returns a Long representing the current position within a file, the value dependant upon which file access mode the file was opened with:

- If the file was opened in Random mode, the Loc function will return a number representing the last record read from or written to the file.
- If the file was opened in Sequential mode, the Loc function will return a number representing the current byte position in the file divided by 128. (However, information returned by Loc for Sequential files is neither used nor required.)
- If the file was opened in Binary mode, the Loc function will return a number representing the position of the last byte read from or written to the file.

**Related Functions**

EOF | FileLen | LOF | Seek

**Example**

```
Dim lonLoc As Long
Dim strLine As String
Open "TESTFILE.txt" For Binary As #1    ' Open file
Do While lonLoc < LOF(1)            ' Loop until end of file
    strLine = strLine & Input(1, #1)    ' Read character into variable
    lonLoc = Loc(1)             ' Get current position within file
Loop
<statements>                ' Do stuff with retrieved data
Close #1                    ' Close file
```

## LOF

LOF function returns a number indicating the byte length of a sequential file opened using the Open statement.

The required *FileNum* argument must contain an Integer representing any valid number associated with an open file.

> **Note:** The file system keeps track of all open files and the current position of access within every file. Every statement or function that accesses the data within a file, alters the current position within that file. The Loc function can be used to determine the current position within an open file.

The LOF function returns the size of a file that is already open.
To obtain the length of a file that is not open, use the FileLen function.

Use the LOF and Loc functions instead of EOF when reading binary files with the Input function.

**Syntax**

**LOF**(*FileNum*)

*FileNum:*

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

**Return Value**

Returns a Long representing the size of a file in bytes.

**Related Functions**

EOF | FileLen | Loc | Seek

**Example**

```
Dim lonFileSize As Long
lonFileSize = LOF "C:\TESTFILE.txt"    ' returns length of file in bytes
```

## MkDir

The MkDir statement creates the directory specified in the *Path* parameter.

The required parameter *Path* must be a string or expression that can represent a valid DOS file structure path value, must contain a directory name, may contain a relative path structure, and may contain a drive letter. The *Path* parameter must be limited to less than 128 characters.

The MkDir statement is relative to the current directory. If no path structure is provided, the directory is created in the current directory. If no drive is specified, the MkDir statement creates the directory on the current drive.

**Note:** The file system keeps track of the current drive, and the current directory of every drive. Use the CurDir statement to determine the current directory. The current drive letter can be extracted from the Left character returned in the CurDir statement.

**Syntax**

**MkDir***Path*

*Path*:

A string or expression that can represent a valid DOS file structure path value. This includes a directory name, and may include a relative or static directory or folder structure and drive letter, in the order:

```
[<driveletter>:][\<rootdirectoryname>][\<subdirectory> ...
\<subdirectory>\] directoryname
```

The path can be relative to the current directory. A single period represents the current directory (.). Two periods represent the parent directory of the current directory (..). For example:

```
chdir ..        ' changes to the parent directory of the current directory
chdir ..\test           ' changes to the test subdirectory of the parent
directory
```

**Related Functions**

ChDir | ChDrive | CurDir, CurDir$ | Dir | RmDir

**Example**

```
Dim strPath As String
Dim strDir As String
strPath = CurDir()    ' store current path
strDir = "Temp"
ChDir "\"         ' change to root dir on current drive
MkDir strDir        ' create new directory
ChDir strPath     ' change back to previous path
```

## Name

The Name statement renames the disk file specified in the *OldFileName* parameter, to the name specified in the *NewFileName* parameter.

The required parameter *OldFileName* must be valid existing file name, may contain a path structure, and may contain a drive letter.

The *NewFileName* parameter must be a string or expression that can represent a valid DOS file name value, may contain a relative path structure, and may contain a drive letter. The *NewFileName* parameter must be limited to less than 128 characters.

The Name statement uses the file system relative to the current directory. If no path structure is provided, the *NewFileName* file is expected to be in the current directory. If no drive is specified, the Name statement expects the file to be on the current drive.

Using Name, you can move a file from one directory or folder to another. If the path in *NewFileName* exists and is different from the path in *OldFileName*, the Name statement moves the file to the new directory or folder and renames the file, if necessary. If *NewFileName* and *OldFileName* have different paths and the same file name, Name moves the file to the new location and leaves the file name unchanged.

Name does not support the use of multiple-character ( * ) and single-character (?) wildcards to specify multiple files.

The Name statement does not work on a currently open file. You must close an open file before renaming it.

**Note:** The file system keeps track of the current drive, and the current directory of every drive. Use the CurDir statement to determine the current directory. The current drive letter can be extracted from the Left character returned in the CurDir statement.

### Syntax

**Name***OldFileNameNewFileName*

*OldFileName*:

A string or expression that can represent a valid file name, and may include a DOS path structure including directory or folder names, and a drive letter.

*NewFileName*:

A string or expression that can represent a valid file name, and may include a DOS path structure including directory or folder names, and a drive letter.

**Related Functions**

Close | FileCopy | FreeFile | Kill | Open

**Example**

```
Dim strNewFileName As String
Dim strOldFileName As String
strOldFileName = "c:\temp\oldfile.txt"
strNewFileName = "newfile.txt"
ChDir "\"        ' change to root dir on current drive
Name strOldFileName strNewFileName    ' moves file to root dir and renames it
ChDir strPath    ' change back to previous path
```

## Open

Open statement enables input/output (I/O) to a disk file.

The required *File* argument must be a valid string expression representing a valid file name. *File* may contain a DOS path structure including directory or folder names, and a drive letter.

The required *Mode* argument must be a valid keyword specifying the file I/O mode: Append, Binary, Input, Output, or Random. If unspecified, the file is opened for Random access.

The optional *Access* argument must be a valid keyword specifying the operations permitted on the open file: Read, Write, or Read Write.

The optional *Lock* argument must be a valid keyword specifying the operations permitted on the open file by other processes: Shared, Lock Read, Lock Write, and Lock Read Write.

The required *FileNum* argument must contain an Integer representing the number that will be associated with the file. This is the file system reference number supplied by the FreeFile statement that can be used in functions such as Get #, Input #, Line Input #, Print #, and Write #. In Binary, Input, and Random modes, you can open a file using a different file number without first closing the file. In Append and Output modes, you must close a file before opening it with a different file number.

**Note:** The file system tracks all open files and the current position of access within every file. Every statement or function that accesses the data within a file, alters the

current position within that file. The Loc function can be used to determine the current position within an open file.

The optional *RecLen* argument must be a number less than or equal to 32,767 (bytes). For files opened for Random access, this value is the record length. For sequential files, this value is the number of characters buffered. The Len clause is ignored if mode is Binary.

You must open a file before any I/O operation can be performed on it. Open allocates a buffer for I/O to the file and determines the mode of access to use with the buffer. If the file is already opened by another process and the specified type of access is not allowed, the Open operation will not succeed and an error message will be generated.

If the file specified by pathname doesn't exist, it is created when a file is opened for Append, Binary, Output, or Random modes.

**Syntax**

**Open**(*File***For** *Mode***Access** *Access Lock* **As** *#FileNum* **Len=***RecLen*)

*File*:

A string or expression that can represent a valid file name, and may include a DOS path structure including directory or folder names, and a drive letter.

*Mode*:

A CitectVBA keyword specifying the file I/O mode: Append, Binary, Input, Output, or Random.

*Lock*:

A CitectVBA keyword specifying the operations permitted on the open file by other processes: Shared, Lock Read, Lock Write, and Lock Read Write.

*Access*:

A CitectVBA keyword specifying the operations permitted on the open file: Read, Write, or Read Write.

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

*RecLen*:

An Integer or numeric expression representing the byte length of a file record as a number less than or equal to 32,767.

**Related Functions**

Close | FileCopy | FreeFile | Kill | Name

**Example**

```
' The following code opens the file TESTFILE in sequential-input
mode.
Open "TESTFILE" For Input As #1
' Close before reopening in another mode.
Close #1

' This example opens the file in Binary mode for writing
operations only.
Open "TESTFILE" For Binary Access Write As #1
' Close before reopening in another mode.
Close #1

' The following example opens the file in Random mode. The file
contains records of the user-defined type Record.
Type Record                ' Define user-defined type.
ID As Integer
Name As String * 20
End Type
Dim recRecord As Record        ' Declare variable.
Open "TESTFILE" For Random As #1
' Close before reopening in another mode.
Close #1

' This code example opens the file for sequential output; any
process can read or write to file.
Open "TESTFILE" For Output Shared As #1
' Close before reopening in another mode.
Close #1

' This code example opens the file in Binary mode for reading;
other processes can't read file.
Open "TESTFILE" For Binary Access Read Lock Read As #1
' Close before reopening in another mode.
Close #1
```

## Print (function)

Displays a message in the runtime Citect Kernel, and the Cicode Editor output window if you are in debug mode.

> **Note:** Do not confuse this function with the Print # statement, which prints data to disk.

**Related Functions**

TraceMsg (Cicode function)

## Print #

Print # statement reads data from *OutputList* and writes that data to a sequential file.

The Print # statement has two parameters *FileNum* and *OutputList*. The required *FileNum* argument is the associated file number used in the Open statement when the file was opened. The required *OutputList* argument is a delimited list of expressions whose values are written to the file.

**Note:** The number sign hash character ( # ) preceding *FileNum*is not optional. This character indicates disk file access with the file referenced by the system file number that follows it. Do not confuse Print # which prints to disk, with Print which displays data on the screen.

Data written with Print # is usually read from a file with Line Input # or Input.

**Note:** If you want to read the data from a file using the Input # statement, use the Write # statement instead of the Print # statement to write the data to the file. Using Write #properly delimits each separate data field, so it can be read back in using Input #. Using Write # also formats the data in a manner that will allow correct read operations in most locales.

If you omit expressionlist, the Print # statement prints a blank line in the file, but you must include the comma. Because Print # writes an image of the data to the file, you must delimit the data so it is printed correctly. If you use commas as delimiters, Print # also writes the blanks between print fields to the file.

The Print # statement usually writes Variant data to a file the same way it writes any other data type. However, there are some exceptions:

If the data being written is a Variant of VarType 0 (Empty), Print # writes nothing to the file for that data item.

If the data being written is a Variant of VarType 1 (Null), Print # writes the literal #NULL# to the file.

If the data being written is a Variant of VarType 7 (Date), Print # writes the date to the file using the Short Date format defined in the WIN.INI file. When either the date or the time component is missing or zero, Print # writes only the part provided to the file.

**Syntax**

**Print** #FileNum, *OutputList*

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

*OutputList*:

One or more formatted numeric and/or string expressions to be written to the file using the following syntax:

```
[ {Spc( s ) | Tab [( n ) ] } ] [expression] [charpos]
```

where:

- [ ] square brackets are used for illustrative purposes to indicate in the code that the arguments they enclose are optionally used in the *OutputList*. Do not use the square brackets themselves in your code.
- { } curly braces are required to encompass and delineate the arguments they enclose, and to separate their contents from the other arguments in the *OutputList*.
- ( | ) vertical line are used for illustrative purposes to indicate in the code that either side of the line is an alternative argument. You can use the argument provided on one of the line or the other, but not both arguments at the same time within the same set of curly braces. Do not include the vertical line in your code.
- {*Spc*(s)} argument is optionally used to insert 's' number of space characters in the output file at the position of the argument in the *OutputList*. The *Spc* argument must be enclosed by curly braces to delineate it from an *expression*. The *Spc* argument can be repeated any number of times to insert spaces in the file between *expressions*. The *Spc* argument is mutually exclusive with the *Tab* argument when used within the same set of curly braces.
- {*Tab(n)*} argument is optionally used to position the insertion point to an absolute column number in the output file at the position of the argument in the *OutputList*, where 'n' is the column number. Use *Tab*with no argument to position the insertion point at the beginning of the next print zone. The *Tab* argument must be enclosed by curly braces to delineate it from an *expression*. The *Tab* argument can be repeated any number of times to insert tabs in the file between *expressions*. The *Tab* argument is mutually exclusive with the *Spc* argument when used within the same set of curly braces.
- *expression* argument represents a valid numeric or string expression to output to the file. The *expression* argument can be repeated any number of times.
- *charpos* is the character that determines the position of the next character in the output. A semicolon means the next character is printed immediately after the last character; a comma means the next character is printed at the start of the next print zone. Print zones begin every 14 columns. If neither character is specified, the next character is printed on the next line.

**Return Value**

Input # statement returns data record by record from a file opened in Input or Binary mode. Data items in a file must appear in the same order as the variables in *VarList* and match variables of the same data type. If a variable is numeric and the data is not numeric, a value of zero is assigned to the variable.

**Related Functions**

Get # | GetAttr | Input | Line Input # | Put # | Write #

**Example**

The following example writes data to a test file.

```
Dim I, FNum, FName    ' Declare variables.
For I = 1 To 3
FNum = FreeFile    ' Determine next file number.
FName = "TEST" & FNum
Open FName For Output As FNum    ' Open file.
Print #1, "This is test #" & I    ' Write string to file.
Print #1, "Here is another "; "line"; I
Next I
Close    ' Close all files.
The following example writes data to a test file and reads it
back.
Dim FileData, Msg, NL    ' Declare variables.
NL = Chr(10)    ' Define newline.
Open "TESTFILE" For Output As #1    ' Open to write file.
Print #1, "This is a test of the Print # statement."
Print #1,    ' Print blank line to file.
Print #1, "Zone 1", "Zone 2"    ' Print in two print zones.
Print #1, "With no space between" ; "."     ' Print two strings
together.
Close #1

Open "TESTFILE" for Input As #2    ' Open to read file.
Do While Not EOF(2)
    Line Input #2, FileData    ' Read a line of data.
    Msg = Msg & FileData & NL    ' Construct message.
    MsgBox Msg
Loop
Close #2    ' Close all open files.
Kill "TESTFILE"    ' Remove file from disk.
```

## Put #

Put # statement writes data from a variable to a disk file.

The required *FileNum* argument is a system reference number associated with an open file.

> **Note:** The number sign ( # ) preceding *FileNum* is not optional.

The optional *RecNum* argument is the byte position where the read starts for files opened in Binary mode. The first record or byte in a file is at position 1, the second record or byte is at position 2, and so on. If you omit *RecNum*, the next record or byte following the last Get or Put statement (or pointed to by the last Seek function) is read. You must include delimiting commas.

> **Note:** The file system keeps track of all open files and the current position of access within every file. Every statement or function that accesses the data within a file, alters the current position within that file. The Loc function can be used to determine the current position within an open file.

The required *VarName* is the name of the variable where the file data is read (copied) from.

Data written with the Put # statement is usually read from a file with the Get # statement.

**Random mode**

For files opened in Random mode, the following rules apply:

- If the length of the data being written is less than the length specified in the Len clause of the Open statement, Put writes subsequent records on record-length boundaries. The space between the end of one record and the beginning of the next record is padded with the existing contents of the file buffer. Because the amount of padding data can't be determined with any certainty, it is generally a good idea to have the record length match the length of the data being written. If the length of the data being written is greater than the length specified in the Len clause of the Open statement, an error occurs.
- If the variable being written is a variable-length string, Put writes a 2-byte descriptor containing the string length and then the variable. The record length specified by the Len clause in the Open statement must be at least 2 bytes greater than the actual length of the string.
- If the variable being written is a Variant of a numeric type, Put writes 2 bytes identifying the VarType of the Variant and then writes the variable. For example, when writing a Variant of VarType 3, Put writes 6 bytes: 2 bytes identifying the Variant as VarType 3 (Long) and 4 bytes containing the Long data. The record length specified by the Len clause in the Open statement must be at least 2 bytes greater than the actual number of bytes required to store the variable.

  > **Note:** You can use the Put statement to write a Variant array to disk, but you can't use Put to write a scalar Variant containing an array to disk. You also can't use Put to write objects to disk.

- If the variable being written is a Variant of VarType 8 (String), Put writes 2 bytes identifying the VarType, 2 bytes indicating the length of the string, and then writes the string data. The record length specified by the Len clause in the Open statement must be at least 4 bytes greater than the actual length of the string.
- If the variable being written is a dynamic array, Put writes a descriptor whose length equals 2 plus 8 times the number of dimensions, that is, 2 + 8 * NumberOfDimensions. The record length specified by the Len clause in the Open statement must be greater than or equal to the sum of all the bytes required to write the array data and the array descriptor. For example, the following array declaration requires 118 bytes when the array is written to disk.

```
Dim MyArray(1 To 5,1 To 10) As Integer
```

The 118 bytes are distributed as follows: 18 bytes for the descriptor (2 + 8 * 2), and 100 bytes for the data (5 * 10 * 2).

- If the variable being written is a fixed-size array, Put writes only the data. No descriptor is written to disk.
- If the variable being written is any other type of variable (not a variable-length string or a Variant), Put writes only the variable data. The record length specified by the Len clause in the Open statement must be greater than or equal to the length of the data being written.

Put writes elements of user-defined types as if each were written individually, except there is no padding between elements. On disk, a dynamic array in a user-defined type written with Put is prefixed by a descriptor whose length equals 2 plus 8 times the number of dimensions, that is, 2 + 8 * NumberOfDimensions. The record length specified by the Len clause in the Open statement must be greater than or equal to the sum of all the bytes required to write the individual elements, including any arrays and their descriptors.

**Binary mode**

For files opened in Binary mode, all of the Random rules apply, except:

- The Len clause in the Open statement has no effect. Put writes all variables to disk contiguously; that is, with no padding between records.
- For any array other than an array in a user-defined type, Put writes only the data. No descriptor is written.
- Put writes variable-length strings that aren't elements of user-defined types without the 2-byte length descriptor. The number of bytes written equals the number of characters in the string. For example, the following statements write 10 bytes to file number 1:

```
VarString$ = String$(10," ")
```

Put writes variable-length strings that are not elements of user-defined types without the 2-byte length descriptor.

```
Put #1,,VarString$
```

**Syntax**

**Put #** FileNum, *RecNum*, *VarName*

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

*RecNum:*

The byte position where the read starts for files opened in Binary mode. The first record or byte in a file is at position 1, the second record or byte is at position 2, and so on. If you omit RecNum, the next record or byte following the last Get or Put statement (or pointed to by the last Seek function) is read.

*VarName*:

A string representing a valid variable name.

**Related Functions**

Get # | GetAttr | Input | Line Input # | Put # | Write #

**Example**

```
' This example uses the Put statement to write data to a file.
' Five records of the user-defined type Record are written to the file.
Type Record        ' Define user-defined type.
ID As Integer
Name As String * 20
End Type

Dim MyRecord As Record, RecordNumber    ' Declare variables.
' Open file for random access.
Open "TESTFILE" For Random As #1 Len = Len(MyRecord)
For RecordNumber = 1 To 5    ' Loop 5 times.
    MyRecord.ID = RecordNumber    ' Define ID.
    MyRecord.Name = "My Name" & RecordNumber  ' Create a string.
    Put #1, RecordNumber, MyRecord    ' Write record to file.
Next RecordNumber
Close #1        ' Close file.
```

## RmDir

The RmDir statement deletes the directory specified in the *Path* parameter.

The required parameter *Path* must be a string or expression that can represent a valid DOS file structure path value, must contain a directory name, may contain a relative path structure, and may contain a drive letter. The *Path* parameter must be limited to less than 128 characters.

The RmDir statement is relative to the current directory. If no path structure is provided, the directory is expected to be a subdirectory of the current directory. If no drive is specified, the RmDir statement deletes the directory on the current drive.

The current directory cannot be deleted. To change the current directory to another directory, use the ChDir statement. The directory to be deleted must be empty and contain no files or sub-directories. To delete files in a directory, use the Kill statement.

**Note:** The file system keeps track of the current drive, and the current directory of every drive. Use the CurDir statement to determine the current directory. The current drive letter can be extracted from the Left character returned in the CurDir statement.

**Syntax**

**RmDir***Path*

*Path*:

A string or expression that can represent a valid DOS file structure path value. This includes a directory name, and may include a relative or static directory or folder structure and drive letter, in the order:

```
[<driveletter>:][\<rootdirectoryname>][\<subdirectory> ...
\<subdirectory>\] directoryname
```

**Note:** The path can be relative to the current directory. A single period represents the current directory (.). Two periods represent the parent directory of the current directory (..). For example, chdir .. changes to the parent directory of the current directory. chdir ..\test changes to the test subdirectory of the parent directory

**Related Functions**

ChDir | ChDrive | CurDir, CurDir$ | Dir | MkDir

**Example**

```
Dim strDir As String
strDir = CurDir    ' retrieve current directory name
Kill "*.*"    ' delete all files from current directory
ChDir "\"    ' change to root dir on current drive
RmDir strDir    ' delete directory
```

## Seek

Sets the current position within a file opened using the Open statement, ready for the next read or write action.

The required *FileNum* argument must contain an Integer representing any valid system file number associated with an open file.

The required *Position* argument must contain an Integer or expression representing a valid number.

> **Note:** The file system keeps track of all open files and the current position of access within every file. Every statement or function that accesses the data within a file, alters the current position within that file. The Loc function can be used to determine the current position within an open file.

### Syntax

**Seek***FileNum, Position*

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

*Position*:

An Integer or expression representing a valid numeric value.

### Related Functions

EOF | FileLen | Loc | LOF

### Example

```
Open "TESTFILE" For Input As #1    ' Open file for reading.
For i = 1 To 24 Step 3    ' Loop until end of file.
    Seek #1, i    ' Seek to byte position
    MyChar = Input(1, #1)    ' Read next character of data.
    Print MyChar    'Print character of data
Next i
Close #1        ' Close file.
```

## Write #

Write # statement writes data to a Sequential file opened in output or append mode and reads that data from a list of variables.

The Write # statement has two parameters *FileNum* and *VarList*. The required *FileNum* argument is the associated file number used in the Open statement when the file was opened. The required *VarList* argument is a comma delimited list of variables that are assigned values read from the file.

> **Note:** The file system keeps track of all open files and the current position of access within every file. Every statement or function that accesses the data within a file, alters the current position within that file. The Loc function can be used to determine the current position within an open file.

Data written to a file with the Write # statement is usually read with the Input # statement.

> **Note:** When saving data to a file for future reading with the Input # statement, use the Write # statement instead of the Print # statement to write the data to the file. Using Write # properly delimits each separate data field , so it can be read back in using Input #. Using Write # also formats the data in a manner that will allow correct read operations in most locales.

**Syntax**

**Write** *#FileNum, VarList*

*FileNum:*

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

*VarList*:

A predefined valid CitectVBA variable name or comma delimited list of valid variable names.

**Related Functions**

Get # | GetAttr | Input | Line Input # | Print # | Put #

**Example**

```
Dim strFileContents As String
Dim strTemp As String
Dim strString As String
Dim intFileNum as Integer
Dim intNumber as Integer
intFileNum = FreeFile    'retrieve next free file number
Open "c:\test.txt" For Output As #intFileNum        ' open file.
Write #intFileNum, "This is a test of the Write # statement."
Close #intFileNum
```

# Math/Trigonometry Functions

CitectVBA math functions are provided to assist with number manipulation and calculation in your formulas. Mathematical functions can be used in CitectVBA statements, and will (like all other functions), return a value to the caller.

## Numeric functions

Vijeo Citect uses the following predefined numeric functions:

| | |
|---|---|
| Abs | returns the absolute value of a number (Num ). |
| Exp | returns base log (e) to the power of (Num ). |
| Fix | returns the Integer value of a number (Num ). |
| Int | returns the Integer value of a number (Num ). |
| Log | returns the natural log of a number (Num ). |
| Rnd | returns a random value influenced by (Num ). |
| Sgn | returns a value indicating the Sign of (Num ). |
| Sqrt | returns the square root value of a number (Num ). |

## Abs

Calculates the absolute (positive) value of a number. The absolute value of a number is the number without its sign. Abs does not round the number, and ignores the fractional value of the number.

**Syntax**

**Abs**(*Num*)

*Num:*

An integer or expression representing a valid numeric value.

**Return Value**

Returns the absolute value of the number (*Num*) provided in the argument.

The data type of the return value is the same as that of the number argument. However, if the number argument is a Variant of VarType (String) and can be converted to a number, the return value will be a Variant of VarType (Double). If the numeric expression results in a null, Abs returns a null.

**Related Functions**

Sgn

**Example**

```
Variable=Abs(-67); ! Sets Variable to 67.
Variable=Abs(67); ! Sets Variable to 67.
```

## Exp

Calculates the exponential of a number. The exponential is the base of the natural logarithm *e* raised to a power (*e*^*Num*). The Exp function complements the Log function and is sometimes referred to as the antilogarithm.

**Note:** The value of the constant *e* is approximately 2.71828.

**Syntax**

**Exp**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

Returns the value equivalent to the base of the natural logarithm (*e*) raised to the power of the number (*Num*) provided in the argument.

**Related Functions**

Log

**Example**

```
Variable=Exp(1);          ! Sets Variable to 2.7182...
```

## Fix

Calculates the integer portion of a number. Fix does not round the number, and ignores the fractional value of the number.

Fix expects the argument (*Num*) to be a valid numeric value. If the argument value is positive, rounds the *Num* down by dropping any fractional value. If the argument value is negative, rounds the *Num* up to the next integer number greater than or equal to *Num*.

Do not confuse Fix with Int , which rounds a negative argument value (*Num*) down to the next integer number less than or equal to *Num*.

**Syntax**

**Fix**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

Returns the Integer value of the number (*Num*) provided in the argument.

**Related Functions**

Abs | Int | Sgn | Sqrt

**Example**

```
Dim vntVar
vntVar = Fix(99.2)    ' returns 99
vntVar = Fix(99.8)    ' returns 99
vntVar = Fix(-99.8)    ' returns -99
vntVar = Fix(-99.2)    ' returns -99
```

## Int

Calculates the integer portion of a number. Int does not round the number, and ignores the fractional value of the number.

Int expects the argument (*Num*) to be a valid numeric value. If the argument value is positive, rounds the *Num* down by dropping any fractional value. If the argument value is negative, rounds the *Num* down to the next integer number less than or equal to *Num*.

Do not confuse Int with Fix, which rounds a negative argument value (*Num*) up to the next integer number greater than or equal to *Num*.

**Syntax**

**Int**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

Returns the integer value of the number (*Num*) provided in the argument. If *Num* contains a Null, Int returns a Null.

**Related Functions**

Abs | Fix | Rnd | Sgn | Sqrt

**Example**

```
Dim vntVar
vntVar = Int(99.2)     ' returns 99
vntVar = Int(99.8)     ' returns 99
vntVar = Int(-99.8)     ' returns -100
vntVar = Int(-99.2)     ' returns -100
```

## Log

Calculates the natural logarithm of a number

Log expects the argument (*Num*) to be a valid numeric value. The argument value must be greater than zero.

The natural logarithm is the logarithm to the base *e*. You can calculate the base-*n* logarithms for any number *X* by dividing the natural logarithm of *X* by the natural logarithm of *n* as follows:

***Logn (X ) = Log(X ) / Log(n )***

> **Note:** The value of the constant e is approximately 2.71828.

**Syntax**

**Log**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

Returns the natural log of the number (*Num*) provided in the argument.

**Related Functions**

Exp

**Example**

```
Variable=Log(100);    ! Sets Variable to 2 (i.e. 100=10 to the power
```

```
of 2).
```

## Rnd

Generates a decimal fraction number using the optional argument value (*Num*) to determine the sequence of the (random) number generation.

Rnd expects the argument (*Num*) if supplied, to be a valid numeric value.
If *Num* is less than zero, Rnd generates the same number every time, using *Num* as the seed. If *Num* is equal than zero, Rnd repeats the most recently generated number. If *Num* is greater than zero, Rnd generates the next random number in the sequence. If *Num* is not supplied, Rnd generates the next random number in the sequence.

Before calling Rnd, use the Randomize statement without an argument to initialise the random-number generator with a seed based on the system timer.

**Note:** The square brackets **[ ]**in the syntax indicate that the argument is optional. Do NOT include the square brackets in your code.

### Syntax

**Rnd**[(*Num*)]

*Num*:

An Integer or expression representing a valid numeric value.

### Return Value

Returns a (random) decimal fraction number influenced by the (*Num*) provided in the argument. The return value lies in the range of less than 1 but greater than or equal to 0.

### Related Functions

Randomize

### Example

```
Dim vntRndValue
Randomize        ' Initialize random-number generator.
vntRndValue = Int((6 * Rnd) + 1)     ' returns a value between 1 and 6
```

## Sgn

Indicates the sign of a number. Sgn does not round the number, and ignores the fractional value of the number.

Sgn expects the argument (*Num*) to be a valid numeric value. If *Num* is greater than zero, Sgn returns the value of 1. If *Num* is equal to zero, Sgn returns the value of 0. If *Num* is less than zero, Sgn returns the value of -1.

**Syntax**

**Sgn**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

Returns a value indicating the Sign (+ or - ) value of the (*Num*) provided in the argument.

**Related Functions**

Abs | Fix | Int | Sqrt

**Example**

```
Dim vntVal
vntVal = Sgn(99.8)        ' returns 1
vntVal = Sgn(-99.8)        ' returns -1
vntVal = Sgn(0)        ' returns 0
```

## Sqrt

Calculates the square root of a number. Sqrt expects the argument (*Num*) to be a valid numeric value greater than or equal to 0.

**Syntax**

**Sqrt**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

Returns the square root value of the (*Num*) provided in the argument.

**Related Functions**

Abs | Fix | Int | Sgn

**Example**

```
Variable=Sqrt(4);
! Sets Variable to 2.
```

## Trigonometric functions

Vijeo Citect uses the following trigonometric functions:

| | |
|---|---|
| Atn | returns the Arctangent value of a number (Num ). |
| Cos | returns the Cosine value of angle (Rad ). |
| Sin | returns the Sine value of angle (Rad ). |
| Tan | returns the Tangent value of angle (Rad ). |

Trigonometry uses angles and ratios, axes, degrees, Pi, radians and angular conversions. CitectVBA supports the use of Decimal numbers by default, as well as Hexadecimal and Octal numbers. See Numbers.

When using numbers in CitectVBA, you must consider the data type of the variables that hold and store the numbers, as well as the behaviour of CitectVBA when dealing with numbers. See Numeric Data Types.

## Atn

Calculates the trigonometric Arctangent value of a Tangent number.

The Atn function expects the argument (*Num*) to be a valid tangent value between the range of - Pi/2 to + Pi/2 (representing the ratio of the two sides of a right-angle triangle), and calculates the corresponding angle in radians.

Atn is the inverse trigonometric function of Tan (which takes an angle as its argument, and returns the ratio of two sides of a right-angle triangle). Do not confuse Atn with the Cotangent, which is the inverse of a Tangent (1/tangent).

### Syntax

**Atn**(*Num*)

*Num*:

An integer or expression representing a valid numeric value.

### Return Value

Returns the Arctangent value of the angle (*Num*) provided in the argument.

**Related Functions**

Cos | Sin | Tan

**Example**

```
Dim Msg, Pi' Declare variables.
Pi = 4 * Atn(1)' Calculate Pi
```

## Cos

Calculates the trigonometric Cosine value of an angle.

The Cos function expects the argument (*Rad*) to be a valid angle value in radians, and calculates the ratio of the two sides of a right-angle triangle on either side of the angle. The ratio is the length of the side adjacent to the angle divided by the length of the hypotenuse.

**Note:** To convert degrees to radians, multiply degrees by Pi/180. To convert radians to degrees, multiply radians by 180/Pi.

**Syntax**

**Cos**(*Rad*)

*Rad*:

An angle expressed in radians. It must be a valid numeric value.

**Return Value**

Returns the Cosine value of the angle (*Rad*) provided in the argument.
The result lies in the range - 1 to +1.

Cos will return a double.

**Related Functions**

Atn | Sin | Tan

**Example**

```
Variable=Cos(0.7854);    ! Sets Variable to 0.7071...
```

## Sin

Calculates the trigonometric Sine value of an angle. The Sin function expects the argument (*Rad*) to be a valid angle value in radians, and calculates the ratio of two sides of a right-angle triangle. The ratio is the length of the side opposite to the angle divided by the length of the hypotenuse.

To convert degrees to radians, multiply degrees by Pi/180 . To convert radians to degrees, multiply radians by 180/Pi. For more information, see Circle Maths.

**Syntax**

**Sin**(*Rad*)

*Rad*:

An angle expressed in radians. Must be a valid numeric value.

**Return Value**

Returns the Sine value of the angle (*Rad*) provided in the argument. The result lies in the range - 1 to + 1.

**Related Functions**

Atn | Cos | Tan

**Example**

```
Variable=Sin(0.7854);        ! Sets Variable to 0.7071
```

## Tan

Calculates the trigonometric Tangent value of an angle. The Tan function expects the argument (*Rad*) to be a valid angle value in radians, and calculates the ratio of two sides of a right-angle triangle. The ratio is the length of the side opposite to the angle divided by the length of the side adjacent to the angle.

**Note:** To convert degrees to radians, multiply degrees by Pi/180. To convert radians to degrees, multiply radians by 180/Pi.

**Syntax**

**Tan**(*Rad*)

*Rad*:

An angle expressed in radians. Must be a valid numeric value.

**Return Value**

Returns the Tangent value of the angle (*Rad*) provided in the argument. Tan will return as a double.

**Example**

```
Variable=Tan(1);        ! Sets Variable to 1.5574...
```

## Miscellaneous Functions

The miscellaneous functions predefined in CitectVBA are:

| | |
|---|---|
| Beep statement | Sounds a tone through the computer's speaker. |
| Randomize statement | Initializes the random number generator. |
| Rem statement | Used to include explanatory remarks in a program. |
| SendKeys statement | Sends keystrokes to the active window as if entered at the keyboard. |

### Beep

The Beep statement sounds a tone through the computer's speaker. The frequency and duration of the beep depends on the computer's hardware.

**Syntax**

**Beep**

**Related Functions**

SendKeys

**Example**

```
If (TestTag_1 <1) OR (TestTag_1 > 100) Then
    Beep
Else
    Startup_AN38.Value = TestTag_1
End If
```

### Randomize

The Randomize statement initialises the random number generator.

It has one optional parameter number. This parameter can be any valid number and is used to initialise the random number generator. If you omit the parameter then the value returned by the Timer event is used as the default parameter to seed the random number generator.

**Syntax**

**Randomize**[*number*]

**Related Functions**

Timer

**Example**

```
Dim MValue
' Initialise random-number generator
Randomize
MValue = Int((6 * Rnd) + 1)
Print MValue
```

## Rem

Used to include explanatory comments in a program.

**Syntax**

**Rem** *Comment*

*Comment*:

The text of any comment you want to include in the code.

**Example**

```
' This is another way to comment
Rem This is a remark
```

## SendKeys

Sends one or more keystrokes to the active window of the active application as if they had been entered at the keyboard.

The value of the Wait argument determines when the SendKeys function completes and returns control to CitectVBA. If omitted, Wait is treated as FALSE by default.

**Note:**You can't use SendKeys to send keystrokes to an application that is not

designed to run in Microsoft Windows. Sendkeys also can't send the PRINT SCREEN key {PRTSC} to any application..

**Syntax**

**SendKeys**(*keys*, *wait*)

*keys*:

The string that is sent to the active window.

*wait*:

Enter TRUE or FALSE.

If wait is true the keystrokes must be processed before control is returned to the calling procedure. This argument is optional. If you omit it, it is assumed to be false.

**Return Value**

None

**Example**

```
Dim intCounter As Integer        ' Declare variables.
Dim dblProgID As Double,         ' Launch Windows Calculator program.
dblProgID = Shell("Calc.exe", 1)     ' Set up counting loop.
For intCounter = 1 To 5            ' Send keystrokes to Calculator
    SendKeys intCounter & "{+}", True    ' to add the value of intCounter each time
Next intCounter                 ' Return focus to Calculator.
AppActivate "Calculator"            ' Send keystrokes toClose Calculator.

SendKeys "%{F4}", True
```

## Procedural Statements

CitectVBA procedural function statements are provided to assist with conditional code execution and program flow:

| | |
|---|---|
| Call statement | Transfers control to a Sub procedure, function procedure, or dynamic-link library (DLL) procedure. |
| Function statement | Declares and defines a procedure that can receive arguments and return a value of a specified data type. |
| End Function statement | Ends a program or a block of statements within a function. |

| | |
|---|---|
| Sub statement | Declares and defines a Sub procedures name, parameters and code. |
| End Sub statement | Ends a program or a block of statements within a subroutine. |
| CicodeCallOpen function | Calls a Cicode function from CitectVBA. |
| CicodeCallReturn function | Obtains the return value of the most recently completed Cicode function opened with the CitectVBA CicodeCallOpen function. |

Cicode functions used to handle CitectVBA functions and statements:

| | |
|---|---|
| VbCallOpen function | Opens a CitectVBA function or subroutine from Cicode. |
| VbCallRun function | Runs the opened CitectVBA function or subroutine from Cicode. |
| VbCallReturn function | Obtains the return value of the completed CitectVBA function previously opened with the Cicode VbCallOpen function. |

## Call

The Call Statement transfers control to a Sub procedure, Function procedure, or dynamic-link library (DLL) procedure.

The required *ProcedureName* is the name of the function or subroutine to call. The optional *Parameters* is the list of arguments to pass to the called function or subroutine.

You are not required to use the Call statement when calling an CitectVBA subroutine or a DLL function. Parentheses must be used in the argument list if the Call statement is being used.

### Syntax

**Call ProcedureName**[*Parameter(s)*]

### Related Functions

End Function | Sub | End Sub | Exit

### Example

```
Call Beep
```

## CicodeCallOpen

The CicodeCallOpen function is used to call a Cicode function from CitectVBA. It is used to initiate and execute a call to the Cicode function and returns an integer value representing either an error code or the success of this CitectVBA function making the call.

**Note:** This CitectVBA function does not return the actual return-value of the Cicode function being called. You can obtain that return value by using the associated CicodeCallReturn function.

**⚠ WARNING**

**UNINTENDED EQUIPMENT OPERATION**

Do not nest the CicodeCallOpen and CicodeCallReturn functions. Nesting these functions can lead to unintended equipment operation when your program is run.

**Failure to follow these instructions can result in death, serious injury, or equipment damage.**

For details, see Calling Cicode from CitectVBA.

### Syntax

*ReturnValue* = **CicodeCallOpen**(*FunctName, ArgList*)

*ReturnValue*:

The return value for the function in the range of 0 to 3.

*FunctName*:

The name of the Cicode function being called.

*Arglist*:

A variable length comma separated argument list of all the arguments to be passed to the Cicode function being opened (dependant upon which Cicode function is being called and the arguments that Cicode function requires). The argument list should not be enclosed within brackets, although when using variable names as arguments, those variable arguments within the list need to be individually enclosed within brackets to force the passing of the variable to Cicode by value.

### Return Value

CicodeCallOpen returns a integer data type containing a value in the range of 0 to 3:

- 0 if CicodeCallOpenfunction was successful
- 1 for CicodeCallOpenfunction general error
- 2 for specified Cicode function not found

- 3 for incorrect number of arguments for specified Cicode function passed in <ArgList>.

**Related Functions**

CicodeCallReturn

**Example**

In the following example, a CitectVBA variable is enclosed in brackets to force the passing of the variable by value. See Passing variables Byref and Byval.

```
Dim vntRet as Variant
' declare modular variant variable to store function results
Function TestCicode() As Integer
' declare local variables
Dim intRet As Integer
Dim strReply as String
Dim intMaxScale as Integer
' copy current tag value to variable
' uses the project variable tag named MAX_SCALE
intMaxScale = MAX_SCALE
' call Cicode function
' for example: TrnSetScale( AN, Pen, Percent, Scale)
intRet = CicodeCallOpen( "TrnSetScale", 53, -1, 100, (IntMaxScale)
)
' Note the syntax used:
'    - brackets around the CitectVBA function argument list.
'    (This is only necessary when the CitectVBA function is preceded
by an equals (=) sign .)
'     - double quotes around the Cicode function name
'    - no brackets around the Cicode function argument list
'    - brackets around individual variable arguments
' test results
If intRet = 0 Then
    '
    ' insert code for successful completion here
    '
    vntRet = CicodeCallReturn()
    strReply = "CicodeCallOpen Function successfully called"
Else
    '
    ' insert code for unsuccessful completion here
    '
    Select Case intRet
    Case = 1
        ' assign return comment for this case
        strReply = "CicodeCallOpen Function call error (unsuccessful)"
    Case = 2
        ' assign return comment for this case
        strReply = "Cicode Function not found"
    Case = 3
        ' assign return comment for this case
        strReply = "Wrong number of arguments "_
        & "in Cicode CallOpen function call"
```

```
    Case Else
        ' assign return comment for this case
        strReply = "Unknown error"
    End Select
End If
' display return comment for your information
MsgBox strReply
' assign return value for this function
TestCicode = intRet
End Function
```

## CicodeCallReturn

The CicodeCallReturn function is used to obtain the return value of the most recently completed Cicode function opened and run with the CitectVBA CicodeCallOpen function.

No arguments are passed to the CicodeCallReturn function, as it will return the result of the most recent return-value for the Cicode function called by the CitectVBA CicodeCallOpen function.

The CicodeCallReturn function should be used in its own separate line of CitectVBA code and must not be nested with the CicodeCallOpen function. For details, see Calling Cicode from CitectVBA.

### Syntax

*ReturnValue* = **CicodeCallReturn**()

*ReturnValue*:

The return value of the Cicode function specified in the most recent call of the CicodeCallOpen function. Note that the return data type of CicodeCallReturn will depend upon the return data type of the completed Cicode function most recently called by the CicodeCallOpen function.

### Return Value

CicodeCallReturn returns the return-value of the completed Cicode function most recently called by the CicodeCallOpen function.

### Related Functions

CicodeCallOpen

### Example

```
' declare modular variant variable to store function results
Dim vntRet as Variant
Function TestCicode() As Integer
' declare local variables
```

```
Dim intRet As Integer
Dim strReply as String
Dim intMaxScale as Integer
' copy current tag value to variable
' uses the project variable tag named MAX_SCALE
intMaxScale = MAX_SCALE
' call Cicode function
' for example: TrnSetScale( AN, Pen, Percent, Scale)
intRet = CicodeCallOpen( "TrnSetScale", 53, -1, 100, (IntMaxScale)
)
' Note the syntax used:
'    - brackets around the CitectVBA function argument list
'     - double quotes around the Cicode function name
'    - no brackets around the Cicode function argument list
'    - brackets around individual variable arguments

' test results
If intRet = 0 Then
    '
    ' insert code for successful completion here
    '
    vntRet = CicodeCallReturn()
    strReply = "CicodeCallOpen Function successfully called"
Else
    '
    ' insert code for unsuccessful completion here
    '
    Select Case intRet
    Case = 1
        ' assign return comment for this case
        strReply = "CicodeCallOpen Function call error (unsuccessful)"
    Case = 2
        ' assign return comment for this case
        strReply = "Cicode Function not found"
    Case = 3
        ' assign return comment for this case
        strReply = "Wrong number of arguments "_
        & "in Cicode CallOpen function call"
    Case Else
        ' assign return comment for this case
        strReply = "Unknown error"
    End Select
End If

' display return comment for your information
MsgBox strReply
' assign return value for this function
TestCicode = intRet

End Function
```

## End Function

The End Function statement ends a program or a block of statements within a function. A CitectVBA function starts with the FUNCTION statement and finishes with the END FUNCTION statement. All other statements that lie between the FUNCTION and END FUNCTION statements will be executed by the function when called to do so.

**Syntax**

**End {Function | Sub | If}**

**Related Functions**

Call | Sub | End Sub | Exit

**Example**

```
Function GetColor2( c% ) As Long
    GetColor2 = c% * 25
    If c% > 2 Then
        GetColor2 = 255        ' 0x0000FF - Red
    End If
    If c% > 5 Then
        GetColor2 = 65280    ' 0x00FF00 - Green
    End If
    If c% > 8 Then
        GetColor2 = 16711680    ' 0xFF0000 - Blue
    End If
End Function

Sub TestColor2
    Dim I as integer
    For I = 1 to 10
        Print GetColor2(I)
    Next I
End Sub
```

## End Sub

The End Sub statement ends a program or a block of statements within a subroutine. A CitectVBA subroutine starts with the SUB statement and finishes with the END SUB statement. All other statements that lie between the SUB and END SUB statements, will be executed by the subroutine, when called to do so.

**Syntax**

**End Sub**

**Related Functions**

Call | End Function | Sub | Exit

**Example**

```
Function GetColor2( c% ) As Long
GetColor2 = c% * 25
If c% > 2 Then
    GetColor2 = 255        ' 0x0000FF - Red
End If
If c% > 5 Then
    GetColor2 = 65280    ' 0x00FF00 - Green
End If
If c% > 8 Then
    GetColor2 = 16711680    ' 0xFF0000 - Blue
End If
End Function

Sub TestColor2
    Dim I as integer
    For I = 1 to 10
    Print GetColor2(I)
    Next I
End Sub
```

## Function

The Function statement declares and defines a function procedure, its name, parameters, and code to be enacted upon when the subroutine is called. Functions differ from subroutines in that functions return a value, whereas subroutines do not.

The required FunctionName is the name of the function being declared. The optional ArgList is the list of arguments used within the function.

A CitectVBA function starts with the FUNCTION statement and finishes with the END FUNCTION statement. All other statements that lie between the FUNCTION and END FUNCTION statements will be executed by the function when called to do so.

### Syntax

**Function** *FunctionName [(ArgList)] [As type]*

### Related Functions

Call | End Function | Sub | End Sub | Exit

### Example

```
Function GetColor2( c% ) As Long
GetColor2 = c% * 25
If c% > 2 Then
    GetColor2 = 255    ' 0x0000FF - Red
End If
If c% > 5 Then
    GetColor2 = 65280    ' 0x00FF00 - Green
End If
```

```
If c% > 8 Then
    GetColor2 = 16711680    ' 0xFF0000 - Blue
End If
End Function

Sub TestColor2
    Dim I as integer
    For I = 1 to 10
    Print GetColor2(I)
    Next I
End Sub
```

## Sub

Declares and defines a subroutine procedure, its name, parameters, and code to be enacted upon when the subroutine is called. Subroutines differ from functions in that functions return a value, whereas subroutines do not.

The required SubroutineName is the name of the subroutine being declared.

The optional ArgList is the list of arguments used within the subroutine.

A CitectVBA subroutine starts with the SUB statement and finishes with the END SUB statement. All other statements that lie between the SUB and END SUB statements, will be executed by the subroutine, when called to do so.

### Syntax

**Sub**

### Related Functions

Call | End Function | End Sub | Exit

### Example

```
Function GetColor2( c% ) As Long
GetColor2 = c% * 25
If c% > 2 Then
    GetColor2 = 255        ' 0x0000FF - Red
End If
If c% > 5 Then
    GetColor2 = 65280    ' 0x00FF00 - Green
End If
If c% > 8 Then
    GetColor2 = 16711680    ' 0xFF0000 - Blue
End If
End Function

Sub TestColor2
    Dim I as integer
```

```
    For I = 1 to 10
    Print GetColor2(I)
    Next I
End Sub
```

## VbCallOpen function

The VbCallOpen function is a Cicode function used to call a CitectVBA function or subroutine from Cicode. It is used to initiate a call to the CitectVBA function or subroutine and returns a handle (of OBJECT data type) to that opened function call.

VbCallOpen is used in conjunction with VbCallRun and VbCallReturn functions, which can all be nested to implement the entire function set with a single line of Cicode. For further information, see the section "Calling CitectVBA from Cicode".

### Syntax

*ReturnValue* = **VbCallOpen**(*FunctName*, *ArgList*)

*ReturnValue*:

The handle to the opened CitectVBA function.

*FunctName*:

The name of the CitectVBA function or subroutine being called.

*ArgList*:

A comma separated list of arguments to pass to the function or subroutine being called.

### Return Value

VbCallOpen returns an Object data type containing a handle to the CitectVBA function being called. If the function cannot open the CitectVBA function or subroutine the return value is zero.

### Related Functions

VbCallRun function | VbCallReturn function

### Example

```
FUNCTION
TestCitectVBA()
INT iRet;
STRING sMsg = "Hello";
INT iVal = 123;

iRet = VbCallReturn(VbCallRun(VbCallOpen("CiVBATest",
```

```
iVal)));
Message("TestCitectVBA Function", "CiVBATest = " +
IntToStr(iRet), 0);
END
```

**Example**

```
Function CiVBATest(Value As Integer) As Integer
CiVBATest = Value * 2
End Function
```

## VbCallReturn function

Used to obtain the return value of the completed CitectVBA function (previously opened with the Cicode VbCallOpen function), and requires the handle returned from the VbCall-Run function call.

VbCallReturn is used in conjunction with VbCallOpen and VbCallRun functions, which can all be nested to implement the entire function set with a single line of Cicode. For further information, see the section Calling CitectVBA from Cicode.

**Syntax**

*ReturnValue* = **VbCallReturn**(*CallHandle*)

*ReturnValue*:

The value returned by the completed CitectVBA function (which was previously opened by the Cicode VbCallOpen function). The data type of the return value is dependent upon the data type of the return value for the CitectVBA function opened.

*CallHandle*:

The handle to the previously opened CitectVBA function as returned by the Cicode VbCallRun function

**Return Value**

VbCallReturn returns the completed return value for the CitectVBA function.

**Related Functions**

VbCallOpen function | VbCallRun function

**Example**

```
FUNCTION
TestCitectVBA()
INT iRet;
```

```
STRING sMsg = "Hello";
INT iVal = 123;

iRet = VbCallReturn(VbCallRun(VbCallOpen("CiVBATest",
iVal)));
Message("TestCitectVBA Function", "CiVBATest = " +
IntToStr(iRet), 0);
END
```

**Example**

```
Function CiVBATest(Value As Integer) As Integer
CiVBATest = Value * 2
End Function
```

## VbCallRun function

Used to execute the CitectVBA function or subroutine (previously opened with the Cicode VbCallOpen function), and requires the handle returned from the VbCallOpen function call.

The VbCallRun function provides an opportunity for the opened CitectVBA function to complete and return a value in the multi-threaded Citect/SCADA environment. It passes its argument value (of OBJECT data type) through as its return value upon completion.

VbCallRun is used in conjunction with VbCallOpen and VbCallReturn functions, which can all be nested to implement the entire function set with a single line of Cicode. For details, see Calling CitectVBA from Cicode.

**Syntax**

*ReturnValue* = **VbCallRun**(*CallHandle*)

*ReturnValue*:

The handle to the opened CitectVBA function passed in as CallHandle.

*CallHandle*:

The handle to the previously opened CitectVBA function as returned by the VbCallOpenfunction.

**Return Value**

VbCallRun (passes through and) returns a Object data type containing a handle to the CitectVBA function being called.

**Related Functions**

VbCallOpen function | VbCallReturn function

**Example**

```
FUNCTION
TestCitectVBA()
INT iRet;
STRING sMsg = "Hello";
INT iVal = 123;
iRet = VbCallReturn(VbCallRun(VbCallOpen("CiVBATest",
iVal)));
Message("TestCitectVBA Function", "CiVBATest = " +
IntToStr(iRet), 0);
END
```

**Example**

```
Function CiVBATest(Value As Integer) As Integer
CiVBATest = Value * 2
End Function
```

## String Functions

CitectVBA strings functions are provided to create, edit and implement strings within CitectVBA code. The strings functions predefined in CitectVBA are:

| | |
|---|---|
| Asc | Returns a numeric value that is the ASCII code for the first character in a string. |
| Chr | Converts an ASCII number to a one character string. |
| InStr | Returns the character position of the first occurrence of string2 within string1. |
| LCase | Returns a copy of string in which all characters have been converted to lowercase. |
| Left, Left$ | Returns the left most characters of a string parameter. |
| Len | Determines the number of characters in the stringargument. |
| Line Input # | Reads a single line from an open sequential file and assigns it to a string variable. |
| LTrim | Strips any leading spaces from a string variable. |
| Mid | Returns a substring within a string. |

| | |
|---|---|
| Option Compare | Determines the default string comparison method. |
| Option Explicit | Forces explicit declaration of all variables. |
| Right | Returns the right most characters of a string parameter. |
| RTrim | Strips any trailing spaces from a string variable. |
| Space | Adds a specified number of spaces in a print statement. |
| StrComp | Returns a variant that is the result of the comparison of two strings. |
| String | Create a string that consists of one character repeated a specific number of times. |
| Trim | Strips any leading and trailing spaces from Str variable. |
| UCase | Returns a copy of string in which all characters have been converted to uppercase. |

## Asc

Converts a text string character to its numeric ASCII code value. The Asc function expects the argument *Str* to be a valid string expression. If *Str*contains no characters, a runtime error occurs. The Asc function performs the opposite of the Chr function, which converts a number into its string character ASCII code value.

### Syntax

**Asc**(*Str*)

*Str*:

A string or expression that can represent a valid text value.

### Return Value

Returns the numeric ASCII code value of the first character in *Str* provided in the argument.

### Related Functions

Chr

### Example

```
Dim vntVar ' declare result holder variable
vntVar = Asc("A")' returns 65
vntVar = Asc("Z")' returns 90
vntVar = Asc("a")' returns 97
vntVar = Asc("z")' returns 122
vntVar = Asc("Apple")' returns 65
vntVar = Asc("Zoe")' returns 90
```

## Chr

Converts a number into its string character ASCII code value.

The Chr function expects the argument *Num* to be a valid numeric integer (whole positive number within the range 0 to 255 inclusive). If *Chr*contains no number, a runtime error occurs.

**Note:** Values 8, 9, 10, and 13 convert to backspace, tab, linefeed, and carriage return characters respectively.

The Chr function performs the opposite of the Asc function, which converts a text string character to it's numeric ASCII code value.

### Syntax

**Chr**(*Num*)

*Num*:

An integer or expression representing a valid numeric value.

### Return Value

Returns a single character string representing the ASCII character code value of the number *Num* provided in the argument.

### Related Functions

Asc

### Example

```
Dim vntVar     ' declare result holder variable
vntVar = Chr(65)    ' returns "A"
vntVar = Chr(97)    ' returns "a"
vntVar = Chr(90)    ' returns "Z"
vntVar = Chr(122)    ' returns "z"
```

## InStr

Returns the character position of the first occurrence of *String2* within *String1*.

**Syntax**

**InStr**(*StartPos*, *StringToSearch*, StringToMatch)

*StartPos*:

A numeric expression that sets the starting position for the search. If omitted, search begins at the first character position. If Num contains Null, an error occurs. An Integer or expression representing a valid numeric value.

*StringToSearch*:

The string expression being searched. A string or expression that can represent a valid text value.

*StringToMatch*:

The string expression being searched for. A string or expression that can represent a valid text value.

**Return Value**

Returns a variant containing a Long data type indicating the result of the string search. Returns 0 if:

- *StringToSearch* is of zero length.
- *StringToMatch* is not found.
- *StartPos* is longer than *StringToMatch*.

Returns a value representing the count position where character match was first found.

Returns Null if StringToSearch or StringToMatch contains null.

**Related Functions**

IsNull | Left, Left$ | Mid | Right | StrComp

**Example**

```
Dim strToSearch as String
Dim strToFind as String
Dim lngPosition as Long
strToSearch = "Good Bye"
' note this has an uppercase "B"
strToFind = "bye"
' note this has a lowercase "b"
lngPosition = InStr(1, strToSearch, strToFind, 0)
' returns 0 (Did not find match)
lngPosition = InStr(1, strToSearch, strToFind, 1)
' returns 6 (Position of first character in match)
```

## LCase

Converts all uppercase letters in *Str* to lowercase letters. All lowercase letters and non-letter characters remain unchanged.

**Syntax**

**LCase**(*Str*)

*Str:*

A string or expression that can represent a valid text value.

**Return Value**

Returns a string.

**Related Functions**

UCase

**Example**

```
Dim strMixedCase as String
Dim strLowerCase as String
Dim strUpperCase as String
strMixedCase = "AbCdE"
strLowerCase = LCase(strMixedCase)    ' returns "abcde"
strUpperCase = UCase(strMixedCase)    ' returns "ABCDE"
```

## Left, Left$

Returns the left most *Num* characters of *Str*.

The required *Str* argument is a String expression from which the leftmost characters are returned. If *Str* contains **Null**, **Null** is returned.

The required *Num* argument is a Variant (Long) numeric expression indicating how many characters to return. If 0, a zero-length string (" ") is returned. If greater than or equal to the number of characters in string, the entire string is returned.

**Syntax**

**Left**(*Str*, *Num*)

*Str*:

A string or expression that can represent a valid text value.

*Num:*

An Integer or expression representing a valid numeric value.

**Return Value**

The Left function returns a variant containing a String data type. The Left$ function returns a String.

**Related Functions**

InStr | Mid | Right

**Example**

```
Dim strGreeting as String
Dim strTest
strGreeting = "Hello World"
strTest = Left(strGreeting, 1)    ' Returns "H".
strTest = Left(strGreeting, 7)    ' Returns "Hello W".
strTest = Left(strGreeting, 20)    ' Returns "Hello World".
```

## Len

The Len function determines the number of characters in the *Str* argument. The LenB function determines the number of bytes in the *VarName* argument.

- The *Str* argument can be any valid string expression. If *Str* contains **Null**, **Null** is returned.
- The *VarName* argument can be any valid variable name. If *VarName* contains **Null**, **Null** is returned. If *VarName* is a Variant, LenB treats it the same as a String and returns the number of characters it contains.

**Syntax**

**Len**(*Str*)

*Str:*

A string or expression that can represent a valid text value.

**Return Value**

Returns a Long.

**Related Functions**

InStr | Left, Left$ | Mid | Right

**Example**

```
Dim strTest as String
Dim lngStringLength as Long
strTest = "CitectVBA"
lngStringLength = Len(strTest)    ' returns 9
```

## Line Input #

Line Input # statement reads a single line from an open sequential file and assigns it to a String variable.

The required *FileNum* argument is a system reference number associated with an open file. The required *VarName* is the name of the variable where the file data is read (copied) to.

> **Note:** The number sign (# ) preceding *FileNum* is not optional.

The Line Input # statement reads from a file one character at a time until it encounters a carriage return (Chr(13)) or carriage return-linefeed (Chr(13) + Chr(10)) sequence. Carriage return - linefeed sequences are skipped rather than appended to the character string.

> **Note:** The file system keeps track of all open files and the current position of access within every file. Every statement or function that accesses the data within a file, alters the current position within that file. The Loc function can be used to determine the current position within an open file.

Data read with the Line Input # statement has usually been written to a file with the Print # statement.

### Syntax

**Line Input** # FileNum, *VarName*

*FileNum*:

An Integer or numeric expression representing any valid number in the range 1 to 511 inclusive, which is referenced by the file system to be associated with an open file.

*VarName:*

A string representing a valid variable name.

### Related Functions

Get # | GetAttr | Input | Print # | Put # | Write #

### Example

```
Dim strTextLine As String
Dim intFileNum As Integer
Open "c:\TEST.txt" For Input As #intFileNum    intFileNum = FreeFile
'retrieve next free file number
Do While Not EOF(intFileNum)            ' Loop until end of file.
```

```
    Line Input #intFileNum, strTextLine    ' Read line into variable.
    Print TextLine               ' Print line.
Loop
Close #intFileNum
```

## LTrim

Strips any leading spaces from *Str* variable.

### Syntax

**LTrim**(*Str*)

*Str*:

A string or expression that can represent a valid text value.

### Return Value

Returns a string.

### Related Functions

RTrim | Trim

### Example

```
Dim strTest as String
Dim strResult as String
Dim lngStartLength as Long
Dim lngFinishLength as Long
strTest = "   CitectVBA"
lngStartLength = Len(strTest)        ' returns 14
strResult = LTrim(strTest)            ' returns "CitectVBA"
lngStringLength = Len(strResult)        ' returns 9
```

## Mid

The Mid Function extracts a portion of a string from Str.

**Note:** To determine the number of characters in a string, use the Len function.

The Str argument can be any valid string expression. If Str contains Null, Null is returned.

The required Num argument is a Long numeric expression that sets the starting position for the extraction. If Num is greater than the number of characters in string, Mid returns a zero-length string ("").

The optional Len argument is a Variant containing a Long data type representing the number of characters to return. If omitted or if there are fewer than Len characters in Str (including the character at position Num ), all characters from the Num position to the end of the string are returned.

**Syntax**

**Mid**(*Str, Num, Len*)

*Str*:

A string or expression that can represent a valid text value. If Str contains Null, Null is returned.

*Num*:

A Long numeric expression that sets the starting position for the extraction. If Num is greater than the number of characters in string, Mid returns a zero-length string ("").

*Len*:

A Variant containing a Long data type representing the number of characters to return. If omitted or if there are fewer than Len characters in Str (including the character at position Num ), all characters from the Num position to the end of the string are returned.

**Return Value**

The Mid function returns a Variant (containing a String data type).

**Related Functions**

InStr | Left, Left$ | Right

**Example**

```
Dim strSource as String
Dim strFirstWord as String
Dim strSecondWord as String
Dim strThirdWord as String
Dim lngPosition as Long
Dim lngNextPosition as Long
Dim lngWordLength as Long
strSource = "Mid Function Demo"    ' Create test string.
lngPosition = 1    ' Start at character position 1
lngNextPosition = Instr(lngPosition, strSource," ")    ' Locate first space char-
acter
lngWordLength = lngNextPosition - lngPosition    ' calculate word length
strFirstWord = Mid(strSource, lngPosition, lngWordLength)   ' Returns first word
"Mid"
lngPosition = lngNextPosition + 1    ' Move to next word position
lngNextPosition = Instr(lngPosition, strSource," ")    ' Locate next space character
lngWordLength = lngNextPosition - lngPosition    ' calculate word length
strSecondWord = Mid(strSource, lngPosition, lngWordLength)    ' Returns second word
```

```
 "Function"
lngPosition = lngNextPosition + 1    ' Move to next word position
lngNextPosition = Instr(lngPosition, strSource," ")    ' Locate next space character
lngWordLength = lngNextPosition - lngPosition    ' calculate word length
strThirdWord = Mid(strSource, lngPosition, lngWordLength)    ' Returns third word
 "Demo"
```

## Option Compare

Determines how strings are compared within a CitectVBA module. The optional Option Compare statement if used, must be placed at the top of the CitectVBA file along with any other Option declarations.

If an Option Compare statement is not included, the default text comparison method is Binary.

**Syntax**

**Option Compare** {**Binary** | **Text**}

**Related Functions**

InStr | StrComp

**Example**

```
Option Compare Binary
Dim vntResult as Variant
vntResult = StrComp("CitectVBA rules!", "Citectvba Rules!")
' returns 1 (strings unequal)
```

**Example**

```
Option Compare Text
Dim vntResult as Variant
vntResult = StrComp("CitectVBA rules!", "Citectvba Rules!")
' returns 0 (strings equal)
```

## Option Explicit

Forces explicit declaration of all variables.

The optional Option Explicit statement if used, must be placed at the top of the CitectVBA file. This causes a check of variable declarations at compile time. Setting this option is a good way to catch misspelling of variables in your code.

**Syntax**

**Option Explicit**

**Related Functions**

Const | Dim

**Example**

```
Option Explicit
' various statements go here
' a compile error will occur with the following line
strMyVar = "This string assignment won't work"
'because strMyVar is not explicitly declared
```

## Right

Returns the right most *Num* characters of *Str*. The required *Str* argument is a String expression from which the rightmost characters are returned. If *Str* contains **Null**, **Null** is returned.

The required *Num* argument is a Variant (Long) numeric expression indicating how many characters to return. If 0, a zero-length string (" ") is returned. If greater than or equal to the number of characters in string, the entire string is returned.

> **Note:** To determine the number of characters in a string, use the Len function.

**Syntax**

**Right**(*Str*, *Num*)

*Str:*

A string or expression that can represent a valid text value.

*Num*:

An Integer or expression representing a valid numeric value.

**Return Value**

The Right function returns a variant containing a string data type.
The Right$ function returns a string.

**Related Functions**

InStr | Left, Left$ | Mid

**Example**

```
Dim strGreeting as String
Dim strTest
strGreeting = "Hello World"
strTest = Right(strGreeting, 1)     ' Returns "d"
strTest = Right(strGreeting, 5)     ' Returns "World"
strTest = Right(strGreeting, 20)     ' Returns "Hello World"
```

## RTrim

Strips any trailing spaces from *Str*variable.

### Syntax

**RTrim**(*Str*)

*Str*:

A string or expression that can represent a valid text value.

### Return Value

Returns a String.

### Related Functions

LTrim | Trim

### Example

```
Dim strTest as String
Dim strResult as String
Dim lngStartLength as Long
Dim lngFinishLength as Long
strTest = "CitectVBA   "
lngStartLength = Len(strTest)     ' returns 14
strResult = RTrim(strTest)     ' returns "CitectVBA"
lngStringLength = Len(strResult)     ' returns 9
```

## Space

Creates a String consisting of the specified number *Num* of spaces. The Space function is useful for formatting output and clearing data in fixed-length strings.

### Syntax

**Space**(*Num*)

*Num:*

An Integer or expression representing a valid numeric value.

**Return Value**

Returns a Variant containing a String data type.

**Related Functions**

String

**Example**

```
Dim strTest as String
' Returns a string with 10 spaces.
strTest = Space(10)

' Insert 10 spaces between two strings.
strTest = "Hello" & Space(10) & "World"
```

## StrComp

Returns an integer that is the result of the comparison of two strings.

The required *String1* argument is any valid string expression. The required *String2* argument is any valid string expression.

The optional *Compare* argument is a numeric expression that specifies the type of string comparison. It can be omitted, 0, or 1. Specify 0 (default) to perform a binary comparison. Specify 1 to perform a textual comparison. If compare is Null, an error occurs.

**Syntax**

**StrComp**(*String1*, *String2*)

*String1*:

A string or expression that can represent a valid text value.

*String2*:

A string or expression that can represent a valid text value.

**Return Value**

Returns a variant containing an integer data type indicating the result of the string compare:

- Returns -1 where *String1* is less than *String2*.
- Returns 0 where *String1* is equal to *String2*.
- Returns 1 where *String1* is greater than *String2*.
- Returns **Null** where *String1* or *String2*s **Null**.

**Example**

```
Dim strTest1 as String
Dim strTest2 as String
Dim strTest3 as String
Dim vntComp
strTest1 = "ABCD"
strTest2 = "abcd"
strTest3 = NULL
vntComp = StrComp(strTest1, strTest2) ' Returns -1 (less than)
vntComp = StrComp(strTest1, strTest1) ' Returns 0 (equal to)
vntComp = StrComp(strTest2, strTest1) ' Returns 1 (greater than)
vntComp = StrComp(strTest1, strTest3) ' Returns NULL (strTest3 is NULL)
```

## String

Creates a string that consists of one character repeated a specific number of times.

The required *Num* argument is Long numeric expression indicating how many characters to return. If *Num* contains **Null**, **Null** is returned.

The required *Character* argument is a String expression from which the first character is repeated and returned, or is a Variant (Long) representing a valid character code. If character contains **Null**, **Null** is returned.

### Syntax

**String**(*Num*)

*Num*:

An Integer or expression representing a valid numeric value.

### Related Functions

Space

### Example

```
Dim strTest as String
strTest = String(5, "*")
' Returns "*****"
strTest = String(5, 42)
' Returns "44444"
strTest = String(10, "Today")
' Returns "TTTTTTTTTT"
```

## Trim

Strips any leading and trailing spaces from *Str* variable.

### Syntax

**Trim**(*Str*)

*Str*:

A string or expression that can represent a valid text value.

**Return Value**

Returns a String.

**Related Functions**

LTrim | RTrim

**Example**

```
Dim strTest as String
Dim strResult as String
Dim lngStartLength as Long
Dim lngFinishLength as Long
strTest = "   CitectVBA   "
lngStartLength = Len(strTest)
' returns 19
strResult = Trim(strTest)
' returns "CitectVBA"
lngStringLength = Len(strResult)
' returns 9
```

## UCase

Converts all lowercase letters in *Str* to uppercase letters. All uppercase letters and non-letter characters remain unchanged.

**Syntax**

**UCase**(*Str*)

*Str*:

A string or expression that can represent a valid text value.

**Return Value**

Returns a string.

**Related Functions**

UCase

**Example**

```
Dim strMixedCase as String
Dim strLowerCase as String
Dim strUpperCase as String
strMixedCase = "AbCdE"
strLowerCase = LCase(strMixedCase)    ' returns "abcde"
strUpperCase = UCase(strMixedCase)    ' returns "ABCDE"
```

# Chapter 6: ASCII/ANSI Character Code Listings

The table below shows the Latin 1 ANSI character set.

Codes 0-31 are control codes. The standard ASCII codes are from 32-127 (decimal) and are common regardless of the ANSI set being used. The remaining codes from 160-255 (decimal) vary between languages dependent upon the ANSI set being used.

| Symbol | Decimal | Hex |
|---|---|---|
| {NUL} | 0 | 00 |
| {SOH} | 1 | 01 |
| {STX} | 2 | 02 |
| {ETX} | 3 | 03 |
| {EOT} | 4 | 04 |
| {ENQ} | 5 | 05 |
| {ACK} | 6 | 06 |
| {BEL} | 7 | 07 |
| {BS} | 8 | 08 |
| {HT} | 9 | 09 |
| {LF} | 10 | 0A |
| {VT} | 11 | 0B |
| {FF} | 12 | 0C |
| {CR} | 13 | 0D |
| {SO} | 14 | 0E |
| {SI} | 15 | 0F |

| Symbol | Decimal | Hex |
|---|---|---|
| {DLE} | 16 | 10 |
| {DC1} | 17 | 11 |
| {DC2} | 18 | 12 |
| {DC3} | 19 | 13 |
| {DC4} | 20 | 14 |
| {NAK} | 21 | 15 |
| {SYN} | 22 | 16 |
| {ETB} | 23 | 17 |
| {CAN} | 24 | 18 |
| {EM} | 25 | 19 |
| {SUB} | 26 | 1A |
| {ESC} | 27 | 1B |
| {FS} | 28 | 1C |
| {GS} | 29 | 1D |
| {RS} | 30 | 1E |
| {US} | 31 | 1F |
| {SPC} | 32 | 20 |
| ! | 33 | 21 |
| " | 34 | 22 |
| # | 35 | 23 |
| $ | 36 | 24 |

| Symbol | Decimal | Hex |
|---|---|---|
| % | 37 | 25 |
| & | 38 | 26 |
| ' | 39 | 27 |
| ( | 40 | 28 |
| ) | 41 | 29 |
| * | 42 | 2A |
| + | 43 | 2B |
| , | 44 | 2C |
| - | 45 | 2D |
| . | 46 | 2E |
| / | 47 | 2F |
| 0 | 48 | 30 |
| 1 | 49 | 31 |
| 2 | 50 | 32 |
| 3 | 51 | 33 |
| 4 | 52 | 34 |
| 5 | 53 | 35 |
| 6 | 54 | 36 |
| 7 | 55 | 37 |
| 8 | 56 | 38 |
| 9 | 57 | 39 |

| Symbol | Decimal | Hex |
|---|---|---|
| : | 58 | 3A |
| ; | 59 | 3B |
| < | 60 | 3C |
| = | 61 | 3D |
| > | 62 | 3E |
| ? | 63 | 3F |
| @ | 64 | 40 |
| A | 65 | 41 |
| B | 66 | 42 |
| C | 67 | 43 |
| D | 68 | 44 |
| E | 69 | 45 |
| F | 70 | 46 |
| G | 71 | 47 |
| H | 72 | 48 |
| I | 73 | 49 |
| J | 74 | 4A |
| K | 75 | 4B |
| L | 76 | 4C |
| M | 77 | 4D |
| N | 78 | 4E |

| Symbol | Decimal | Hex |
|---|---|---|
| O | 79 | 4F |
| P | 80 | 50 |
| Q | 81 | 51 |
| R | 82 | 52 |
| S | 83 | 53 |
| T | 84 | 54 |
| U | 85 | 55 |
| V | 86 | 56 |
| W | 87 | 57 |
| X | 88 | 58 |
| Y | 89 | 59 |
| Z | 90 | 5A |
| [ | 91 | 5B |
| \ | 92 | 5C |
| ] | 93 | 5D |
| ^ | 94 | 5E |
| _ | 95 | 5F |
| ` | 96 | 60 |
| a | 97 | 61 |
| b | 98 | 62 |
| c | 99 | 63 |

| Symbol | Decimal | Hex |
|---|---|---|
| d | 100 | 64 |
| e | 101 | 65 |
| f | 102 | 66 |
| g | 103 | 67 |
| h | 104 | 68 |
| i | 105 | 69 |
| j | 106 | 6A |
| k | 107 | 6B |
| l | 108 | 6C |
| m | 109 | 6D |
| n | 110 | 6E |
| o | 111 | 6F |
| p | 112 | 70 |
| q | 113 | 71 |
| r | 114 | 72 |
| s | 115 | 73 |
| t | 116 | 74 |
| u | 117 | 75 |
| v | 118 | 76 |
| w | 119 | 77 |
| x | 120 | 78 |

| Symbol | Decimal | Hex |
|---|---|---|
| y | 121 | 79 |
| z | 122 | 7A |
| { | 123 | 7B |
| \| | 124 | 7C |
| } | 125 | 7D |
| ~ | 126 | 7E |
| {Delete} | 127 | 7F |
| | 128 | 80 |
| | 129 | 81 |
| ‚ | 130 | 82 |
| ƒ | 131 | 83 |
| „ | 132 | 84 |
| … | 133 | 85 |
| † | 134 | 86 |
| ‡ | 135 | 87 |
| ˆ | 136 | 88 |
| ‰ | 137 | 89 |
| Š | 138 | 8A |
| < | 139 | 8B |
| Œ | 140 | 8C |
| | 141 | 8D |

| Symbol | Decimal | Hex |
|---|---|---|
| | 142 | 8E |
| | 143 | 8F |
| | 144 | 90 |
| ` | 145 | 91 |
| ' | 146 | 92 |
| " | 147 | 93 |
| " | 148 | 94 |
| · | 149 | 95 |
| - | 150 | 96 |
| - | 151 | 97 |
| ~ | 152 | 98 |
| ™ | 153 | 99 |
| š | 154 | 9A |
| > | 155 | 9B |
| œ | 156 | 9C |
| | 157 | 9D |
| | 158 | 9E |
| Ÿ | 159 | 9F |
| {NBSP} | 160 | A0 |
| ¡ | 161 | A1 |
| ¢ | 162 | A2 |

| Symbol | Decimal | Hex |
|---|---|---|
| £ | 163 | A3 |
| ¤ | 164 | A4 |
| ¥ | 165 | A5 |
| ¦ | 166 | A6 |
| § | 167 | A7 |
| ¨ | 168 | A8 |
| © | 169 | A9 |
| ª | 170 | AA |
| « | 171 | AB |
| ¬ | 172 | AC |
|  | 173 | AD |
| ® | 174 | AE |
| ¯ | 175 | AF |
| ° | 176 | B0 |
| ± | 177 | B1 |
| ² | 178 | B2 |
| ³ | 179 | B3 |
| ´ | 180 | B4 |
| µ | 181 | B5 |
| ¶ | 182 | B6 |
| · | 183 | B7 |

| Symbol | Decimal | Hex |
|---|---|---|
| ¸ | 184 | B8 |
| ¹ | 185 | B9 |
| º | 186 | BA |
| » | 187 | BB |
| ¼ | 188 | BC |
| ½ | 189 | BD |
| ¾ | 190 | BE |
| ¿ | 191 | BF |
| À | 192 | C0 |
| Á | 193 | C1 |
| Â | 194 | C2 |
| Ã | 195 | C3 |
| Ä | 196 | C4 |
| Å | 197 | C5 |
| Æ | 198 | C6 |
| Ç | 199 | C7 |
| È | 200 | C8 |
| É | 201 | C9 |
| Ê | 202 | CA |
| Ë | 203 | CB |
| Ì | 204 | CC |

| Symbol | Decimal | Hex |
|---|---|---|
| Í | 205 | CD |
| Î | 206 | CE |
| Ï | 207 | CF |
| Ð | 208 | D0 |
| Ñ | 209 | D1 |
| Ò | 210 | D2 |
| Ó | 211 | D3 |
| Ô | 212 | D4 |
| Õ | 213 | D5 |
| Ö | 214 | D6 |
| × | 215 | D7 |
| Ø | 216 | D8 |
| Ù | 217 | D9 |
| Ú | 218 | DA |
| Û | 219 | DB |
| Ü | 220 | DC |
| Ý | 221 | DD |
| Þ | 222 | DE |
| ß | 223 | DF |
| à | 224 | E0 |
| á | 225 | E1 |

| Symbol | Decimal | Hex |
|---|---|---|
| â | 226 | E2 |
| ã | 227 | E3 |
| ä | 228 | E4 |
| å | 229 | E5 |
| æ | 230 | E6 |
| ç | 231 | E7 |
| è | 232 | E8 |
| é | 233 | E9 |
| ê | 234 | EA |
| ë | 235 | EB |
| ì | 236 | EC |
| í | 237 | ED |
| î | 238 | EE |
| ï | 239 | EF |
| ð | 240 | F0 |
| ñ | 241 | F1 |
| ò | 242 | F2 |
| ó | 243 | F3 |
| ô | 244 | F4 |
| õ | 245 | F5 |
| ö | 246 | F6 |

| Symbol | Decimal | Hex |
|---|---|---|
| ÷ | 247 | F7 |
| ø | 248 | F8 |
| ù | 249 | F9 |
| ú | 250 | FA |
| û | 251 | FB |
| ü | 252 | FC |
| ý | 253 | FD |
| þ | 254 | FE |
| ÿ | 255 | FF |

Index


## A

Abs function 194
access, file 101
Application Programming Interface (API) 84
arguments 82, 87, 89
arithmetical operators 64
array subscripts 44
arrays 43
    declaration 43
    dimensions 44
    dynamic size 47
    fixed size 45
    multi-dimensional 47
    subscripts 44
Asc function 117, 218
assigning references 93
assignment operators 63
Atn function 200

## B

Beep statement 203
bounds 44
ByRef 87
ByVal 87

## C

calendars, in databases 62
Call 206
Call statement 206
CDate function 119
CDbl function 120
character
    line continuation 32
    underscore 32
ChDir statement 162
ChDrive statement 163
Chr function 118, 219
CInt function 120
CLng function 121
Close statement 164
coercion and variant data types 48
comments 30
    file header 30
comparing strings 68
concatenation 68
Const 137
Const statement 137
constant declaration 38
constant naming 33
constants 33, 37
    date 55
    declaring 38
    scope 28
constants, intrinsic 39
constraints, date and time 59
control structures 70
    DO statement 70
    WHILE statement 71
Cos function 201
CSng function 121
CStr function 122
CurDir function 165
CVar function 122

## D

data types 36
    arrays 43
    coercion 48
    default 48
    numeric 50
    variant as default 48
databases and calendars 62
date 60
    data type structure 60
date and time data constraints 59
Date Cicode function 150
date constants 55
date data type structure 60
date formatting 57
date functions 148

date handling 54
date values 61
DateSerial function 123, 151
DateValue function 152
Day function 152
decimal numbers 50
decision making
  DO Statement 70
  WHILE statement 71
declaration, object 92
deletion, object 100
Dim statement 103, 139
dimension 40
  array declaration 43
  array subscript declaration 44
  variable declaration 40
Dir function 109, 166
DO statement 70
DO Statement 70
double precison numbers 50
Dynamic Linked Libraries (DLLs) 84
dynamic size arrays 47

## E

End Function statement 110, 211
END statement 76
End Sub statement 211
EOF function 168
Erase statement 104
EXIT statement 76
Exp function 195
exponential notation 51

## F

file access 101
file I/O functions 160
FileCopy function 169
FileLen function 170
files 27
Fix function 195
fixed size arrays 45
floating point calculation rules 52
floating point numbers 50
FOR statement 71
Format function 125
formatting, date 57
FreeFile function 170
function
  Abs 194
  Asc 117, 218
  Atn 200
  Beep 203
  CDate 119
  CDbl 120
  Chr 118, 219
  CInt 120
  CLng 121
  Const statement 137
  Cos 201
  CSng 121
  CStr 122
  CurDir 165
  CVar 122
  DateSerial 123, 151
  DateValue 152
  Day 152
  Dim statement 103, 139
  Dir 166
  EOF 168
  Erase statement 104
  Exp 195
  FileCopy 169
  FileLen 170
  Fix 195
  Format 125
  FreeFile 170
  GetAttr 173
  Hex 132
  Hour 153
  Input # 175
  InStr function 220
  Int 196
  IsDate function 140
  IsEmpty 140
  IsNull 141
  IsNumeric 142
  Lbound 105

LCase 221
Left 221
Left$ 221
Len 222
Loc 178
LOF 179
Log 197
LTrim 224
Mid 224
Minute 153
Month 154
Now 155
Oct 133
Option Base statement 106, 143
Option Explicit 226
Print # 184
Put # 187
ReDim 107, 145
Rem 204
Right 227
Rnd 198
RTrim 228
Second 155
Seek 192
SendKeys 204
Sgn 198
Sin 202
Space 228
Sqr 199
Str 134
StrComp 229
String 230
Tan 202
Timer event 157
TimeSerial 124, 158
TimeValue 158
Trim 230
Ubound 108
UCase 231
Val 134
VarType 147
WeekDay 159
Write # 192
Year 160

Function statement 212
functions 33, 78, 80, 203

## G

Get statement 171
GetAttr function 173
global scope 29
GOTO statement 70

## H

handling, date 54
headers, file 30
Hex function 132
hexadecimal numbers 50
Hour function 153

## I

IF statement 72
initializing variables 42
Input # function 175
InStr function 220
Int function 196
intrinsic constants 39
IsDate function 140
IsEmpty function 140
IsNull function 141
IsNumeric function 142

## K

keywords 33
Kill statement 176

## L

labels 32-33
Lbound function 105
LCase function 221
Left function 221
Left$ function 221
Len function 222
LenB function 222
lifetime, scope 28
line continuation character 32

Line Input # statement 177, 223
Loc function 178
local scope 28
LOF function 179
Log function 197
logical operators 66
loops
    DO Statement 70
    WHILE statement 71
lower bound 44
LTrim function 224

## M

math functions 194
mathematical operators 64
Microsoft Excel OLE 100
Microsoft Word OLE 99
Mid function 224
Minute function 153
MkDir statement 180
models, object 95
modular scope 29
Month function 154
multi-dimensional arrays 47

## N

Name statement 181
naming 33
    labels 32
notation, exponential 51
Now function 155
numbers 50
    data types 50
numbers, rounding rules for 53
numeric data types 50
numeric precision 50

## O

object declaration 92
object deletion 100
object models 95
Oct function 133
octal numbers 50
OLE automation objects 91, 94
OLE services 90
OnError statement 77
Open statement 182
operator precedence 66
operators 63
    arithmetic 64
operators, assignment 63
operators, logical 66
operators, relational 65
option base statement 35
Option Base statement 106, 143
option compare statement 35
option explicit 34
option explicit statement 34
Option Explicit statement 226
option statements 34

## P

precedence, operator 66
precision, numeric 50
Print # function 184
private scope 28
procedure functions 205-206
    Call 206
    End Function statement 110, 211
    End Sub statement 211
    Function 212
    Sub 213
public scope 28
Put # function 187

## R

ReDim statement 107, 145
relational operators 65
Rem statement 204
Right function 227
RmDir statement 190
Rnd function 198
RTrim function 228
rules, floating point 52
rules, rounding 53

## S

scope 28
Second function 155
Seek function 192
SELECT CASE statement 74
SendKeys function 204
services, OLE 90
Sgn function 198
Sin function 202
single precision numbers 50
Space function 228
Sqr function 199
statement
  Beep 203
  ChDir 162
  ChDrive 163
  Close 164
  Const 137
  Dim 103, 139
  Erase 104
  Get 171
  Kill 176
  Line Input # 177, 223
  MkDir 180
  Name 181
  Open 182
  Option Base 106, 143
  Option Explicit 226
  ReDim 107, 145
  Rem 204
  RmDir 190
statements 30
  END 76
  EXIT 76
  FOR 71
  GOTO 70
  IF 72
  OnError 77
  option 34
  option explicit 34
  SELECT CASE 74
  STOP 77
  WITH 78
static variable scope 28
STOP statement 77
Str function 134
StrCompare function 229
string comparison 68
string concatenation 68
String function 230
string functions 217
strings 67
structures, control 70
Sub statement 213
subroutines 33, 78-79
subscripts 44

## T

Tan function 202
Time Cicode function 156
time functions 148
time values 61
Timer event 157
TimeSerial function 124, 158
TimeValue function 158
to clause within array subscripts 44
trigonometry functions 194
Trim function 230

## U

Ubound function 108
UCase function 231
underscore character 32
upper bound 44

## V

Val function 134
values, date 61
values, time 61
variable declaration 40
variable initialization 42
variable naming 33
variables 33, 40
  lifetime 28
  scope 28
variant data type 48

variant variables 48
VarType function 147

## W

WeekDay function 159
WHILE statement 71
WITH statement 78
Write # function 192

## Y

Year function 160